

Masterarbeit

Entwicklung und Implementierung eines Backends für den innovativen Desktop-as-a-Service DESIGN

Eingereicht durch: **Zakaria Jouahri**
Matrikelnummer: **1246478**
Erstprüfer: **Prof. Dr. Christian Baun**
Zweitprüfer: **Prof. Dr. Matthias Deegener**
Startdatum: **22.06.2026**
Abgabedatum: **25.11.2026**

Eidesstattliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe. Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht. Die Zeichnungen oder Abbildungen dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellennachweis versehen. Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

_____

Datum, Unterschrift des Studierenden

Inhalt

Inhalt **II**

Abbildungsverzeichnis **IV**

Codeverzeichnis **V**

Tabellenverzeichnis **VI**

1 Einführung **1**

1.1 Live REPLY GmbH 1

1.2 Problemstellung 2

2 Technische Grundlagen **3**

2.1 Software-Defined Networking (SDN) 3
2.1.1 OpenFlow-Protokoll 5
2.1.2 SDN-Controller 5

2.2 Docker-Container 9

2.3 Network Function Virtualization (NFV) 10
2.3.1 NFV Infratructure (NFVI) 12
2.3.2 NFV Management and Orchestration (MANO) 12
2.3.3 Unterschied zwischen VNF und CNF 12

2.4 Kubernetes 13
2.4.1 API Server 14
2.4.2 etcd 15
2.4.3 Scheduler 15
2.4.4 Controller Manager 15
2.4.5 kubelet 15
2.4.6 kube-proxy 15

2.5 Asterisk 16

3 Anforderungsanalyse **17**

3.1 Orchestrierung durch Kubernetes 17

3.2 Die Kommunikation zwischen den Nodes 18

3.3 Anwendungen in der Anwendungsschicht 19

3.4 Die Voraussetzung des Zielzustandes 20

3.5 Der Zeitplan 21

4 Realisierung **22**

4.1 Installation der benötigten Anwendungen 22
4.1.1 Installation von Docker 22
4.1.2 Installation von Kubernetes 23
4.1.3 Installation von OpenvSwitch 25
4.1.4 Installation von ONOS 26

4.2 Erstellung eines Kubernetes-Clusters 28
4.2.1 Initialisierung von Cluster auf der Master-Node 29
4.2.2 Anfügen der Worker-Nodes 30
4.3 Vernetzung zwischen den Nodes 31
4.4 Die Konfigurationen von Asterisk 34
4.5 Anfügen des Asterisk-Containers an Pods 35
4.6 Die Anwendungen in der Anwendungsschicht 37
4.6.1 getInfos.py 37
4.6.2 addInfos.py 39
4.6.3 deleteInfos.py 40
5 Fazit und Ausblick 42
5.1 Fazit 42
5.2 Ausblick 43
6 Abkürzungen 45
7 Referenzen 48
8 Anhang 54
8.1 addInfos.py 54
8.2 getInfos.py 55
8.3 deleteInfos.py 55

Abbildungsverzeichnis

Abbildung 2.1: Architekturübersicht von DESIGN mit den beteiligten Komponenten 6

Abbildung 2.2: Der Unterschied zwischen virtueller Maschine und Container 7

Abbildung 2.3: Vereinfachte interne Architektur von Apache Guacamole 9

Abbildung 2.4: Abstrakter OAuth2-Protokollablauf nach RFC 6749 11

Abbildung 2.5: Authorization-Code-Grant mit PKCE 12

Abbildung 2.6: Client-Credentials-Grant 12

Abbildung 2.7: Schematischer Aufbau eines JSON Web Tokens aus Header 14

Abbildung 2.8: Ablauf der Anfrageprüfung durch den OAuth2-Ressourcenserver 18

Abbildung 2.9: Spring Authorization Server und Resource-Server mit Token-Ausstellung 19

Abbildung 2.10: Grundprinzip von RBAC im Vergleich zu ABAC 21

Abbildung 2.11: Schematischer Entscheidungsbaum einer mehrstufigen Berechtigungskaskade 22

Abbildung 2.12: Maven-Build-Lifecycle mit den Standardphasen 23

Abbildung 2.13: Objektrelationales Mapping zwischen Java-Entity-Klasse und Datenbanktabelle 26

Abbildung 3.1: Anfrageverarbeitung im alten PHP-Backend

Abbildung 3.2: Domänenmodell des alten PHP-Backends

Abbildung 3.3: Zielzustand des neuen Backends

Abbildung 3.4: Login und Berechtigungsprüfung (AF-1) als Sequenzdiagramm

Abbildung 4.1: Swagger-UI: Password Grant über POST /oauth2/user/token

Abbildung 4.2: Swagger-UI: Berechtigungsabfrage über GET /permissions/{functionName}/{userId}

Codeverzeichnis

Code 4.1: Paketstruktur des neuen Backends

Code 4.2: ArchUnit-Regel zur Schichtenarchitektur (gekürzt)

Code 4.3: Grant-Dispatch in TokenService.grantToken

Code 4.4: Schutz vor zeitbasierter Benutzer-Enumeration

Code 4.5: Zählung von Fehlversuchen und Kontosperrung

Code 4.6: Aufbau und Signatur des Zugriffstokens

Code 4.7: Erzeugung des opaken Refresh-Tokens

Code 4.8: Refresh-Token-Rotation in TokenService.refreshGrant

Code 4.9: RateLimitingFilter (gekürzt)

Code 4.10: Sperrenfreie Aktualisierung in TokenBucket.tryConsume (gekürzt)

Code 4.11: Vierstufige Berechtigungskaskade in PermissionService.checkPermission

Code 4.12: Konfliktauflösung innerhalb einer Kaskadenstufe

Code 4.13: Fail-closed-Behandlung ungültiger Eingaben

Code 4.14: Bedarfsgesteuertes Laden der Gruppenzugehörigkeit

Code 4.15: Erzeugung des Registrierungscodes

Code 4.16: E-Mail-Versand nach erfolgreichem Commit

Code 4.17: Prüfung des Registrierungscodes in UserService.validateEmail

Code 4.18: Invalidierung aller Sitzungen bei Passwortänderung und Deaktivierung

Code 4.19: Anfrage einer Anwendungsfreischaltung in UserService.requestApplication

Code 4.20: Anlage eines Standard-Administrators beim Start (gekürzt)

Code 4.21: Anlegen eines Desktops mit Untergruppen

Code 4.22: Verknüpfung von Nutzergruppe und Desktop-Gruppe

Code 4.23: Konfiguration der zustandslosen Sicherheitsfilterkette (gekürzt)

Code 4.24: Zentrale Passwortrichtlinie

Code 4.25: OpenAPI-Konfiguration mit Bearer-Sicherheitsschema

Code 4.26: Mehrstufiges Dockerfile (gekürzt)

Code 4.27: Umgebungsvariablen der Demonstrationsumgebung (Auszug aus .env.example)

Code 4.28: JaCoCo-Mindestabdeckung in der pom.xml (gekürzt)

Tabellenverzeichnis

Tabelle 3.1: Technologie-Stack des alten PHP-Backends

Tabelle 3.2: API-Endpunkte des alten PHP-Backends (Ist-Zustand)

Tabelle 3.3: Umfeld und Rahmenbedingungen

Tabelle 3.4: Funktionale Anforderungen (FA)

Tabelle 3.5: Nutzung der Altschnittstelle durch Frontend und DaaS-Anwendung

Tabelle 3.6: Nicht-funktionale Anforderungen (NFA)

Tabelle 3.7: Zeitplan der Arbeit

Tabelle 3.8: Anwendungsfälle für den Prototyp

Tabelle 4.1: Zentrale Architekturentscheidungen und ihr Anforderungsbezug

Tabelle 4.2: Testsuite nach Art

Tabelle 4.3: JaCoCo-Testabdeckung, Gesamtprojekt

Tabelle 4.4: Qualitätssicherung, altes Backend (PHP) vs. neues Backend (Java)

Tabelle 5.1: Erfüllungsgrad der funktionalen Anforderungen

Tabelle 5.2: Erfüllungsgrad der nicht-funktionalen Anforderungen

Tabelle A.1: Endpunkte des neuen Backends (Version 2.0.0)

1 Einführung

Cloud- und browserbasierte Arbeitsumgebungen gewinnen zunehmend an Bedeutung, weil sie einen geräteunabhängigen Zugriff auf Anwendungen ermöglichen, ohne dass eine lokale Installation nötig ist. Das System DESIGN verfolgt mit seinem Desktop-as-a-Service-Ansatz (DaaS) das Ziel, die Vorteile lokal installierter Anwendungen (Datenschutz, Unterstützung von Legacy-Software, kein Vendor-Lock-in) mit denen webbasierter Software (geräteunabhängiger Zugriff über den Browser, keine Installation) zu verbinden (Baun & Bouché, 2023). DESIGN gliedert sich in drei eigenständige Teile: die **DaaS-Anwendung** (Python, verwaltet Container, virtuelle Maschinen, Guacamole und Proxmox), das **Frontend** (JavaScript, React) und das **Backend** (Baun & Bouché, 2024).

1.1 Motivation

Laut Aufgabenstellung dieser Arbeit (Baun, 2026) ist das bestehende Backend von DESIGN in **PHP** implementiert und dabei „qualitativ schwach“ sowie „schlecht dokumentiert“. Das erschwert Wartung und Weiterentwicklung des Gesamtsystems, obwohl die beiden anderen Teile, DaaS-Anwendung und Frontend, nach gleicher Quelle qualitativ gut, gut dokumentiert und vergleichsweise einfach zu installieren sind. Das Backend bildet jedoch die Grundlage für Authentifizierung, Autorisierung sowie Benutzer- und Desktop-Verwaltung des gesamten Systems. Fehler oder eine mangelnde Nachvollziehbarkeit an dieser Stelle wirken sich daher auf alle anderen Systemteile aus.

1.2 Problemstellung

Aus der Ausschreibung ergibt sich die konkrete Aufgabe, ein neues Backend für DESIGN zu entwickeln und zu implementieren, wahlweise in Python oder einer anderen modernen Programmiersprache, das die bestehende Funktionalität des alten PHP-Backends ersetzt, ohne dabei Frontend oder DaaS-Anwendung zu verändern. Da beide bestehenden Komponenten

nicht Teil dieser Arbeit sind, muss das neue Backend deren bisherige Schnittstellenerwartungen (Endpunkte, Feldformate, Verhalten der Berechtigungsprüfung) unverändert erfüllen. Das setzt voraus, dass das Verhalten des alten Backends zunächst hinreichend verstanden und dokumentiert wird, bevor es neu implementiert werden kann. Diese Aufgabe wird durch die mangelhafte Dokumentation des Ist-Systems zusätzlich erschwert.

1.3 Zielsetzung

Ziel dieser Arbeit ist die Entwicklung und Implementierung eines neuen Backends für DESIGN auf Basis von **Java 21** und **Spring Boot**. Das neue Backend soll funktional äquivalent zum alten PHP-Backend sein und dessen zentrale Verhaltenseigenschaften, insbesondere die mehrstufige Berechtigungskaskade, beibehalten. Gleichzeitig sollen Wartbarkeit und Nachvollziehbarkeit gegenüber dem Ist-Zustand verbessert werden, unter anderem durch automatisiert geprüfte Architekturregeln und eine Mindest-Testabdeckung.

1.4 Forschungsfrage

Aus der Aufgabenstellung und der beschriebenen Problemstellung ergibt sich die folgende Forschungsfrage:

„Wie lässt sich das Backend des Desktop-as-a-Service-Systems DESIGN von PHP nach Java und Spring Boot migrieren, sodass Frontend und DaaS-Anwendung ohne Änderungen an ihrer fachlichen Logik funktionsfähig bleiben, und welche architektonischen Maßnahmen verbessern dabei die Wartbarkeit des Backends gegenüber dem bisherigen Stand?“

Diese Frage wird in zwei Teilen beantwortet. Kapitel 3 analysiert das bestehende PHP-Backend und leitet daraus die funktionalen und nicht-funktionalen Anforderungen an das neue Backend ab. Kapitel 4 beschreibt, wie diese Anforderungen in Java und Spring Boot umgesetzt wurden und welche architektonischen Maßnahmen (zum Beispiel automatisiert

geprüfte Architekturregeln und eine Mindest-Testabdeckung) zur besseren Wartbarkeit beitragen.

1.5 Abgrenzung

Diese Arbeit beschränkt sich auf das **Backend**-Repository (*design-daas-backend-java*). Frontend (*design-daas-application-frontend*) und DaaS-Anwendung beziehungsweise Engine (*design-daas*) werden nicht verändert. Ihre bestehenden Schnittstellenverträge gelten als gegebene Randbedingung. Komponenten, die laut Recherche (Kapitel 2) zur DaaS-Anwendung oder zur Infrastrukturebene von DESIGN gehören, etwa Apache Guacamole, Proxmox VE oder der Container- und VM-Lebenszyklus, sind nicht Gegenstand der Implementierung dieser Arbeit und werden nur insoweit behandelt, wie sie zur Einordnung des Backends in das Gesamtsystem nötig sind (Kapitel 2.1, 3.1).

Abweichend von diesem Grundsatz waren an beiden Clients kleine Anpassungen notwendig, die ausschließlich die Anbindung an das Backend betreffen. Im Frontend wurden das Aktivieren und Deaktivieren von Nutzerkonten auf die HTTP-Methode PATCH umgestellt, ein Logout über den neuen Endpunkt POST /oauth2/user/logout ergänzt und einzelne fehlerhafte oder unübersichtlich strukturierte Stellen bereinigt, soweit dies für eine korrekte Anbindung nötig war. In der DaaS-Anwendung wurde die Auswertung der Session-Antwort um das Feld role ergänzt und eine nicht mehr benötigte Abfrage von POST /permissions_info entfernt. Eine darüber hinausgehende Überarbeitung, Refaktorierung oder funktionale Erweiterung der beiden Clients ist nicht Ziel dieser Arbeit.

1.6 Methodisches Vorgehen

Das Vorgehen orientiert sich an den in der Ausschreibung genannten Arbeitspaketen **AP1–AP6**: Recherche des DESIGN-Gesamtsystems (AP1), Analyse des alten PHP-Backends (AP2), Entwurf (AP3) und Implementierung (AP4) des neuen Backends, Aufbau eines Demonstrators (AP5) sowie Test, Evaluation und Dokumentation (AP6). Kapitel 3 leitet aus AP1 und AP2 die funktionalen und nicht-funktionalen Anforderungen ab. Kapitel 4

beschreibt die Ergebnisse von AP3 und AP4, den Demonstrator aus AP5 (Abschnitt 4.7) sowie Test und Evaluation aus AP6 (Abschnitt 4.8). Methodisch handelt es sich um eine Reverse-Engineering-Anforderungsanalyse mit anschließender Neuimplementierung; bewertet wird das Ergebnis über einen systematischen Anforderungsabgleich, automatisierte Tests und einen Demonstrator (Kapitel 5).

1.7 Aufbau der Arbeit

Kapitel 2 stellt die theoretischen Grundlagen zum DaaS-Konzept von DESIGN, zu OAuth2/JWT-basierter Authentifizierung, zu Autorisierungsmodellen sowie zu den verwendeten Spring-Technologien dar. Kapitel 3 analysiert Ist- und Zielzustand des Backends und leitet daraus funktionale sowie nicht-funktionale Anforderungen ab. Kapitel 4 beschreibt die Realisierung der einzelnen Backend-Module (Authentifizierung, Autorisierung, Benutzerverwaltung, Desktop-Inventar) sowie die querschnittlichen Sicherheits- und Qualitätsmaßnahmen, den Demonstrator und die Evaluation. Kapitel 5 fasst die Ergebnisse zusammen, beantwortet die Forschungsfrage, gleicht die Ergebnisse mit den Anforderungen aus Kapitel 3 ab, reflektiert die Grenzen der Arbeit und gibt einen Ausblick auf mögliche Weiterentwicklungen.

2 Technische Grundlagen

Dieses Kapitel beschreibt die theoretischen Grundlagen, Standards, Bibliotheken und Werkzeuge, auf denen das in dieser Arbeit entwickelte Backend aufbaut. Zunächst wird in 2.1 das Gesamtsystem DESIGN und die darin verwendete Desktop-as-a-Service-Architektur vorgestellt, einschließlich der zugrundeliegenden Virtualisierungskonzepte (Hypervisoren, Container) und der internen Funktionsweise von Apache Guacamole. Anschließend werden drei Themenfelder behandelt: die Authentifizierung über OAuth2, JSON Web Token, RS256, Transport Layer Security und Passwort-Hashing mit bcrypt (2.2), die Autorisierung über Zugriffskontrollmodelle wie ACL, RBAC und ABAC sowie kaskadierende Entscheidungsmodelle (2.3) sowie Werkzeuge aus dem Java-Ökosystem für Build-Automatisierung, Anwendungsentwicklung, Persistenz, Tests sowie Architektur- und Qualitätssicherung, u. a. Maven, das Spring-Framework/Spring Boot, Hibernate/JPA, MariaDB, JUnit 5/Mockito, ArchUnit, Flyway und JaCoCo (2.4).

2.1 DaaS und die Architektur von DESIGN

Desktop-as-a-Service (DaaS) bezeichnet die Bereitstellung vollständiger Arbeitsumgebungen über das Netzwerk, wobei Anwendungen nicht lokal installiert, sondern auf entfernten Ressourcen ausgeführt und über einen Client dargestellt werden. Im Gegensatz zu klassischer lokaler Software bietet dieser Ansatz einfache Fernzugriffsmöglichkeiten und zentrale Administration, bringt jedoch typische Nachteile von Software-as-a-Service (SaaS) mit sich, etwa Abhängigkeit von einer Netzwerkverbindung und potenzielle Datenschutz- und Sicherheitsbedenken (Baun & Bouché, 2024).

Das System **DESIGN** verfolgt das Ziel, die Vorteile lokal installierter Anwendungen (Datenschutz, Unterstützung von Legacy-Software, kein Vendor-Lock-in) mit denen webbasierter Software (geräteunabhängiger Zugriff über den Browser, keine Installation) zu

verbinden, ohne deren jeweilige Nachteile zu übernehmen (Baun & Bouché, 2023). Um unveränderte Linux- und Windows-Anwendungen bereitzustellen, nutzt DESIGN Containervirtualisierung (Docker) sowie virtuelle Maschinen (KVM, verwaltet über die Virtualisierungsplattform Proxmox VE); die Fernsteuerung der jeweiligen Anwendung erfolgt über VNC- bzw. RDP-Dienste innerhalb des Containers/der VM, die durch das Gateway **Apache Guacamole** in einen Web-Stream übersetzt und so im Browser des Nutzers dargestellt werden (Baun & Bouché, 2024). Dieser Architekturansatz erlaubt drei Bereitstellungsvarianten: (1) Linux-Anwendungen in Linux-Containern, (2) Windows-Anwendungen in Linux-Containern mittels *Wine*, sofern dies funktioniert, und (3) Windows-Anwendungen in Windows-VMs, falls *Wine* nicht einsetzbar ist (Baun & Bouché, 2023).

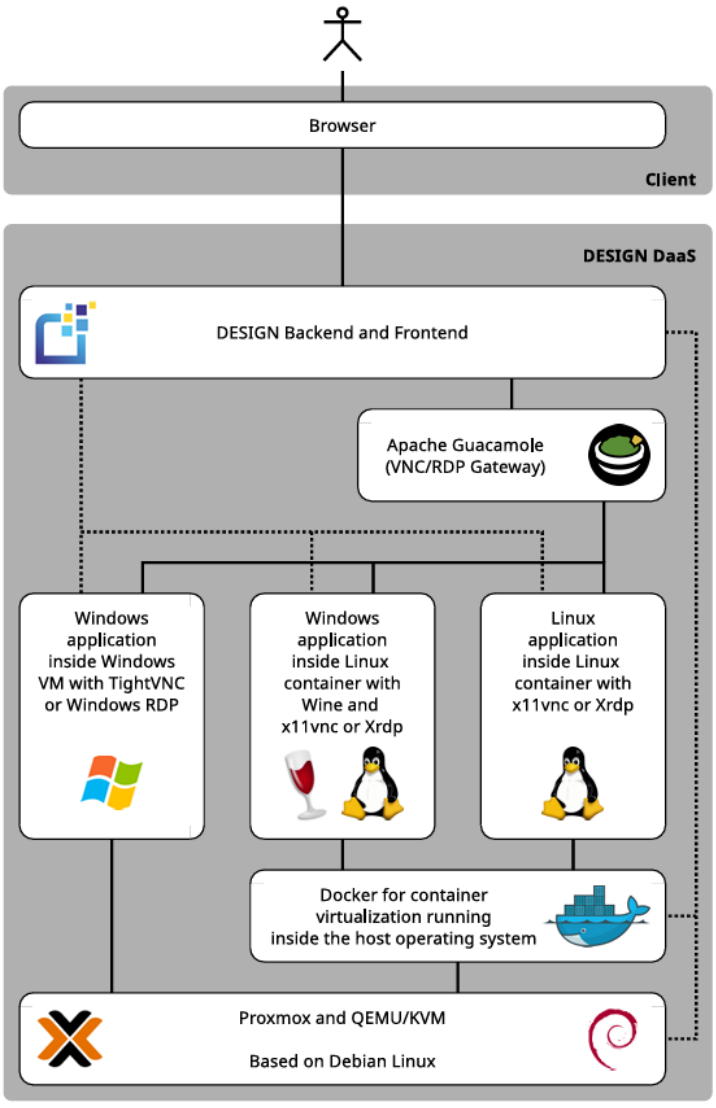


Abbildung 2.1: Architekturübersicht von DESIGN mit den beteiligten Komponenten (Baun & Bouché, 2024)

Abbildung 2.1 fasst Proxmox VE/KVM, Docker und Apache Guacamole als Komponenten der DESIGN-Architektur zusammen; die beiden folgenden Unterabschnitte vertiefen die technischen Grundlagen dieser Komponenten.

2.1.1 Virtualisierungsarten: Hypervisoren und Container

Die beiden in DESIGN eingesetzten Virtualisierungsformen beruhen auf unterschiedlichen technischen Grundprinzipien. Bei der klassischen Hardware-Virtualisierung unterscheidet die grundlegende Arbeit von Popek und Goldberg zwischen zwei Bauarten eines *Hypervisors*: Ein **Typ-1-Hypervisor** (auch *Bare-Metal-Hypervisor* genannt) läuft direkt auf der physischen Hardware ohne zwischengeschaltetes Betriebssystem und verwaltet selbst die virtuellen Maschinen, während ein **Typ-2-Hypervisor** als gewöhnliche Anwendung innerhalb eines bestehenden Host-Betriebssystems ausgeführt wird (Popek & Goldberg, 1974). KVM (Kernel-based Virtual Machine), auf dem Proxmox VE aufbaut, verwandelt den Linux-Kernel selbst in einen Typ-1-Hypervisor, indem es dessen Prozessor- und Speicherverwaltung um Virtualisierungserweiterungen der CPU ergänzt; jede virtuelle Maschine erhält dadurch ein vollständig eigenes Betriebssystem mit eigenem Kernel, eigenem Dateisystem und eigenen Gerätetreibern, was hohe Isolation, aber auch höheren Ressourcenverbrauch und längere Start- und Bereitstellungszeiten mit sich bringt.

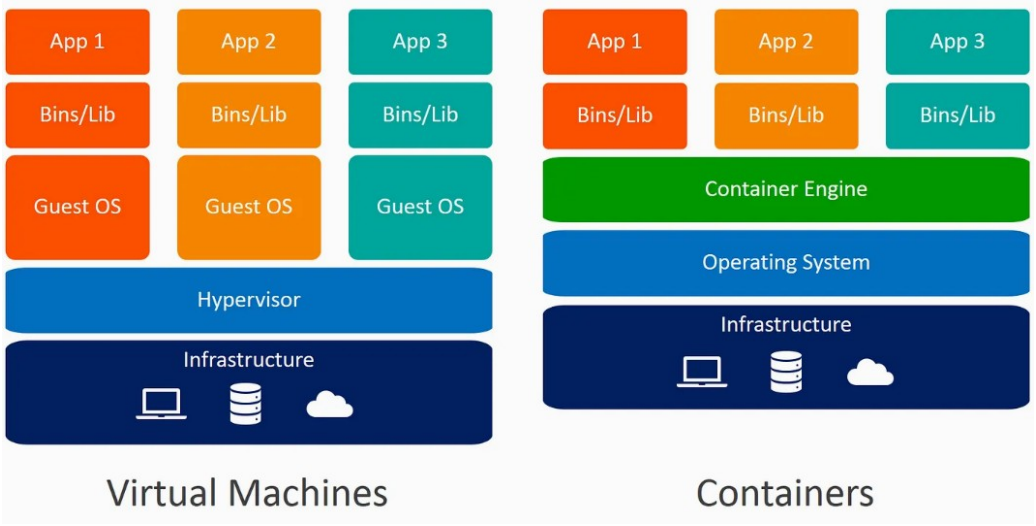


Abbildung 2.2: Der Unterschied zwischen virtueller Maschine und Container (The Coding Gopher, 2026)

Containervirtualisierung mit **Docker** verfolgt demgegenüber einen grundsätzlich anderen Ansatz: Statt ein eigenes Betriebssystem zu virtualisieren, teilen sich alle Container denselben Host-Betriebssystem-Kernel und werden lediglich durch zwei Linux-Kernel-Mechanismen voneinander isoliert: *Namespaces* kapseln pro Container jeweils eigene Sichten auf Prozess-IDs, Netzwerkschnittstellen, Dateisystem-Mountpunkte und Hostnamen, sodass ein Prozess innerhalb eines Containers glaubt, exklusiven Zugriff auf diese Ressourcen zu haben, während *Control Groups* (cgroups) begrenzen, wie viel CPU-Zeit, Arbeitsspeicher und I/O-Bandbreite
ein Container maximal verbrauchen darf (Docker Inc., 2023). Eine Docker-Anwendung wird zudem als *Image* ausgeliefert, ein schreibgeschütztes, in übereinanderliegenden Schichten (*Layers*) aufgebautes Dateisystem-Archiv, aus dem zur Laufzeit ein beschreibbarer Container-Prozess gestartet wird. Weil dabei kein vollständiges Betriebssystem gebootet werden muss, lassen sich Container in der Regel in Sekunden statt Minuten starten und klonen, was die von Baun und Bouché gemessenen Zeitunterschiede zwischen Docker-Containern und Windows-VMs erklärt (Baun & Bouché, 2024).

2.1.2 Apache Guacamole: clientloses Remote-Desktop-Gateway

Apache Guacamole ist ein quelloffenes Remote-Desktop-Gateway, über das DESIGN die in Containern und virtuellen Maschinen laufenden Anwendungen im Browser darstellt. Technisch besteht es aus zwei Hauptkomponenten: dem *guacamole-client*, einer Java-Webanwendung, die im Browser über HTML5 und *WebSockets* eine grafische Oberfläche bereitstellt, und *guacd*, einem nativen Hintergrunddienst, der die eigentliche Übersetzung zwischen Remote-Desktop-Protokollen (VNC, RDP, SSH) und dem schlanken, textbasierten *Guacamole-Protokoll* vornimmt (Apache Software Foundation, 2025). Der Browser spricht dabei zu keinem Zeitpunkt direkt VNC oder RDP; stattdessen sendet *guacd* die Anzeige des entfernten Containers bzw. der virtuellen Maschine als eine Folge einfacher Zeichenbefehle (z. B. "zeichne dieses Bildfragment an Position x,y") über eine WebSocket-Verbindung an den Browser, der daraus mittels eines HTML5-Canvas-Elements das Bild zusammensetzt. Dieser Ansatz erklärt, warum Guacamole als *clientlos* bezeichnet wird: Es genügt ein

gewöhnlicher Webbrowser, ohne dass Plugins, Java-Applets oder native RDP-/VNC-Clients installiert werden müssen.

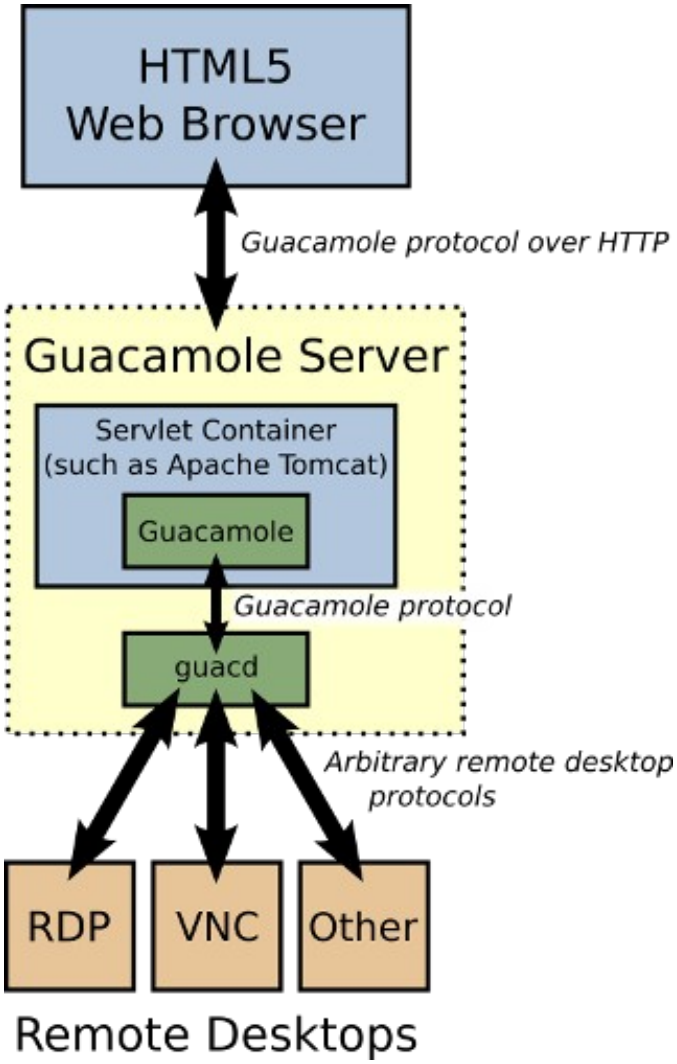


Abbildung 2.3: Vereinfachte interne Architektur von Apache Guacamole (Apache Software Foundation, 2025).

2.2 Authentifizierung mit OAuth2 und JSON Web Token

Die Authentifizierung über OAuth2 und JSON Web Token beruht auf drei Standards, die unterschiedliche Aufgaben übernehmen und erst im Zusammenspiel das vollständige Bild

ergeben. OAuth 2.0 *(RFC 6749)* legt fest, ***wer*** unter welchen Bedingungen ein Zugriffstoken bekommen darf, und definiert dafür Rollen und Abläufe, sagt aber nichts darüber aus, wie dieses Token konkret aussieht (Hardt, 2012). JSON Web Token *(JWT, RFC 7519)* ist genau dieses konkrete Tokenformat: ein kompaktes, textbasiertes Format, mit dem sich Ansprüche *(Claims)* über einen Nutzer transportieren lassen (Jones et al., 2015a). RS256 schließlich ist kein eigener Standard, sondern ein in RFC 7518 definierter Signaturalgorithmus, der festlegt, wie ein solches JWT gegen Manipulation abgesichert wird (Jones, 2015a). Die folgenden Unterabschnitte erläutern diese Bausteine einzeln und beschreiben anschließend in 2.2.5, wie Spring Security sie technisch zu einem vollständigen Authentifizierungs- und Autorisierungsablauf zusammensetzt.

2.2.1 OAuth2: Rollen, Protokollablauf und Grant-Typen

RFC 6749 unterscheidet vier Rollen: den ***Resource Owner*** (die Person, der die geschützten Daten gehören, also der Nutzer eines Systems), den ***Client*** (die Anwendung, die stellvertretend für den Nutzer auf eine Ressource zugreifen möchte, z. B. eine Frontend-Anwendung), den ***Authorization Server*** (die Stelle, die nach erfolgreicher Prüfung der Zugangsdaten ein Zugriffstoken ausstellt) und den ***Resource Server*** (die Stelle, die dieses Token entgegennimmt, prüft und daraufhin die eigentlich angefragte Ressource liefert) (Hardt, 2012). In vielen OAuth2-Systemen sind *Authorization Server* und *Resource Server* getrennte, eigenständige Dienste, zum Beispiel wenn sich eine Anwendung über einen externen Anbieter wie Google oder Microsoft anmeldet. In sogenannten Single-Vendor-Systemen, in denen Client und die eigenen APIs von derselben Partei betrieben werden, ist es dagegen üblich, dass ein und derselbe Dienst beide Rollen gleichzeitig übernimmt: Er prüft die Zugangsdaten und stellt das Token aus (*Authorization Server*) und validiert dieses Token anschließend bei jeder nachfolgenden API-Anfrage, um die eigenen Endpunkte zu schützen (*Resource Server*).

Der abstrakte Protokollablauf nach RFC 6749 besteht aus vier Schritten: Der Client fordert vom Ressourceninhaber eine Autorisierung an, erhält im Erfolgsfall einen ***Authorization Grant*** (einen Nachweis dieser Zustimmung), tauscht diesen Grant beim *Authorization Server* gegen ein Zugriffstoken ein und verwendet dieses Token schließlich beim *Resource Server*,

um auf die geschützte Ressource zuzugreifen (Hardt, 2012). RFC 6749 definiert für den zweiten Schritt mehrere *Grant-Typen*, von denen die wichtigsten sind: der ***Authorization-Code-Grant***, bei dem der Nutzer über eine Weiterleitung auf einer separaten Login-Seite des *Authorization Servers* seine Zugangsdaten eingibt, ohne dass der Client sie je zu sehen bekommt (das Standardverfahren für Anwendungen Dritter); der ***Client-Credentials-Grant***, bei dem kein Nutzer beteiligt ist und eine Anwendung im eigenen Namen auf eine Ressource zugreift (etwa für Server-zu-Server-Kommunikation); und der ***Resource Owner Password Credentials Grant*** (ROPC), bei dem der Client Benutzername und Passwort direkt entgegennimmt und im Austausch gegen ein Token an den Authorization Server weiterreicht (Hardt, 2012). Ergänzend beschreibt RFC 6749 den ***Refresh-Token***-Mechanismus, mit dem ein Client ein abgelaufenes Zugriffstoken erneuern kann, ohne dass der Nutzer seine Zugangsdaten erneut eingeben muss (Hardt, 2012).

Abbildung 2.4 zeigt diesen abstrakten Ablauf am Beispiel des ROPC-Grants als konkreter Ausprägung des zweiten Schritts.

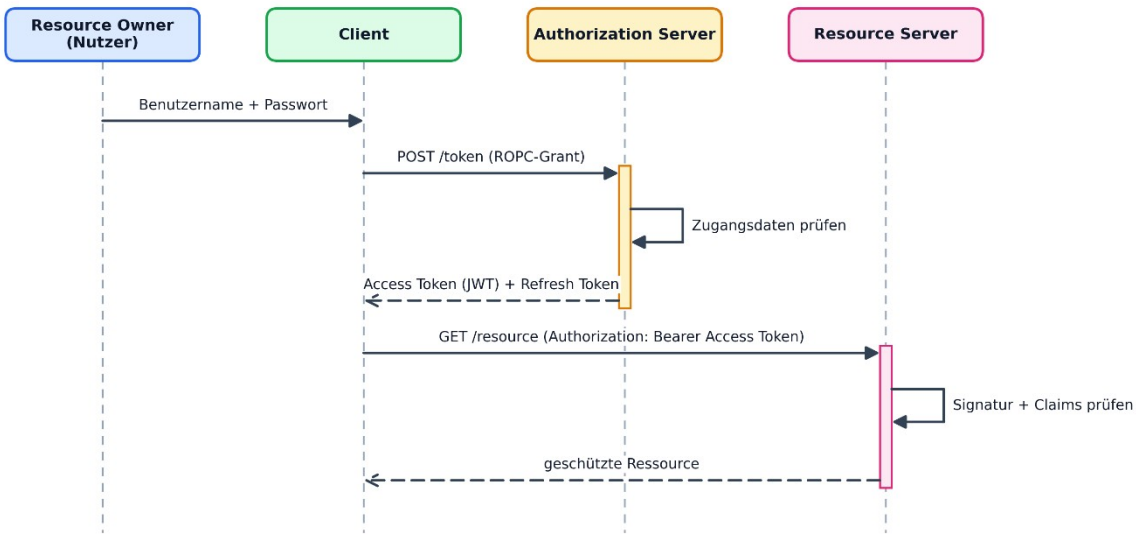


Abbildung 2.4: Abstrakter OAuth2-Protokollablauf nach RFC 6749 am Beispiel des ROPC-Grants (eigene Darstellung nach Hardt, 2012)

Die neuere IETF-Sicherheitsempfehlung RFC 9700 (*Best Current Practice for OAuth 2.0 Security*) empfiehlt, auf den ROPC-Grant in neuen Systemen möglichst zu verzichten, unter anderem, weil er sich schlecht mit Mehr-Faktor-Authentifizierung verträgt und den Client dazu zwingt, mit Klartext-Zugangsdaten umzugehen (Lodderstedt et al., 2025). Als sicherere Alternative gilt der Authorization-Code-Grant in Kombination mit der Erweiterung ***Proof Key for Code Exchange*** (PKCE, RFC 7636) (Sakimura et al., 2015): Der Client erzeugt dabei vor der Weiterleitung zufällig einen geheimen Wert (*code_verifier*) und übermittelt nur dessen gehashten Wert (*code_challenge*) an den *Authorization Server*; beim späteren Eintausch des Autorisierungscodes muss der Client den ursprünglichen *code_verifier* erneut mitsenden, den der *Authorization Server* gegen die zuvor übermittelte *code_challenge* prüft. Ein Angreifer, der lediglich den Autorisierungscode abfängt, kann diesen damit nicht mehr gegen ein Token eintauschen, weil ihm der *code_verifier* fehlt. Abbildung 2.5 stellt diesen Ablauf dem clientlosen Client-Credentials-Grant gegenüber, der ganz ohne Resource Owner auskommt.

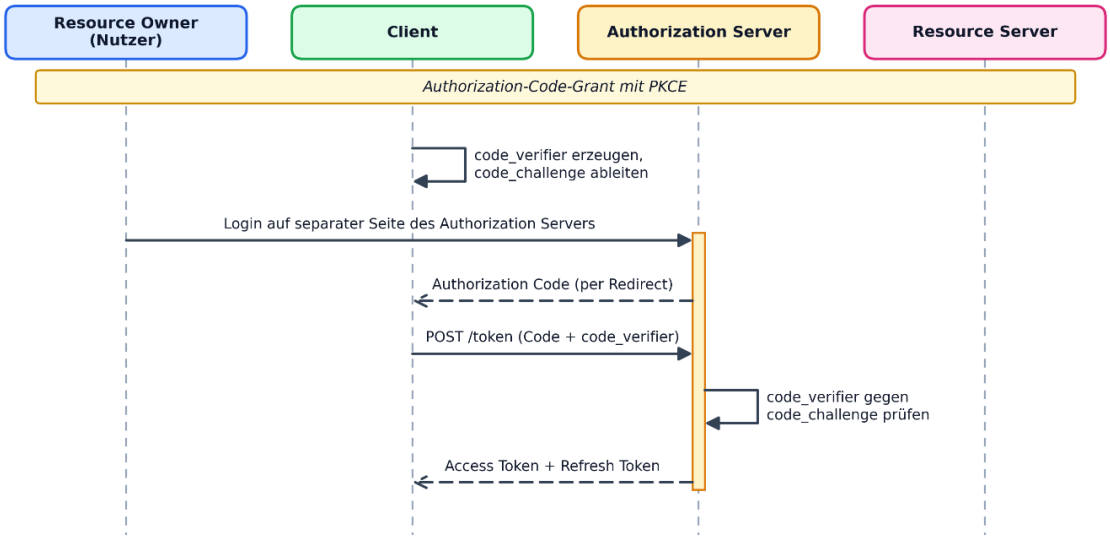


Abbildung 2.5: Authorization-Code-Grant mit PKCE (eigene Darstellung nach Sakimura et al., 2015)

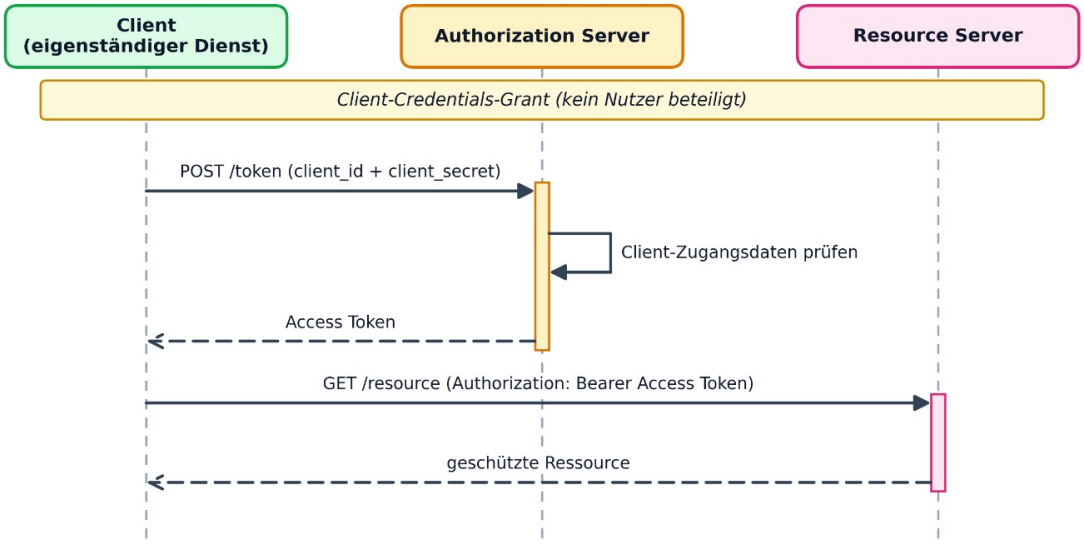


Abbildung 2.6: Client-Credentials-Grant (eigene Darstellung nach Hardt, 2012)

Die Rückgabe und Verwendung des ausgestellten Tokens sind in RFC 6750 als sogenanntes Bearer-Token beschrieben: Wer das Token besitzt, kann damit auf die Ressource zugreifen, unabhängig davon, ob er der ursprüngliche Nutzer ist, weshalb die Übertragung ausschließlich über TLS erfolgen sollte (Jones & Hardt, 2012). Transport Layer Security (TLS) ist ein kryptografisches Protokoll, das eine Netzwerkverbindung durch eine Kombination aus asymmetrischer Schlüsselaushandlung und anschließender symmetrischer Verschlüsselung gegen Mitlesen und Manipulation absichert und die Identität des Servers über ein Zertifikat bestätigt (Rescorla, 2018); im Web wird TLS in Kombination mit HTTP als HTTPS bezeichnet. Da ein Bearer-Token nicht an eine bestimmte Verbindung gebunden ist, würde ein Angreifer, der ein Token auf einer unverschlüsselten Verbindung abfängt, damit uneingeschränkt auf die Ressource zugreifen können; TLS ist daher für den produktiven Einsatz von Bearer-Token obligatorisch. Der Client überträgt das Token bei jeder Anfrage im HTTP-Header ***Authorization: Bearer <token>;*** der *Resource Server* muss daraus im Umkehrschluss keine eigene Sitzung führen, sondern kann die Anfrage allein anhand des mitgesendeten Tokens beurteilen.

Da OAuth2 selbst keine Aussage über die Identität des angemeldeten Nutzers trifft, sondern nur über dessen Zugriffsrechte, wurde darauf aufbauend der Standard OpenID Connect

(OIDC) entwickelt: OIDC ergänzt OAuth2 um einen zusätzlichen, standardisierten ID-Token (selbst ein JWT) mit Identitätsangaben zum Nutzer sowie um einen /userinfo-Endpunkt, über den weitere Profildaten abgerufen werden können (OpenID Foundation, 2023). OAuth2 allein beantwortet damit die Frage "Darf dieser Client auf diese Ressource zugreifen?", während OIDC zusätzlich die Frage "Wer ist der angemeldete Nutzer?" beantwortet.

2.2.2 JSON Web Token: Aufbau und Claims

Ein JWT ist, unabhängig vom OAuth2-Kontext, zunächst nur ein Format, um eine Menge von Ansprüchen (*Claims*) kompakt und überprüfbar zu kodieren (Jones et al., 2015a). Ein JWT besteht aus drei durch Punkte getrennten, jeweils Base64url-kodierten Teilen: einem Header, der unter anderem den Signaturalgorithmus benennt, einer Payload mit den eigentlichen Claims und einer Signatur über die ersten beiden Teile (Jones et al., 2015a). Abbildung 2.7 zeigt diesen Aufbau schematisch.

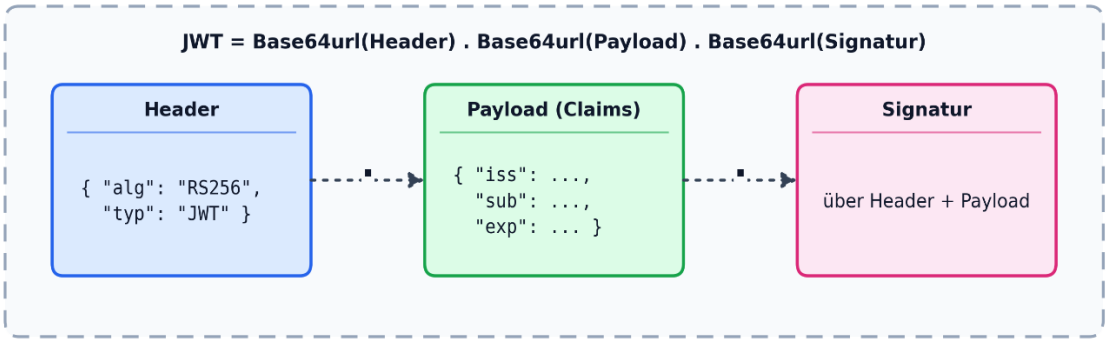


Abbildung 2.7: Schematischer Aufbau eines JSON Web Tokens aus Header, Payload und Signatur (eigene Darstellung nach Jones et al., 2015a).

RFC 7519 unterscheidet dabei registrierte Claims mit festgelegter Bedeutung, etwa ***iss*** (Aussteller), ***jti*** (eine eindeutige Token-ID), ***exp*** (Ablaufzeitpunkt), ***sub*** (Subjekt, meist die Nutzer-ID), und ***iat*** (Ausstellungszeitpunkt), von frei definierbaren privaten Claims, die eine Anwendung nach eigenem Bedarf ergänzen kann (Jones et al., 2015a). In der Praxis werden häufig beide Kategorien kombiniert: Ein Zugriffstoken enthält dann sowohl die registrierten Claims (*iss, sub, iat, exp, jti*) als auch anwendungsspezifische private Claims, über die beispielsweise eine Autorisierungsentscheidung getroffen werden kann (siehe 2.3).

Ein wichtiger Aspekt betrifft die Zustandslosigkeit von JWTs: Da ein signiertes JWT allein anhand seiner Signatur als gültig erkannt wird, lässt es sich vor Ablauf der *exp*-Zeit nicht ohne Weiteres widerrufen. Für kurzlebige Zugriffstoken wird dieses Verhalten häufig in Kauf genommen, weil eine kurze Gültigkeitsdauer das Risiko begrenzt und sich durch eine zusätzliche Widerrufsprüfung (siehe 2.2.5) weiter abfedern lässt. Für langlebigere, sicherheitskritische Token wie ein Refresh-Token wird dagegen häufig bewusst kein JWT verwendet, sondern ein zufällig erzeugter, undurchsichtiger Wert, der serverseitig in einer Datenbanktabelle gespeichert und bei Bedarf (z. B. beim Abmelden oder Sperren eines Nutzers) gezielt gelöscht werden kann. Dieser Kontrast verdeutlicht eine grundsätzliche Abwägung beim Einsatz von JWTs: Ein JWT eignet sich gut für kurzlebige, häufig geprüfte Token, bei denen der Verzicht auf eine Datenbankabfrage einen echten Performancevorteil bringt, während ein langlebiges, sicherheitskritisches Token von der serverseitigen Kontrollierbarkeit eines klassischen, zustandsbehafteten Tokens profitiert.

2.2.3 Signaturverfahren: RS256 statt HS256

Damit ein JWT gegen Manipulation geschützt ist, muss es signiert werden. Das Format und die zulässigen Algorithmen für diese Signatur sind in zwei weiteren RFCs geregelt: RFC 7515 ***(JSON Web Signature, JWS***) beschreibt den strukturellen Aufbau eines signierten Tokens aus Header, Payload und Signatur (Jones et al., 2015b), und RFC 7518 ***(JSON Web Algorithms, JWA***) definiert die konkreten Signaturalgorithmen, unter anderem HS256 und RS256 (Jones, 2015a). Der naheliegendste Algorithmus, HS256 (HMAC mit SHA-256), ist symmetrisch: Aussteller und Prüfer eines Tokens benötigen denselben geheimen Schlüssel. In einem System mit mehreren Diensten, die Token nur lesen und prüfen müssen, aber selbst kein Token ausstellen dürfen, ist das ein Problem, denn jeder Dienst, der ein Token verifizieren können soll, müsste das geheime Schlüsselmaterial erhalten, mit dem sich auch beliebige neue Token erzeugen ließen.

RS256 (*RSASSA-PKCS1-v1_5 mit SHA-256*) löst dieses Problem, weil es ein asymmetrisches Verfahren ist: Die Signatur wird mit einem privaten RSA-Schlüssel erzeugt, die Prüfung

erfolgt dagegen mit dem zugehörigen öffentlichen Schlüssel. Der private Schlüssel bleibt ausschließlich bei der ausstellenden Stelle, während der öffentliche Schlüssel beliebig verteilt werden kann, zum Beispiel über einen JWK-Set-Endpunkt, ohne dass dadurch die Möglichkeit entsteht, selbst gültige Token zu fälschen. In einer Systemlandschaft mit mehreren Diensten bedeutet das: Nur die ausstellende Stelle besitzt den privaten Schlüssel und darf Token erzeugen, während alle anderen Dienste Token ausschließlich mit dem öffentlichen Schlüssel prüfen können. Diese Trennung ist der Hauptgrund, warum RS256 in Systemen mit mehreren Token-prüfenden Komponenten häufig gegenüber HS256 bevorzugt wird.

Damit ein *Resource Server* den öffentlichen Schlüssel eines Ausstellers überhaupt kennt, definiert RFC 7517 ***(JSON Web Key, JWK***) ein standardisiertes JSON-Format, um einen einzelnen kryptografischen Schlüssel (Modulus und Exponent bei RSA, Kurve und Punkt bei elliptischen Kurven) zu beschreiben (Jones, 2015b). Mehrere solche Schlüssel lassen sich zu einem ***JSON Web Key Set (JWKS)*** zusammenfassen, das üblicherweise über einen öffentlich erreichbaren HTTP-Endpunkt (*häufig /.well-known/jwks.json oder /oauth2/jwks*) bereitgestellt wird. Jeder Schlüssel im Set trägt eine eindeutige Kennung (***kid***), die auch im Header eines JWT enthalten ist, sodass ein Resource Server bei mehreren gleichzeitig gültigen Schlüsseln den passenden auswählen kann, etwa während eines geplanten Schlüsselwechsels (Key Rotation), bei dem ein neuer Schlüssel bereits zum Signieren neuer Token verwendet wird, während der alte öffentliche Schlüssel noch zur Prüfung bereits ausgestellter, aber noch nicht abgelaufener Token vorgehalten wird.

2.2.4 Passwort-Hashing: bcrypt

Während RS256 und JWTs die Integrität und Übertragung eines bereits ausgestellten Tokens absichern, stellt sich bei der ursprünglichen Anmeldung eine andere Frage: wie ein Passwort sicher gespeichert werden kann, um es gegen einen erfolgreichen Datenbankeinbruch zu schützen. Die naheliegende Lösung, Passwörter mit einer gewöhnlichen kryptografischen Hashfunktion wie SHA-256 zu speichern, ist dafür ungeeignet, weil solche Hashfunktionen absichtlich sehr schnell berechenbar sind und sich damit in sehr kurzer Zeit Milliarden

möglicher Passwörter durchprobieren lassen (Brute-Force- bzw. Rainbow-Table-Angriffe). ***bcrypt*** ist ein von Provos und Mazières vorgestelltes Passwort-Hashing-Verfahren, das auf der Blockchiffre Blowfish aufbaut und absichtlich einen einstellbaren Kostenfaktor (auch Work Factor genannt) enthält, der bestimmt, wie viele Runden der internen Schlüsselexpansion durchlaufen werden (Provos & Mazières, 1999). Eine Erhöhung dieses Kostenfaktors um eins verdoppelt den Rechenaufwand pro Hash-Berechnung, wodurch sich bcrypt ansteigende Rechenleistung von Angreifern anpassen lässt, ohne den Algorithmus selbst zu ändern. Zusätzlich erzeugt *bcrypt* für jedes Passwort automatisch einen zufälligen Salt, der zusammen mit Kostenfaktor und Hash-Wert in einer einzigen Zeichenkette gespeichert wird; dadurch führt dasselbe Passwort bei zwei unterschiedlichen Nutzern zu unterschiedlichen gespeicherten Werten, was vorberechnete Rainbow-Tables wirkungslos macht.

2.2.5 Spring Security und OAuth2-Ressourcenserver

Spring Security ist die Sicherheitskomponente des Spring-Frameworks und stellt Authentifizierung und Autorisierung über eine Kette von *Servlet-Filtern* bereit, die sogenannte ***SecurityFilterChain*** (VMware, 2026a). Technisch registriert Spring Security dafür einen einzigen Servlet-Filter namens ***DelegatingFilterProxy*** beim Anwendungsserver, der jede Anfrage an eine intern verwaltete Kette von Sicherheitsfiltern weiterreicht; diese Kette lässt sich pro Anwendung frei konfigurieren, ohne dass der Anwendungsserver selbst etwas davon merkt (VMware, 2026a). Jede eingehende Anfrage durchläuft diese Kette, bevor sie den eigentlichen Controller erreicht. Für JWT-basierte APIs bietet Spring Security die Rolle des OAuth2 Resource Server: Über den Baustein *spring-boot-starter-oauth2-resource-server* wird der Filterkette automatisch ein ***BearerTokenAuthenticationFilter*** hinzugefügt, der den Authorization-Header einer Anfrage ausliest und das darin enthaltene Bearer-Token zur Prüfung an einen ***AuthenticationManager*** weiterreicht (VMware, 2026a). Dieser delegiert an einen ***JwtDecoder***, der Signatur, Ablaufdatum, Aussteller und den anwendungsspezifischen Widerruf prüft. Ist das Token gültig, wandelt ein ***JwtAuthenticationConverter*** die Claims in ein Spring-Security-Berechtigungsobjekt (Authentication) um; dabei greift standardmäßig ein ***JwtGrantedAuthoritiesConverter***, der den Inhalt des scope-Claims (siehe 2.2.2) in einzelne *GrantedAuthority*-Objekte mit dem

Präfix *SCOPE_* übersetzt, etwa *SCOPE_admin* oder *SCOPE_user* (VMware, 2026a). Auf Basis dieser Authorities lassen sich sowohl URL-basierte Sicherheitsregeln als auch methodenbasierte *@PreAuthorize*-Ausdrücke auf einzelnen Methoden formulieren, die über die Annotation *@EnableMethodSecurity* aktiviert werden. Erst nachdem die Authentication erfolgreich im ***SecurityContextHolder*** abgelegt wurde, können nachgelagerte Autorisierungsregeln (siehe 2.3) darauf zugreifen.

Abbildung 2.8 fasst diesen Ablauf für eine bereits authentifizierte Anfrage zusammen und zeigt, an welcher Stelle die in 2.2.2 bis 2.2.5 beschriebenen Konzepte (Claims, RS256-Signatur, Widerrufsprüfung) technisch verankert sind.

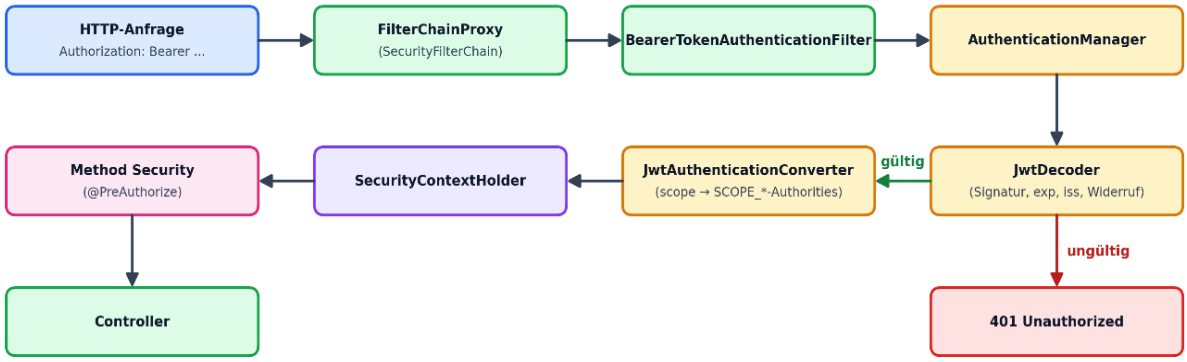


Abbildung 2.8: Ablauf der Anfrageprüfung durch den OAuth2-Ressourcenserver in Spring Security (eigene Darstellung nach VMware, 2026a).

Neben Spring Security pflegt Broadcom/VMware ein eigenständiges Projekt namens Spring *Authorization Server*, das einen vollständigen OAuth2- und *OpenID-Connect*-*Autorisierungsserver* mit eigenen Endpunkten für Token-Ausstellung, Introspektion und Widerruf bereitstellt (VMware, 2026b). Dieses Projekt unterscheidet sich von der in 2.2.5 beschriebenen Resource-Server-Rolle dadurch, dass es selbst sämtliche OAuth2-Endpunkte *(/oauth2/authorize, /oauth2/token, /oauth2/introspect usw.*) bereitstellt, anstatt lediglich eingehende Token zu prüfen. Als Alternative dazu kann ein Resource-Server-Baustein mit einer eigenen, schlankeren Token-Ausstellung kombiniert werden: Der Starter *spring-boot-starter-oauth2-resource-server* bringt transitiv die Bibliothek ***Nimbus JOSE+JWT*** mit, die Spring Security intern für das Erzeugen (*JwtEncoder*) und Prüfen (*JwtDecoder*) von JWTs

nutzt (Connect2id Ltd., 2026). Auf dieser Grundlage lässt sich serverseitig ein RSA-Schlüsselpaar erzeugen, lokal als *JSON Web Key* persistieren und sowohl zum Signieren neu ausgestellter Token als auch, über den zugehörigen öffentlichen Schlüssel, zu deren Prüfung verwenden. Zusätzlich zur Standardprüfung von Signatur, Ablaufdatum und Aussteller (*issuer*) lässt sich die Prüfkette um einen anwendungsspezifischen Validator erweitern, der bereits ausgestellte, aber zwischenzeitlich widerrufene Token anhand eines Datenbankeintrags erkennt. Das ist notwendig, weil ein JWT laut Spezifikation zustandslos ist und ohne eine solche Zusatzprüfung bis zu seinem Ablauf gültig bliebe, selbst wenn sich der Nutzer zwischenzeitlich abgemeldet hat oder gesperrt wurde.

Abbildung 2.9 stellt die beiden Architekturvarianten einander gegenüber.

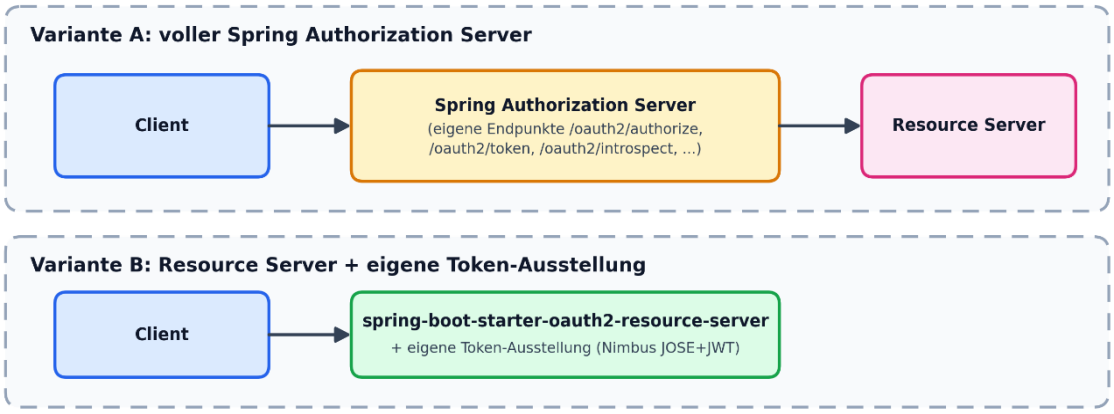


Abbildung 2.9: Spring Authorization Server und Resource-Server-Starter mit eigener Token-Ausstellung (eigene Darstellung).

Eine ausführliche Darstellung der Filterkette und der Rolle des Resource Servers in Spring Security findet sich bei Spilca (2024); eine Framework-unabhängige Einführung in die OAuth2-Abläufe aus Sicht von Client, Authorization Server *und Resource Server* geben Richer und Sanso (2017).

2.3 Autorisierung und Berechtigungsmodelle

Neben der Authentifizierung (Wer ist der Nutzer?) ist die Autorisierung (Was darf der Nutzer?) ein eigener Belang, für den sich in der Literatur mehrere grundlegende Modelle etabliert haben.

2.3.1 Zugriffskontrollmodelle: ACL, RBAC und ABAC

Das historisch älteste Modell ist die Zugriffskontrollliste (*Access Control List, ACL*): Jeder geschützten Ressource wird dabei eine Liste zugeordnet, die für einzelne Nutzer oder Nutzergruppen festlegt, welche Operationen (lesen, schreiben, ausführen usw.) erlaubt sind (Samarati & De Capitani di Vimercati, 2001). ACLs sind konzeptionell einfach, skalieren aber schlecht, sobald sich Berechtigungen für viele Nutzer gleichzeitig ändern sollen, weil dann unter Umständen jede einzelne Liste angepasst werden muss.

Die rollenbasierte Zugriffskontrolle (*Role-Based Access Control, RBAC*) löst dieses Skalierungsproblem, indem sie eine zusätzliche Indirektionsebene einführt: Berechtigungen werden nicht direkt an einzelne Nutzer, sondern an Rollen gebunden, denen wiederum beliebig viele Nutzer zugeordnet werden können. Eine Änderung an den Rechten einer Rolle wirkt sich dadurch sofort auf alle ihr zugeordneten Nutzer aus. Das Modell wurde durch den Standard ANSI/INCITS 359 formalisiert, der zusätzlich vier aufeinander aufbauende Ausbaustufen unterscheidet: Flat RBAC als Minimalvariante mit reiner Nutzer-Rolle-Zuordnung, *Hierarchical RBAC*, das zusätzlich Vererbungsbeziehungen zwischen Rollen erlaubt (eine spezialisierte Rolle erbt automatisch die Rechte einer allgemeineren Rolle), *Constrained RBAC*, das Nebenbedingungen wie eine Pflicht zur Funktionstrennung (Separation of Duty) zwischen bestimmten Rollen durchsetzt, und *Symmetric RBAC*, das zusätzlich eine effiziente Abfrage ermöglicht, welche Nutzer eine bestimmte Berechtigung besitzen (American National Standards Institute, 2004; Ferraiolo et al., 2001).

Ein drittes, weiter gefasstes Modell ist die attributbasierte Zugriffskontrolle (*Attribute-Based Access Control, ABAC*): Statt Berechtigungen über vordefinierte Rollen zu vergeben, trifft

ABAC die Zugriffsentscheidung zur Laufzeit anhand einer Kombination von Attributen des Nutzers (z. B. Abteilung, Sicherheitsfreigabe), der Ressource (z. B. Vertraulichkeitsstufe), der geplanten Aktion und des Kontexts (z. B. Uhrzeit, Standort, Netzwerk) (Hu et al., 2014). ABAC ist dadurch deutlich ausdrucksstärker als RBAC und erlaubt feingranulare, kontextabhängige Regeln, erkauft dies aber mit höherer Komplexität bei Entwurf, Pflege und Nachvollziehbarkeit der Regeln. In der Praxis werden RBAC und ABAC häufig kombiniert, etwa indem eine Rolle die grobe Zugriffskategorie vorgibt und zusätzliche Attributregeln diese im Einzelfall weiter einschränken. Abbildung 2.10 stellt das Grundprinzip von RBAC dem von ABAC gegenüber.

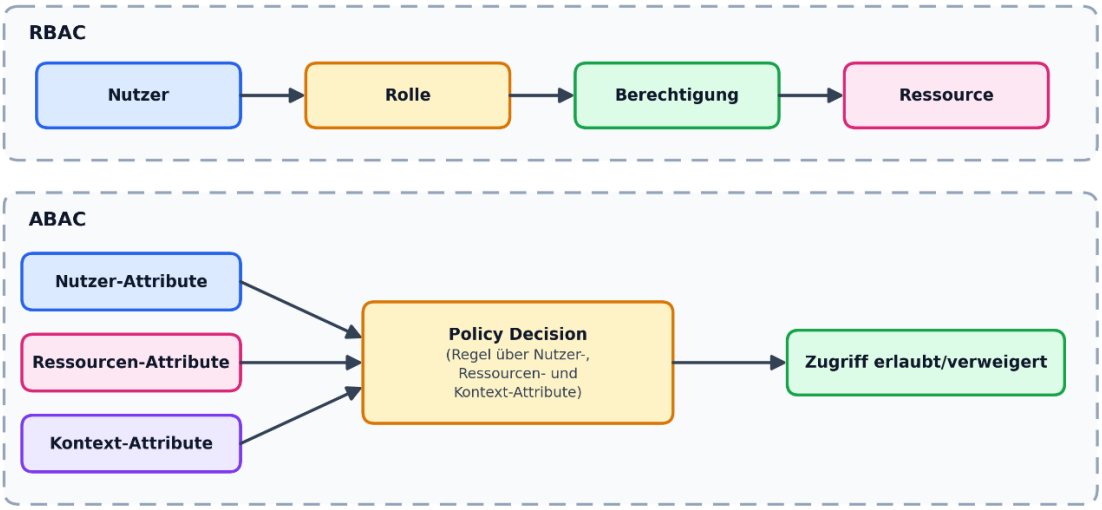


Abbildung 2.10: Grundprinzip von RBAC im Vergleich zu ABAC (eigene Darstellung nach Ferraiolo et al., 2001; Hu et al., 2014)

2.3.2 Mehrstufige Entscheidungskaskaden und Policy Enforcement

Systeme, die zusätzlich Berechtigungen für Gruppen von Nutzern und für einzelne Endpunkte oder Funktionen getrennt verwalten, kombinieren RBAC häufig mit mehrstufigen Entscheidungs-Kaskaden (OASIS, 2013). Dabei werden mehrere Regelebenen nacheinander ausgewertet, zum Beispiel zuerst eine nutzerspezifische Regel, dann eine gruppenspezifische Regel und erst zuletzt eine Standardregel, bis die erste Regel greift, die zu dieser Anfrage passt. Begrifflich lässt sich eine solche Prüfung in zwei Rollen aufteilen, wie sie unter

anderem im XACML-Referenzmodell der OASIS verwendet werden: ***Der Policy Enforcement Point (PEP)*** ist die Stelle, die eine Anfrage tatsächlich abfängt und entweder durchlässt oder blockiert (z. B. ein Sicherheitsfilter einer Webanwendung), während der *Policy Decision Point (PDP)* die eigentliche Entscheidung anhand der hinterlegten Regeln trifft und dem PEP lediglich ein Ergebnis ("erlaubt"/"verweigert") zurückliefert (OASIS, 2013). Diese Trennung erlaubt es, Entscheidungslogik (PDP) und Durchsetzung (PEP) unabhängig voneinander zu ändern, etwa um dieselbe Entscheidungslogik an mehreren Durchsetzungspunkten wiederzuverwenden. Abbildung 2.11 zeigt eine solche mehrstufige Kaskade als Entscheidungsbaum.

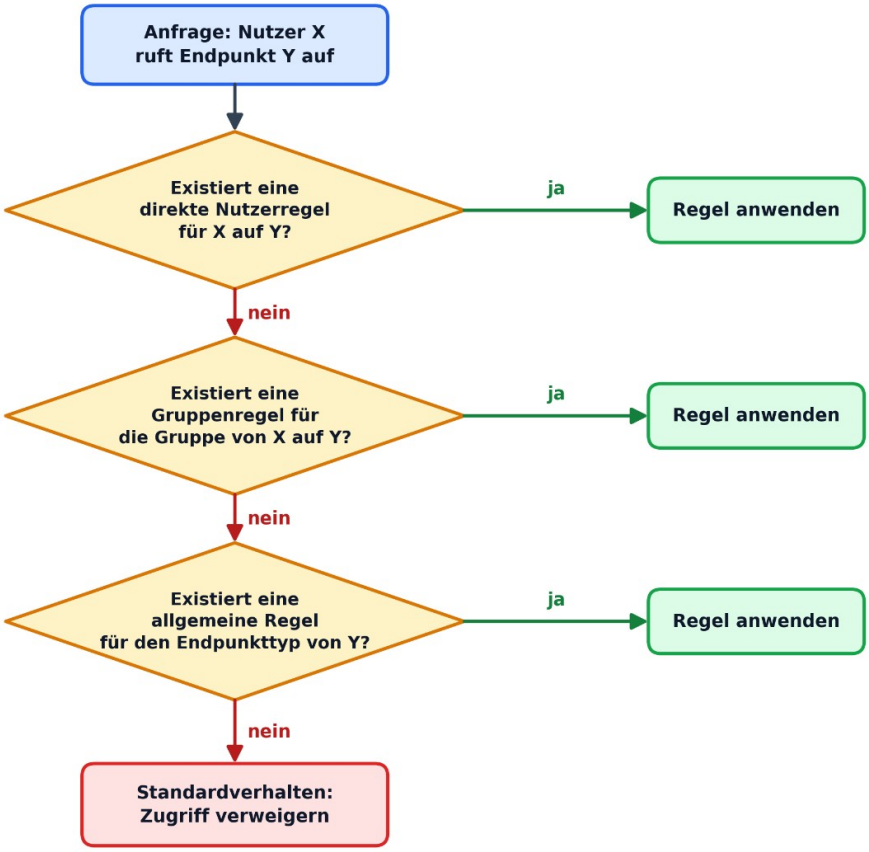


Abbildung 2.11: Schematischer Entscheidungsbaum einer mehrstufigen Berechtigungskaskade (eigene Darstellung).

2.3.3 Fail-Safe Defaults und Default-Deny

Für den Fall, dass keine dieser Regeln zutrifft, stellt sich die Frage nach dem Standardverhalten des Systems. Eine klassische Antwort liefert ein Prinzip des sicheren Systementwurfs, das bereits 1975 von Saltzer und Schroeder unter dem Namen *fail-safe defaults* beschrieben wurde: Eine Zugriffsentscheidung sollte grundsätzlich auf Verweigerung beruhen, und Zugriff darf nur gewährt werden, wenn eine Bedingung dies ausdrücklich erlaubt, nicht umgekehrt (Saltzer & Schroeder, 1975). In der Praxis wird dieses Verhalten häufig als *fail-closed* oder *deny-by-default* bezeichnet und taucht auch in aktuellen Sicherheitsempfehlungen wieder auf, etwa in den OWASP Proactive Controls, die eine konsequente Durchsetzung von Zugriffskontrollen mit Default-Deny als eine der wichtigsten Maßnahmen gegen fehlerhafte Autorisierung nennen (OWASP Foundation, 2024). Dieses Prinzip lässt sich auf zwei Ebenen anwenden: auf Ebene eines Sicherheits-Frameworks wie Spring Security, das standardmäßig eine Authentifizierung für jede Anfrage verlangen und nur einzelne, bewusst ausgewählte Endpunkte davon ausnehmen kann, sowie auf Ebene einer mehrstufigen Berechtigungskaskade, die nur dann ein Ergebnis ungleich "verweigert" liefert, wenn in mindestens einer ihrer Stufen ausdrücklich eine erlaubende Regel gefunden wird; ist keine Regel vorhanden oder ist der Nutzer unbekannt, bleibt es bei der Verweigerung.

2.4 Frameworks und Werkzeuge des Java-Ökosystems

2.4.1 Apache Maven: Build-Automatisierung und Abhängigkeitsverwaltung

Bevor eine Java-Anwendung ausgeführt werden kann, müssen ihre Quelldateien kompiliert, ihre Abhängigkeiten aufgelöst und die Ergebnisse zu einem lauffähigen Artefakt (z. B. einer JAR-Datei) zusammengefügt werden. Apache Maven ist ein Build-Automatisierungs- und Projektmanagement-Werkzeug für das Java-Ökosystem, das diese Schritte deklarativ über eine zentrale Projektbeschreibungsdatei, die Project Object Model-Datei (pom.xml), steuert

(Apache Software Foundation, 2026). In dieser Datei werden unter anderem die benötigten Abhängigkeiten (Bibliotheken und deren Versionen) sowie zusätzliche Plugins für Spezialaufgaben (z. B. Testausführung, Code-Coverage-Messung) deklariert; Maven lädt diese Abhängigkeiten automatisch aus einem zentralen Repository herunter und löst dabei auch transitive Abhängigkeiten auf, also Abhängigkeiten der eigenen Abhängigkeiten. Maven strukturiert jeden Build zudem in einen festen Lifecycle aus aufeinanderfolgenden Phasen, von denen die wichtigsten *validate* (Projektstruktur prüfen), *compile* (Quellcode übersetzen), *test* (Tests ausführen), *package* (Artefakt erzeugen) und *install* (Artefakt im lokalen Repository ablegen) sind; der Aufruf einer späteren Phase führt dabei automatisch alle vorangehenden Phasen mit aus. Abbildung 2.12 zeigt diesen Lifecycle vereinfacht.

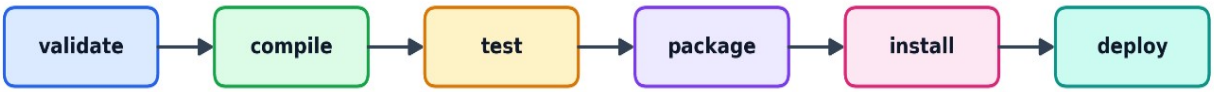


Abbildung 2.12: Maven-Build-Lifecycle mit den Standardphasen (Apache Software Foundation, 2026).

2.4.2 Spring Framework: Inversion of Control und Dependency Injection

Spring Boot baut auf dem klassischen Spring Framework auf, dessen zentrales Konzept die *Inversion of Control* (*IoC*) ist: Statt dass ein Objekt seine benötigten Abhängigkeiten selbst erzeugt (z. B. über den *new-Operator*), deklariert es lediglich, welche Abhängigkeiten es benötigt, und überlässt deren Erzeugung und Verdrahtung einem übergeordneten *IoC-Container* (VMware, 2026d). Dieses Prinzip, bei dem die Kontrolle über den Objektlebenszyklus vom Objekt selbst zu einem Container verlagert wird, wird auch als *Dependency Injection* (DI) bezeichnet. Der Spring-IoC-Container verwaltet dafür sogenannte Beans: von Spring verwaltete Objektinstanzen, die typischerweise über Annotationen wie *@Component*, *@Service* oder *@Repository* markiert und beim Anwendungsstart automatisch erkannt werden. Benötigt eine Bean eine andere Bean, wird diese Abhängigkeit meist über einen Konstruktorparameter deklariert; der Container erkennt diesen Bedarf, sucht eine passende Bean-Instanz (oder erzeugt sie bei Bedarf selbst) und übergibt sie automatisch, ohne dass die abhängige Klasse wissen muss, wie die konkrete Implementierung erzeugt wird.

Dieser Mechanismus erleichtert insbesondere automatisiertes Testen, weil sich echte Abhängigkeiten in Tests leicht durch Platzhalterobjekte (Mocks, siehe 2.4.6) ersetzen lassen, sowie die lose Kopplung zwischen Schichten einer Anwendung, da eine Klasse nur von einer Schnittstelle, nicht von einer konkreten Implementierung abhängt. Spring Boot setzt auf diesem Container auf und ergänzt ihn um Auto-Konfiguration und Starter-Abhängigkeiten, wie im folgenden Abschnitt beschrieben.

2.4.3 Spring Boot und ausgewählte Starter-Bausteine

Spring Boot ist ein Framework für das Java-Ökosystem, das auf dem klassischen Spring Framework aufbaut und durch Konventionen und Auto-Konfiguration den Aufwand reduziert, eine eigenständig lauffähige, produktionsreife Anwendung aufzusetzen (VMware, 2026c). Statt zahlreicher XML-Konfigurationsdateien genügt es in der Regel, eine passende Abhängigkeit (einen sogenannten Starter) einzubinden, damit Spring Boot die zugehörige Komponente automatisch mit sinnvollen Standardwerten konfiguriert; eigene Konfigurationsklassen überschreiben diese Standardwerte bei Bedarf gezielt. Im Java-Ökosystem werden für typische Backend-Aufgaben unter anderem folgende Bausteine eingesetzt:

- *spring-boot-starter-web* stellt die REST-Schicht bereit, also die Infrastruktur, über die Controller HTTP-Anfragen entgegennehmen und JSON-Antworten zurückgeben.
- *spring-boot-starter-security* und *spring-boot-starter-oauth2-resource-server* stellen zusammen die in 2.2 beschriebene JWT-basierte Authentifizierung als OAuth2-Ressourcenserver bereit.
- *spring-boot-starter-data-jpa* bindet Hibernate als Implementierung der Jakarta Persistence API (JPA) ein (siehe 2.4.4).
- *spring-boot-starter-validation* ermöglicht deklarative Eingabevalidierung über Jakarta-Bean-Validation-Annotationen (z. B. Pflichtfelder, Längen- oder Formatprüfungen) direkt an den Datenübertragungsobjekten.
- *spring-boot-starter-mail* und *spring-boot-starter-thymeleaf* werden häufig zusammen für den Versand von E-Mails mit dynamischem Inhalt genutzt: *Thymeleaf* ist eine serverseitige Template-Engine, die HTML-Inhalte aus Templates und Daten rendert;

die resultierende HTML-Mail wird anschließend über die Mail-Starter-Komponente verschickt.

- *mariadb-java-client* ist der JDBC-Treiber, über den eine Java-Anwendung mit einer MariaDB-Datenbank kommuniziert (siehe 2.4.5).
- *springdoc-openapi-starter-webmvc-ui* erzeugt aus den Controller- und DTO-Klassen automatisch eine OpenAPI-Beschreibung der Schnittstelle sowie eine interaktive Swagger-UI zum Testen der Endpunkte.
- *lombok* reduziert Boilerplate-Code (Getter, Setter, Konstruktoren) über Annotationen, die zur Kompilierzeit in echten Java-Code übersetzt werden.
- *spring-boot-starter-test*, *spring-security-test* und *h2* bilden die Testinfrastruktur (siehe 2.4.6): Der Test-Starter bringt JUnit 5 und Mockito mit, *spring-security-test* stellt Hilfsmittel bereit, um authentifizierte Anfragen in Tests zu simulieren, und die eingebettete Datenbank H2 ersetzt in Integrationstests eine produktive Datenbank, ohne dass dafür eine externe Datenbank laufen muss.
- *spring-boot-starter-actuator* stellt Health- und Monitoring-Endpunkte bereit, über die sich zur Laufzeit prüfen lässt, ob die Anwendung und ihre Abhängigkeiten (z. B. die Datenbankverbindung) korrekt funktionieren.

2.4.4 Objektrelationales Mapping: Hibernate und Jakarta Persistence API

Eine relationale Datenbank speichert Daten in Tabellen mit Zeilen und Spalten, während eine Java-Anwendung mit Objekten und Referenzen zwischen ihnen arbeitet; dieser strukturelle Unterschied wird als *Impedance Mismatch* bezeichnet. Die Jakarta Persistence API (JPA, vormals Java Persistence API) ist eine Spezifikation, die festlegt, wie Java-Klassen (Entities) über Annotationen *(@Entity, @Id, @Column, @OneToMany usw.*) auf Tabellen, Primärschlüssel, Spalten und Beziehungen abgebildet werden (Eclipse Foundation, 2022; Hibernate, 2026). Hibernate ist die am weitesten verbreitete Implementierung dieser Spezifikation und übernimmt die eigentliche Umsetzung: Es übersetzt Lese- und Schreiboperationen auf Entity-Objekten automatisch in SQL-Anweisungen, verwaltet pro Transaktion einen *Persistence Context* (auch *Session* genannt), der geladene Entities

zwischenspeichert und Änderungen an ihnen bis zum Transaktionsende sammelt, und erzeugt für häufige Zugriffsmuster automatisch Datenbankabfragen, ohne dass dafür handgeschriebenes SQL notwendig ist. Abbildung 2.13 veranschaulicht diese Abbildung zwischen einer Java-Klasse und einer Datenbanktabelle.

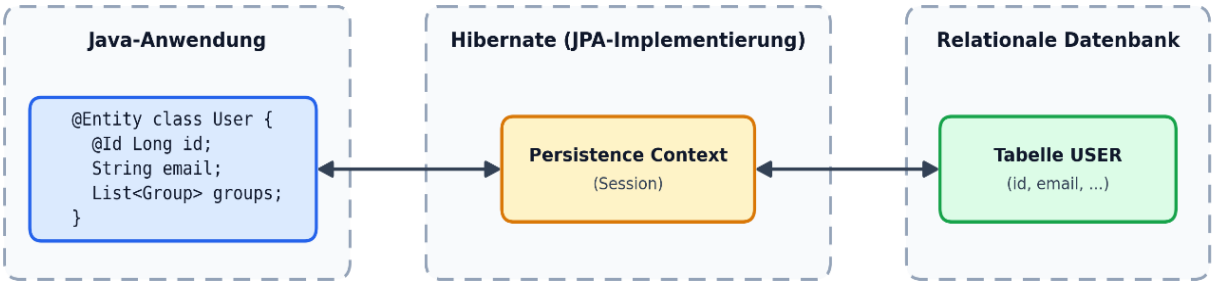


Abbildung 2.13: Objektrelationales Mapping zwischen einer Java-Entity-Klasse und einer Datenbanktabelle über Hibernate als JPA-Implementierung (eigene Darstellung).

2.4.5 MariaDB

MariaDB ist ein quelloffenes, relationales Datenbankmanagementsystem, das 2009 als Fork von MySQL entstand, nachdem MySQL von Oracle übernommen wurde, und seither von der unabhängigen MariaDB Foundation weiterentwickelt wird (MariaDB Foundation, 2026). MariaDB ist weitgehend protokoll- und datenformatkompatibel zu MySQL, bringt darüber hinaus aber eigene Speicher-Engines (u. a. Aria, ColumnStore) und Erweiterungen mit. Der Zugriff einer Java-Anwendung erfolgt klassisch über das Java Database Connectivity-API (JDBC), für das der *mariadb-java-client* den datenbankspezifischen Treiber bereitstellt, der die JDBC-Aufrufe in das MariaDB-eigene Netzwerkprotokoll übersetzt; Hibernate (siehe 2.4.4) setzt wiederum auf diesem JDBC-Treiber auf.

2.4.6 Testwerkzeuge: JUnit 5 und Mockito

JUnit 5 ist das Standard-Testframework für die JVM und stellt die Grundinfrastruktur bereit, um einzelne Testfälle (annotiert mit *@Test*) zu deklarieren, zu gruppieren und automatisiert auszuführen; es bildet die Basis der meisten Build-Werkzeuge (einschließlich Maven) zur Testausführung (JUnit Team, 2026). JUnit-Tests folgen üblicherweise dem Schema *Arrange-*

Act-Assert: Zunächst wird der Ausgangszustand hergestellt, dann die zu testende Operation ausgeführt und abschließend das Ergebnis mit einer Erwartung verglichen. Mockito ist eine Bibliothek, die dieses Schema um Mock-Objekte ergänzt: Statt eine echte Abhängigkeit (z. B. eine Datenbankanbindung oder einen externen Dienst) in einem Test zu verwenden, erzeugt Mockito zur Laufzeit ein Platzhalterobjekt, das dieselbe Schnittstelle implementiert, sich aber exakt so verhält, wie es der Test vorgibt (z. B. "gib bei diesem Methodenaufruf diesen Wert zurück") (Mockito contributors, 2026). Dadurch lässt sich eine einzelne Klasse isoliert von ihren Abhängigkeiten testen (Unit-Test), ohne dass eine vollständige Anwendung mit Datenbank oder Netzwerkzugriff gestartet werden muss; das durch die in 2.4.2 beschriebene *Dependency Injection* ermöglichte lose Koppeln von Schnittstelle und Implementierung ist dabei die technische Voraussetzung dafür, dass sich echte Abhängigkeiten überhaupt durch Mocks ersetzen lassen.

2.4.7 ArchUnit: Architekturregeln als Test

Um die Einhaltung einer vorgesehenen Schichtenarchitektur (z. B. Controller ruft nur Service, Service ruft nur Repository, nicht umgekehrt) über die gesamte Projektlaufzeit hinweg automatisiert zu prüfen, existiert das Architektur-Testwerkzeug *ArchUnit* (ArchUnit, n.d.). Mit *ArchUnit* lassen sich Regeln über erlaubte und verbotene Paketabhängigkeiten als ganz normale JUnit-5-Tests formulieren, die bei jedem Build mitlaufen und damit verhindern, dass sich eine vorgesehene Architektur unbemerkt im Laufe der Entwicklung auflöst.

2.4.8 Flyway: versionierte Datenbankmigrationen

Eine relationale Datenbank entwickelt sich über die Projektlaufzeit genauso weiter wie der Code: Tabellen kommen hinzu, Spalten ändern sich, Indizes werden ergänzt. *Flyway* ist ein Werkzeug, das diese Änderungen als nummerierte, unveränderliche SQL-Skripte (z. B. *V1__init.sql*, *V2__add_user_group.sql*) verwaltet (Redgate Software, 2026). Beim Start der Anwendung vergleicht *Flyway* die im Projekt vorhandenen Migrationsskripte mit einer eigenen Tabelle in der Zieldatenbank, in der bereits ausgeführte Migrationen protokolliert sind, und führt dann automatisch genau die Skripte aus, die noch nicht angewendet wurden.

Jedes ausgeführte Skript wird dabei mit einer Prüfsumme versehen; wird eine bereits angewendete Migration nachträglich verändert, bricht *Flyway* beim nächsten Start mit einem Fehler ab, anstatt eine möglicherweise inkonsistente Datenbank stillschweigend zu akzeptieren. Das erlaubt es, ein Datenbankschema unter Versionskontrolle zu nehmen und nachvollziehbar zwischen Entwicklungs-, Test- und Produktivumgebung synchron zu halten. Da dieselben Migrationsskripte auch in Integrationstests gegen eine eingebettete Datenbank wie H2 ausgeführt werden können, lässt sich zudem erreichen, dass Testumgebung und Produktivumgebung bezogen auf das Schema weitgehend identisch bleiben.

2.4.9 JaCoCo: Testabdeckung messen

JaCoCo (Java Code Coverage) ist ein Werkzeug, das zur Laufzeit misst, welche Teile des kompilierten Bytecodes während der Testausführung tatsächlich durchlaufen wurden (Mountainminds GmbH & Co. KG, 2026). Technisch geschieht das über einen Java-Agenten, der sich beim Start der Testausführung in die JVM einklinkt (bei Maven-Projekten üblicherweise über das Build-Ziel prepare-agent) und jede ausgeführte Code-Zeile beziehungsweise jeden ausgeführten Zweig (branch) mitzählt. Nach Abschluss der Tests erzeugt das zugehörige Ziel report einen HTML- und XML-Bericht, der pro Klasse und Paket ausweist, wie hoch der Anteil an durch Tests abgedecktem Code ist. Ein solcher Bericht dient häufig als objektive, automatisiert erzeugte Kennzahl zur Bewertung der Testabdeckung und Testqualität eines Projekts.

3 Anforderungsanalyse

Dieses Kapitel beschreibt, welches Problem gelöst werden soll und welche Eigenschaften die Lösung erfüllen muss – nicht, mit welchen Klassen, Bibliotheken oder Algorithmen dies in Kapitel 4 geschieht. Grundlage sind zwei Quellen: die offizielle Ausschreibung (Arbeitspakete AP1–AP6) und die vollständige Quellcode-Analyse des bestehenden PHP-Backends *‚design-daas-application-backend‘* (Abschnitt 3.1). Jede genannte Anforderung lässt sich auf eine konkrete Stelle im Code, in einer Konfigurationsdatei oder in der Ausschreibung zurückführen. Abschnitt 3.4 beschreibt zusätzlich den tatsächlich erreichten Zielzustand anhand des neuen Java-Quellcodes (Paketstruktur, Datenbankschema, Testklassen). Ergänzend wurde der Quellcode von Frontend und DaaS-Anwendung daraufhin untersucht, welche Endpunkte diese tatsächlich aufrufen (Abschnitt 3.2.2).

3.1 Generelle Zielsetzung

Das Backend des Desktop-as-a-Service-Systems **DESIGN** (Kapitel 2.1) soll vollständig neu entwickelt werden. Das bestehende Backend ist in PHP implementiert und wird in der Ausschreibung ausdrücklich als „qualitativ schwach“ und „schlecht dokumentiert“ bezeichnet; die beiden anderen Systemteile, DaaS-Anwendung und Frontend, gelten dagegen als qualitativ gut und ausreichend dokumentiert. Daraus ergibt sich die doppelte Zielsetzung dieses Kapitels: Erstens muss das sichtbare Verhalten des alten Backends (Schnittstellen, Datenformate, Berechtigungslogik) vollständig verstanden werden – eine externe Dokumentation existiert dafür nicht, der Quellcode ist die einzige verlässliche Quelle. Zweitens wird daraus ein Anforderungskatalog abgeleitet, der festlegt, was ein funktional gleichwertiger, aber wart- und testbarer Nachfolgedienst leisten muss, ohne bereits festzulegen, wie dies technisch umgesetzt wird (Kapitel 4).

3.1.1 Genereller Aufbau des Gesamtsystems

Das Gesamtsystem **DESIGN** besteht aus drei eigenständigen Repositories: der **DaaS-Anwendung** (Python; verwaltet Container, virtuelle Maschinen, Proxmox VE und Apache Guacamole, also die eigentliche Bereitstellung von Desktops), dem **Frontend** (React; grafische Oberfläche für Endnutzer und Administratoren) und dem **Backend**, das Gegenstand dieser Arbeit ist. Die drei Teile kommunizieren ausschließlich über REST-Schnittstellen; es gibt keine gemeinsame Codebasis. Frontend und DaaS-Anwendung werden in dieser Arbeit nicht verändert (Kapitel 1, Abgrenzung). Das neue Backend muss daher deren bestehende Schnittstellenerwartungen – Pfade, HTTP-Methoden, JSON-Feldnamen, Statuscodes und die Semantik der Berechtigungsprüfung – exakt so erfüllen wie bisher das alte PHP-Backend. Diese sogenannte *Verhaltenskompatibilität* ist der wichtigste limitierende Faktor für die in Abschnitt 3.2 abgeleiteten Anforderungen.

3.1.2 Ist-Zustand des alten Backends (AP1/AP2)

Die folgende Analyse beruht auf der direkten Durchsicht des Quellcodes von *‚design-daas-application-backend‘* (Klassenstruktur, Routen-Attribute, Entitäten, Migrationsskripte, *composer.json*, *docker-compose.yml*).

Technologie-Stack

Das alte Backend ist in PHP 8.2 implementiert und läuft als Apache-Modul in einem Docker-Container. Tabelle 3.1 zeigt die eingesetzten Technologien je Schicht.

Tabelle 3.1: Technologie-Stack des alten PHP-Backends

Schicht	Technologie
Sprache	PHP 8.2
HTTP-Server	Apache (im PHP-Container)
ORM / Datenbank	Doctrine ORM 2.x + MariaDB
Schema-Migrationen	Doctrine Migrations

Dependency Injection	PHP-DI (Autowiring + Annotationen)
HTTP-Routing	Symfony Routing (über PSR-7-Adapter)
HTTP-Nachrichten	PSR-7 (Guzzle-Implementierung)
OAuth2	*league/oauth2-server* (Password Grant)
API-Dokumentation	OpenAPI/Swagger (*zircote/swagger-php*)
Logging	Monolog (PSR-3)
Event-Bus	Symfony EventDispatcher (PSR-14)
Warteschlange	DB-gestützt, über *enqueue/dbal*
E-Mail-Versand	Symfony Mailer
Cron	Yacron
Containerisierung	Docker + Docker Compose
Tests/Statische Analyse	PHPUnit 10, PHPStan, PHPCS, PHPat

Modulstruktur und Schichtung

Der Code ist nicht nach technischen Schichten, sondern nach fachlichen Bereichen in *vertikale Module* gegliedert:

- ***Main***: Bootstrap, Fehlerbehandlung, Versions-Endpunkt
- ***Util***: Querschnittsfunktionen wie Mailversand, OAuth2-Middleware, Swagger
- ***OAuth2***: Token-Endpunkt, Session-Info
- ***User***: Nutzer-/Admin-Verwaltung, Gruppen, Registrierung
- ***Desktop***: Desktop- und Desktop-Gruppen-Verwaltung
- ***Permissions***: feingranulare Zugriffskontrolle

Jedes Modul ist intern gleich geschichtet: „***Model*** → ***Repository*** → ***Service*** → ***Controller***“, ergänzt um lokale ***DTO***-, ***Enum***- und ***Exception***-Unterpakete. Dieses Muster – fachliche Module mit identischer interner Schichtung – wird in Abschnitt 3.4 1:1 in Java-Pakete übersetzt.

Anfrageverarbeitung (Request-Lifecycle)

Jede HTTP-Anfrage durchläuft denselben Ablauf: Nach dem Bootstrap (Fehlerbehandlung, Doctrine-Konfiguration, Dependency-Injection-Container) übernimmt ein *HttpDispatcher* auf Basis von Symfony Routing die Anfrage. Vor dem Routing laufen Middleware-Komponenten zur Auflösung der Basis-URL und zum Setzen der CORS-Header; nach dem Routing folgen zwei weitere: eine zur Behandlung ungültiger Eingaben und die *OAuth2AccessControlMiddleware*. Letztere liest eine je Route hinterlegte OpenAPI-Sicherheitsannotation (z. B. *#[OA\Get(security: [["oauth2-user" => ["admin"]]])]*), validiert das mitgesendete Zugriffstoken und injiziert den authentifizierten Benutzer in die Anfrage. Erst danach wird die passende Controller-Aktion aufgerufen, die über Service- und Repository-Schicht auf den Doctrine *EntityManager* und damit auf MariaDB zugreift; die Antwort wird als JSON über eine PSR-7-Response zurückgegeben.

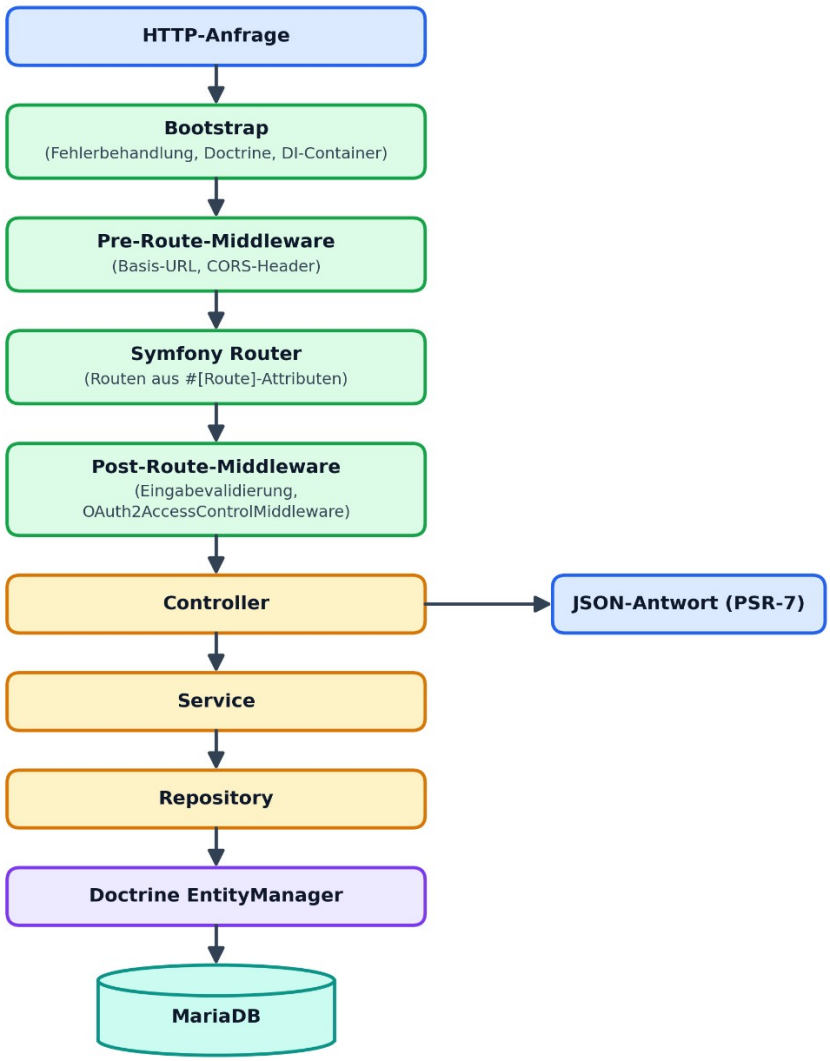


Abbildung 3.1: Anfrageverarbeitung im alten PHP-Backend (eigene Darstellung)

Domänenmodell

Abbildung 3.2 zeigt die zentralen Entitäten und ihre Beziehungen. ***BaseUser*** ist eine abstrakte Basis-Entität, von der ***Admin*** und ***User*** per *Single Table Inheritance (STI)* erben: Beide Rollen liegen in einer gemeinsamen Tabelle ***users***, unterschieden über die Spalte ***role***. Jeder Nutzer hat zusätzlich eine ***guid*** (UUID v4), einen ***registrationCode*** (8-stelliger Hex-Wert, 24 Stunden gültig, für die E-Mail-Verifikation) und ein ***enabled***-Flag; Passwörter werden nur als Hash über ein Password-Value-Object gespeichert. Nutzer stehen in einer n:m-Beziehung zu ***UserGroup***, diese wiederum n:m zu ***DesktopGroup***, die selbst n:m mit ***Desktop***-Entitäten

verknüpft ist. Getrennt davon existiert ein eigenes Berechtigungsmodell: ***Endpoint***-Entitäten (identifiziert über ***functionName***, plus URL und HTTP-Methode) können zu ***EndpointGroups*** zusammengefasst werden; vier Zuordnungstabellen (***EndpointUserAccess***, ***EndpointUserGroupAccess***, ***EndpointGroupUserAccess***, ***EndpointGroupUserGroupAccess***) verknüpfen einzelne Nutzer bzw. Nutzergruppen mit einzelnen Endpunkten bzw. Endpunktgruppen und tragen je ein Allow/Deny-Flag. Alle Entitäten sind ***final***, um unerwartete Vererbung zu verhindern.

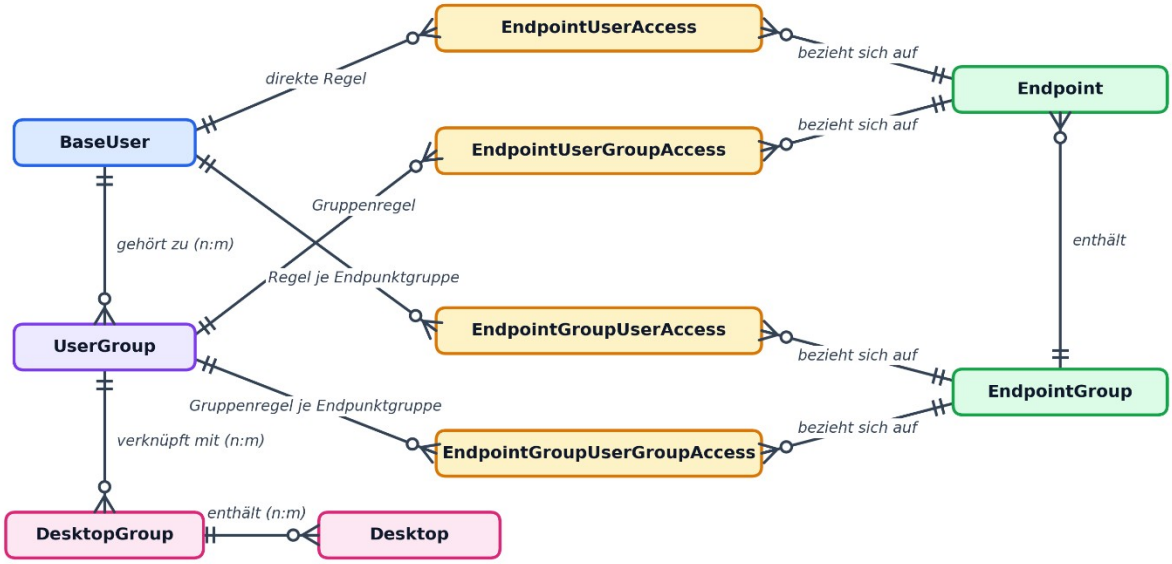


Abbildung 3.2: Domänenmodell des alten PHP-Backends (eigene Darstellung)

Berechtigungsmodell (Permissions-Engine)

Zusätzlich zur groben OAuth2-Unterscheidung zwischen den Scopes ***user*** und ***admin*** besitzt das alte Backend ein zweites, feingranulares, datenbankgetriebenes Berechtigungssystem. Die zentrale Methode *PermissionCalculationService::getPermission($functionName, $userId)* prüft in fester Reihenfolge vier Regeltypen und wählt die erste zutreffende:

1. ***EndpointUserAccess***: eine Regel für genau diesen Nutzer und genau diesen Endpunkt,
2. ***EndpointUserGroupAccess***: eine Regel für eine Nutzergruppe des Nutzers und genau diesen Endpunkt,

3. ***EndpointGroupUserAccess***: eine Regel für genau diesen Nutzer und eine Endpunktgruppe, der dieser Endpunkt angehört,
4. ***EndpointGroupUserGroupAccess***: eine Regel für eine Nutzergruppe des Nutzers und eine Endpunktgruppe.

Greift keine dieser vier Regeln, gilt per Voreinstellung **Zugriffsverweigerung** (*fail-closed*). Dieses Verhalten – insbesondere die Reihenfolge der vier Stufen und die Default-Deny-Regel – ist für die DaaS-Anwendung sicherheitskritisch, da sie sich bei jeder Nutzeraktion darauf verlässt. Das neue Backend muss es daher in gleicher Reihenfolge reproduzieren (Abschnitt 3.2). Innerhalb der Schritte 2 und 4 iteriert die Methode über alle Nutzergruppen des Nutzers und gibt das Ergebnis der ersten Gruppe zurück, für die eine passende Regel existiert. Da die Reihenfolge, in der *BaseUser::groups()* diese Gruppen liefert, im Code nicht sortiert wird, ist das Verhalten bei mehreren widersprüchlichen Gruppenregeln desselben Nutzers nicht deterministisch, sondern hängt von der internen Reihenfolge der Doctrine-Collection ab. Diese Beobachtung ist Ausgangspunkt für die Entscheidung in Abschnitt 3.4.2, im neuen Backend eine explizite, deterministische Konfliktauflösung zu ergänzen, statt die unspezifizierte alte Reihenfolge nachzubilden. Ergänzend existiert ein Desktop-Zugriffstest (***GET /users/access/{desktopId}***), der prüft, ob der authentifizierte Nutzer über seine Nutzergruppe mit einer Desktop-Gruppe verknüpft ist, die den angefragten Desktop enthält.

Vollständige Schnittstelle (Ist-Zustand)

Tabelle 3.2 listet alle vom alten Backend bereitgestellten Endpunkte auf, wie sie sich aus den Routen-Attributen der Controller-Klassen ergeben. Diese Tabelle bildet die Grundlage für die funktionalen Anforderungen in Abschnitt 3.2, Welche dieser Endpunkte die Clients tatsächlich verwenden und die daher zwingend in äquivalenter Form angeboten werden müssen, untersucht Abschnitt 3.2.2.

Tabelle 3.2: API-Endpunkte des alten PHP-Backends (Ist-Zustand)

Modul	Endpunkt (Methode + Pfad)	Beschreibung (Rolle)
Main	GET /version	Anwendungsversion (offen)

OAuth2	POST /oauth2/user/token	Token anfordern (offen)
OAuth2	GET /oauth2/user/session	Session-Info (user/admin)
OAuth2	GET /ping/user	Diagnose (user)
OAuth2	GET /ping/admin	Diagnose (admin)
User	GET /users	Nutzer auflisten (admin)
User	GET /user/{id}	Einzelnen Nutzer abrufen (admin)
User	PATCH /user/{id}	Nutzerdaten ändern (admin)
User	DELETE /user/{id}	Nutzer dauerhaft löschen (admin)
User	POST /user/{id}/enable	Konto aktivieren (admin)
User	POST /user/{id}/disable	Konto deaktivieren (admin)
User	POST /user/validate_email	E-Mail verifizieren (offen)
User	GET /users/access/{desktopId}	Desktop-Zugriff prüfen (user)
User	POST /user/request-application/{app}/{lang}	Freischaltung anfragen (user)
Admin	GET /admins	Alle Admins auflisten (admin)
Admin	GET /admin/{id}	Einzelnen Admin abrufen (admin)
Admin	POST /admin/{id}	Admin anlegen (admin)
Admin	DELETE /admin/{id}	Admin dauerhaft löschen (admin)
Desktop	GET /desktops	Desktops auflisten (admin)
Desktop	GET /desktop/{id}	Desktop lesen (admin)
Desktop	POST /desktop	Desktop anlegen (admin)
Desktop	GET /desktop-groups	Desktop-Gruppen auflisten (admin)
Desktop	GET /desktop_group/{id}	Desktop-Gruppe lesen (admin)
Desktop	POST /desktop-group	Desktop-Gruppe anlegen (admin)
Desktop	POST /desktop_group/{id}/user_group/{uId}	Nutzergruppe verknüpfen (admin)
Permissions	GET /permissions/{functionName}/{userId}	Berechtigung abfragen (admin)
Permissions	POST /permissions_info	Berechtigungsinfo abfragen (admin)
UserGroup	GET /user_groups	Alle Nutzergruppen auflisten (admin)
UserGroup	POST /user_group	Neue Nutzergruppe anlegen (admin)
UserGroup	PATCH /user_group/{id}	Nutzergruppe ändern (admin)
UserGroup	POST /user_group/{ugId}/{dgId}	Mit Desktop-Gruppe verknüpfen

		(admin)
EndpointGroup	PUT /endpoint_group/{id}/{ugId}/{access}	Zugriffsregel setzen (admin)
EndpointGroup	DELETE /endpoint_group/{id}/{ugId}	Zugriffsregel entfernen (admin)

Die *Endpoint*- und *EndpointGroup*-Stammdaten (welche Funktion zu welcher Endpunktgruppe gehört) werden nicht über HTTP verwaltet, sondern per CLI-Kommando (***permissions:import***) aus einer JSON-Datei importiert. Über die HTTP-API lassen sich nur die Allow/Deny-Zuordnungen zwischen Nutzergruppen und bestehenden Endpunktgruppen pflegen.

Konfiguration und Betrieb

Die Konfiguration erfolgt über Umgebungsvariablen, die beim Containerstart in den Dependency-Injection-Container abgebildet werden. Der Betrieb läuft über Docker Compose mit zwei Profilen (***dev-environment***, ***release-test-environment***); die Test-Datenbank ist eine eigenständige, flüchtige MariaDB-Instanz im Arbeitsspeicher (***tmpfs***), getrennt von der Entwicklungsdatenbank. Cronjobs (z. B. zum Aufräumen abgelaufener Registrierungscodes) laufen als separater ***yacron***-Dienst; Jobs sind bewusst kurz und über eine Nachrichtenwarteschlange entkoppelt, da verpasste Ausführungen nicht automatisch nachgeholt werden.

3.1.3 Umfeldbeschreibung

Tabelle 3.3 fasst die vorhandene Infrastruktur und die daraus resultierenden Rahmenbedingungen für das neue Backend zusammen.

Tabelle 3.3: Umfeld und Rahmenbedingungen

Umfeldelement	Beschreibung	Auswirkung auf das neue Backend
MariaDB	Alleiniger Datenspeicher (Nutzer, Gruppen, Permissions, Desktops)	Bleibt Datenspeicher; Schema muss versioniert migrierbar sein
SMTP-Relay	Versand von	Muss weiterhin HTML-E-Mails für

	Registrierungs-/Verifikations-E-Mails	dieselben Anlässe versenden
React-Frontend	Konsumiert Auth-/User-/Desktop-Endpunkte	Erwartet unveränderte REST-Verträge; CORS muss Frontend-Origin erlauben
Python-DaaS-Anwendung	Konsumiert Token-, Session- und Permissions-Endpunkte	Erwartet unveränderten Permissions-Vertrag inkl. 4-stufiger Kaskade
Docker/Docker Compose	Alleinige Betriebsform (Dev-/Testprofile)	Muss ebenfalls containerisiert und per Docker betreibbar sein
Linux-Zielplattform	Produktionsbetrieb auf Linux mit Apache	Muss ohne Anpassung auf vergleichbarer Linux-Infrastruktur laufen

Frontend und DaaS-Anwendung liegen außerhalb des Scopes dieser Arbeit (Kapitel 1, Abgrenzung) und werden nicht angepasst; jede hier genannte Rahmenbedingung ist daher eine feste, nicht verhandelbare Vorgabe.

3.2 Klärung der Anforderungen

Die Anforderungen wurden ausschließlich durch Quellcode-Analyse des alten PHP-Backends sowie durch die Ausschreibung ermittelt. Dieses Vorgehen entspricht einer *Reverse-Engineering*-Anforderungsanalyse: Statt Anforderungen aus Interviews mit Stakeholdern oder einem Pflichtenheft abzuleiten, werden sie aus dem beobachtbaren Verhalten des bestehenden Systems rekonstruiert.

3.2.1 Klärung der Hauptfunktionalität und Problemfeldabgrenzung

Die Hauptfunktionalität lässt sich aus der Modulstruktur des alten Systems (Abschnitt 3.1.2) in vier Bereiche gliedern, die das neue Backend unverändert anbieten muss:

1. **Authentifizierung:** Ausstellen, Erneuern, Widerrufen und Prüfen von Zugriffstoken (vormals OAuth2-Modul).

2. **Autorisierung:** Beantwortung der Frage „darf Nutzer X Funktion Y aufrufen?" über die 4-stufige Kaskade aus Abschnitt 3.1.2 (vormals Permissions-Modul).
3. **Benutzerverwaltung:** Selbstregistrierung mit E-Mail-Verifikation, administrative Verwaltung von Nutzern, Admins und Nutzergruppen sowie Anfragen einer Anwendungsfreischaltung (vormals User-Modul).
4. **Desktop-Inventarverwaltung**: Anlegen, Auflisten und Verknüpfen von Desktops und Desktop-Gruppen mit Nutzergruppen (vormals Desktop-Modul).

Nicht Teil dieser Arbeit ist die eigentliche Bereitstellung von Desktops – das Starten, Klonen oder Beenden von Containern und virtuellen Maschinen sowie die Steuerung von Apache Guacamole. Diese Aufgaben übernimmt ausschließlich die Python-DaaS-Anwendung. Das Backend stellt dafür nur die Stammdaten (welcher Desktop existiert, welcher Nutzergruppe ist er zugeordnet) sowie Authentifizierungs- und Berechtigungsauskünfte bereit, auf deren Basis die DaaS-Anwendung ihre eigenen Entscheidungen trifft. Ebenfalls außerhalb des Scopes liegen funktionale Änderungen an Frontend oder DaaS-Anwendung, die über die in Kapitel 1 genannten punktuellen Korrekturen hinausgehen.

3.2.2 Ableitung der funktionalen Anforderungen

Die meisten funktionalen Anforderungen (FA) entsprechen mindestens einem Endpunkt aus Tabelle 3.2 und sind unabhängig von der Implementierungssprache formuliert: Sie beschreiben, welche Funktion das System erfüllen muss, nicht, mit welcher Klasse oder welchem Framework dies geschieht. Eine Ausnahme (FA-3) hat kein Gegenstück im alten Backend; sie leitet sich aus den nicht-funktionalen Anforderungen in Abschnitt 3.2.3 ab.

Tabelle 3.4: Funktionale Anforderungen (FA)

ID	Anforderung
FA-1	Login mit Benutzername/Passwort liefert ein Zugriffstoken
FA-2	Ein abgelaufenes Zugriffstoken kann per Refresh-Token erneuert werden, ohne erneute Zugangsdaten
FA-3	Aktives Abmelden widerruft das Token serverseitig, auch vor Ablauf

FA-4	Zu einem gültigen Token kann die Session/Identität abgefragt werden
FA-5	Selbstregistrierung mit E-Mail-Bestätigung über zeitlich begrenzten Code
FA-6	Admin kann Nutzer auflisten, einsehen, ändern, aktivieren/deaktivieren
FA-7	Admin kann weitere Admin-Konten anlegen, auflisten, löschen
FA-8	Admin kann Nutzergruppen anlegen, ändern, auflisten und mit Desktop-Gruppe verknüpfen
FA-9	Nutzer kann Anwendungsfreischaltung anfragen; Admin wird per E-Mail benachrichtigt
FA-10	Admin kann Desktops anlegen, einsehen und auflisten
FA-11	Admin kann Desktop-Gruppen anlegen, einsehen und mit Nutzergruppen verknüpfen
FA-12	Nutzer kann prüfen, ob er Zugriff auf einen Desktop hat
FA-13	Prüfung, ob ein Nutzer eine Funktion aufrufen darf, über die 4-stufige Kaskade

FA-12 wird getrennt von FA-13 aufgeführt, da der zugehörige Endpunkt im User-Controller separat vom allgemeinen Permissions-Endpunkt implementiert ist und eine eigene, Desktop-Gruppen-basierte Prüflogik besitzt. FA-3 hat dagegen kein Gegenstück im Altsystem: Die Durchsicht von OAuth2HttpController und UserHttpController zeigt, dass das alte Backend keinen Logout-Endpunkt besitzt.

Da Frontend und DaaS-Anwendung unverändert bleiben sollen, ist für die Kompatibilität nicht entscheidend, welche Endpunkte das alte Backend anbietet, sondern welche davon die beiden Clients tatsächlich aufrufen. Deshalb wurde der Quellcode beider Clients (Stand: November 2026) nach Aufrufen des Backends durchsucht. Tabelle 3.5 stellt das Ergebnis den Endpunkten aus Tabelle 3.2 gegenüber und zeigt zugleich, wie das neue Backend mit dem jeweiligen Endpunkt umgeht.

Tabelle 3.5: Nutzung der Altschnittstelle durch Frontend und DaaS-Anwendung

Endpunkt (alt)	Aufgerufen von	Umsetzung im neuen Backend

POST /oauth2/user/token	Frontend, DaaS-Anwendung	übernommen
GET /oauth2/user/session	Frontend, DaaS-Anwendung	übernommen, Antwort um Feld role ergänzt
GET /users, PATCH /user/{id}	Frontend	übernommen
POST /user/{id}/enable, POST /user/{id}/disable	Frontend	als PATCH umgesetzt, Frontend angepasst
POST /user/validate_email	Frontend	übernommen
POST /user/request-application/{app}/{lang}	Frontend	übernommen
GET /admins, POST /admin/{id}	Frontend	übernommen, Anlegen als POST /admin
GET /desktops, GET /desktop/{id}, POST /desktop	Frontend	übernommen
GET /desktop-groups, POST /desktop-group	Frontend	als /desktop_groups und /desktop_group (Schreibweise des Frontends)
POST /desktop_group/{id}/user_group/{uId}	Frontend	übernommen
GET /user_groups, POST /user_group, PATCH /user_group/{id}	Frontend	übernommen
GET /permissions/{functionName}/{userId}	DaaS-Anwendung	übernommen
POST /permissions_info	– (bis Nov. 2026 DaaS-Anwendung)	übernommen
GET /version, GET /ping/user,	–	entfällt; Health-Check über

GET /ping/admin		GET /actuator/health
GET /user/{id}, DELETE /user/{id}	–	entfällt
GET /admin/{id}, DELETE /admin/{id}	–	entfällt
GET /desktop_group/{id}	–	entfällt
GET /users/access/{desktopId}	–	entfällt (FA-12)
POST /user_group/{ugId}/{dgId}	–	entfällt (gleichwertig zu POST /desktop_group/{id}/user_group/{uId})
PUT /endpoint_group/…, DELETE /endpoint_group/…	–	entfällt; Regeln werden direkt in der Datenbank gepflegt
neu: POST /user, POST /user/resend_code, POST /oauth2/user/logout	Frontend (Registrierung, Logout)	neu umgesetzt

Die Analyse zeigt, dass beide Clients nur einen Teil der alten Schnittstelle verwenden. Für das neue Backend wurde deshalb festgelegt, alle genutzten Endpunkte mit unveränderten Pfaden, Feldern und Statuscodes zu übernehmen und auf nicht genutzte Endpunkte zu verzichten, statt ungenutzte und damit auch ungetestete Funktionalität mitzuportieren. Abweichungen vom alten Vertrag beschränken sich auf die in Tabelle 3.5 genannten Fälle; ihre Auswirkungen auf FA-6, FA-7, FA-11, FA-12 und NFA-3 werden in Abschnitt 5.3 bewertet.

3.2.3 Ableitung der nicht-funktionalen Anforderungen

Die nicht-funktionalen Anforderungen (NFA) ergeben sich zum einen aus Eigenschaften, die das alte Backend bereits besitzt und die aus Kompatibilitätsgründen erhalten bleiben müssen (NFA-1 bis NFA-3), zum anderen aus generischen Sicherheits- und Betriebsanforderungen, die üblicherweise an einen Authentifizierungsdienst gestellt werden und deren Hintergrund in den Abschnitten 2.2 bis 2.4 erläutert wird (NFA-4 bis NFA-7), sowie aus Qualitätszielen, die

der Ausschreibung ausdrücklich entnommen sind: Das neue Backend soll, im Gegensatz zum alten, wart- und testbar sein (NFA-8 und NFA-9).

Tabelle 3.6: Nicht-funktionale Anforderungen (NFA)

ID	Anforderung
NFA-1	Fail-closed: Unbekannter Nutzer, unbekannte Funktion oder keine passende Regel führt immer zu Zugriffsverweigerung, nie zu impliziter Erlaubnis
NFA-2	Gleiche Priorität der vier Regelstufen (Nutzerregel vor Gruppenregel vor Endpunktgruppenregel vor Gruppen-Endpunktgruppenregel) sowie fail-closed als Standard
NFA-3	Gleiche Pfade, HTTP-Methoden, JSON-Felder und Statuscodes wie im alten Backend
NFA-4	Passwörter nur als Hash; Konten werden nach wiederholten Fehlversuchen gesperrt
NFA-5	Login und Registrierung sind ratenbegrenzt
NFA-6	Zugriffstoken sind serverseitig widerrufbar, obwohl JWTs grundsätzlich zustandslos sind
NFA-7	Dienst ist zustandslos und horizontal skalierbar; kein Server-Session-State außerhalb der DB
NFA-8	Interne Schichtung (*Controller* → *Service* → *Repository* → *Model*) wird automatisiert geprüft, nicht nur per Konvention
NFA-9	Mindest-Testabdeckung wird im Build erzwungen; fehlgeschlagener Build verhindert Mergen

Die Reihenfolge NFA-1 bis NFA-3 ist bewusst gewählt: Sie bilden die Bedingung, unter der das neue Backend als „funktional gleichwertig" zum alten gilt, und sind die Messlatte für die Evaluation in den Kapiteln 4 und 5. NFA-2 beschränkt sich bewusst auf Regelstufen-Reihenfolge und Default-Deny, da sich die DaaS-Anwendung laut Quellcode-Prüfung genau darauf verlässt. Wie in Abschnitt 3.1.2 gezeigt, löst der alte *PermissionCalculationService* mehrere widersprüchliche Gruppenregeln desselben Nutzers innerhalb einer Stufe nicht nach einer dokumentierten Regel auf, sondern nach der nicht spezifizierten internen Reihenfolge

der Gruppen-Collection; dieses unspezifizierte Verhalten wird daher in NFA-2 bewusst nicht als nachzubildende Anforderung übernommen. Stattdessen wird in Abschnitt 3.4.2 eine zusätzliche, im alten System nicht vorhandene nicht-funktionale Eigenschaft eingeführt: Die neue Kaskade löst mehrere gleichrangige, widersprüchliche Regeln innerhalb derselben Stufe deterministisch auf, indem ein expliziter Deny-Eintrag stärker wiegt als ein Allow-Eintrag.

3.3 Zeitliche Rahmenbedingungen

Dieser Abschnitt hält den Start- und Endzeitpunkt der Arbeit sowie die zugehörigen Meilensteine fest. Beginn und Ende der Bearbeitungszeit sind durch die Vorgaben des Prüfungsamts festgelegt. Die übrigen Meilensteine wurden davon ausgehend eigenständig entlang der Arbeitspakete AP1–AP6 aus der Ausschreibung geplant.

Tabelle 3.7: Zeitplan der Arbeit

Meilenstein	Zeitraum
AP1 – Recherche Gesamtsystem DESIGN	22.06.–29.06.2026
AP2 – Analyse altes PHP-Backend	30.06.–13.07.2026
AP3 – Entwurf neues Backend (Konzept/Architektur)	14.07.–03.08.2026
AP4 – Implementierung neues Backend (Java/Spring Boot)	04.08.–02.10.2026
AP6 – Test, Evaluation, Dokumentation	03.10.–28.10.2026
AP5 – Aufbau des Demonstrators	29.10.–05.11.2026
Literaturrecherche / Kapitel 2 (theoretischer Hintergrund)	22.06.–10.08.2026
Anfertigung der schriftlichen Ausarbeitung	22.06.–11.11.2026
Abgabe des vollständigen schriftlichen Entwurfs	11.11.2026
Ende der Bearbeitungszeit, Einreichen im Prüfungsamt	25.11.2026

AP1 und AP2 folgen unmittelbar aufeinander und sind inhaltlich eng verzahnt, da das Verständnis des Gesamtsystems DESIGN (AP1) und die Detailanalyse des alten Backends (AP2) in der Praxis nicht vollständig trennbar sind: Ohne die in Abschnitt 3.1 beschriebene Code-Analyse ließen sich die Rolle und Schnittstellen des Backends innerhalb von DESIGN

nicht einordnen. Die schriftliche Ausarbeitung sowie die Literaturrecherche zu Kapitel 2 laufen während der gesamten Bearbeitungszeit begleitend mit, da Erkenntnisse aus AP3–AP6 unmittelbar in Kapitel 4 und 5 einfließen. Der Demonstrator (AP5) wurde bewusst nach Test und Evaluation (AP6) eingeplant, damit er auf dem bereits getesteten Stand des Backends aufsetzt.

3.4 Zielzustand

Dieser Abschnitt beschreibt, wie das neue Backend die in Abschnitt 3.2 abgeleiteten Anforderungen erfüllen soll: zunächst den Architekturüberblick (3.4.1), danach das Authentifizierungs- und Autorisierungskonzept (3.4.2), Datenhaltung und Betrieb (3.4.3) sowie die Maßnahmen zur Qualitätssicherung (3.4.4). Die Anwendungsfälle für den Prototyp folgen in Abschnitt 3.5.

3.4.1 Architekturüberblick

Das Zielsystem ist ein eigenständiger Java-21-/Spring-Boot-Dienst mit dem Basispaket *de.frauas.design.backend*, der das gleiche Konzept vertikaler, fachlicher Module wie das alte Backend übernimmt (Abschnitt 3.1.2), jedoch mit automatisiert erzwungener statt nur konventioneller interner Schichtung. Die vier Module ***auth***, ***permission***, ***user*** und ***desktop*** entsprechen den vier Funktionsbereichen aus Abschnitt 3.2.1 und damit den alten Modulen *OAuth2*, *Permissions*, *User* und *Desktop*. Ergänzt werden sie durch drei Querschnittspakete: ***shared*** (modulübergreifend genutzte DTOs, Exceptions, Sicherheits-/Hilfsklassen), ***config*** (Spring-*@Configuration*-Klassen) und ***bootstrap*** (Aufgaben beim Anwendungsstart, z. B. Anlage eines Standard-Adminkontos). Jedes Modul ist intern gleich geschichtet in *controller* → *service* → *repository* → *model*, ergänzt um lokale *dto*- und *exception*-Unterpakete: eine direkte Übersetzung des alten Musters *Model* → *Repository* → *Service* → *Controller* in eine für Spring Boot idiomatische Reihenfolge. Anders als im alten Backend ist diese Schichtung nicht nur Konvention, sondern wird durch eine dedizierte ArchUnit-Testklasse automatisiert geprüft (Abschnitt 3.4.4).

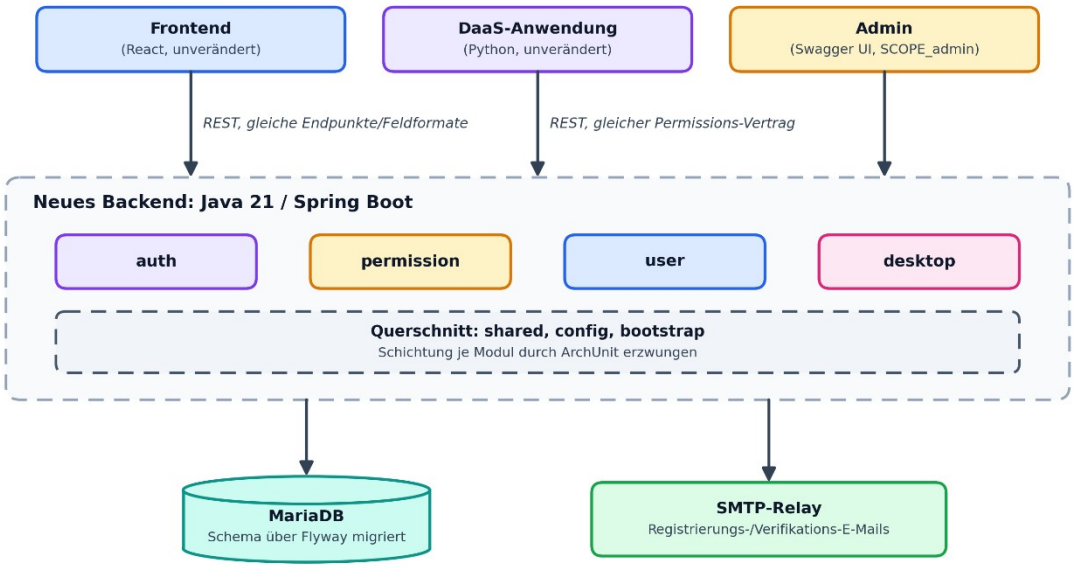


Abbildung 3.3: Zielzustand des neuen Backends (eigene Darstellung).

3.4.2 Authentifizierungs- und Autorisierungskonzept

Authentifizierung erfolgt zustandslos über RS256-signierte JSON Web Token (Kapitel 2.2). Ein eigener Token-Aussteller validiert Benutzername und Passwort bzw. ein Refresh-Token und stellt im Erfolgsfall ein JWT aus. Die Signatur wird über ein beim ersten Start erzeugtes RSA-Schlüsselpaar geprüft; ein Verlust dieses Schlüssels entwertet alle zuvor ausgestellten Tokens. Damit ein eigentlich zustandsloses Token dennoch serverseitig widerrufen werden kann (NFA-6), führt das Zielsystem zwei Datenbanktabellen (***access_tokens***, ***refresh_tokens***, Flyway-Migration *V1__init.sql*) mit je einem ***revoked***-Flag und einem ***revoked_at***-Zeitstempel; bei jeder Anfrage wird zusätzlich zur Signatur geprüft, ob das Zugriffstoken widerrufen wurde, und ein Refresh-Token wird bei seiner Einlösung entsprechend geprüft. ***access_tokens*** trägt zudem ein eindeutiges ***jti***-Feld, über das ein einzelnes Token gezielt widerrufen werden kann, ohne alle Tokens des Nutzers ungültig zu machen. Ein ***RateLimitingFilter*** (*shared.security.filter*, Token-Bucket-Algorithmus) begrenzt zusätzlich die Login- und Registrierungsversuche je Client-IP (NFA-5); die Felder ***failedLoginAttempts*** und ***lockedUntil*** auf der Nutzer-Entität sperren ein Konto nach wiederholten Fehlversuchen vorübergehend (NFA-4).

Autorisierung erfolgt zweistufig: grobgranular über OAuth2-Scopes (***SCOPE_user***/ ***SCOPE_admin***, geprüft durch Spring Security), feingranular über die 4-stufige, *fail-closed* Kaskade aus Abschnitt 3.1.2. Priorität und Fail-closed-Verhalten übernimmt das Zielsystem unverändert aus dem alten ***PermissionCalculationService*** (NFA-2). Für den Fall mehrerer gleichrangiger, widersprüchlicher Regeln innerhalb derselben Stufe – im alten Backend nicht spezifiziert (Abschnitt 3.2.3) – führt die neue ***PermissionService***-Klasse eine zusätzliche, deterministische Konfliktauflösung ein: Ein expliziter Deny-Eintrag wiegt stärker als ein Allow-Eintrag. Diese Regel ist eine bewusste Neuerung gegenüber dem alten Backend und schließt eine dort vorhandene Verhaltenslücke.

3.4.3 Datenhaltung und Betrieb

MariaDB dient, wie im alten Backend, als alleiniger Datenspeicher. Das Schema wird jedoch ausschließlich über versionierte Flyway-Migrationsskripte unter src/main*/resources/db/migration* verändert (*V1__init.sql* für das initiale Schema, gefolgt von inkrementellen Skripten wie *V2__registration_timestamps_to_instant.sql* bis V4__desktop_group_description_only.*sql*). Hibernate validiert die gemappten Entitäten beim Start nur gegen das bestehende Schema (***ddl-auto: validate***) und nimmt nie selbst Schemaänderungen vor. Ein SMTP-Relay versendet, wie im alten Backend, Registrierungs- und Verifikations-E-Mails sowie Benachrichtigungen über angefragte Anwendungsfreischaltungen (FA-9), hier auf Basis von Thymeleaf-HTML-Vorlagen. Der Dienst ist als einzelnes Docker-Image paketiert (Spring-Boot-Fat-JAR), über Umgebungsvariablen konfigurierbar (mit separaten Profilen *local*/*dev*/*prod*) und hält keinerlei Server-seitigen Sitzungszustand außerhalb der Datenbank bzw. des signierten Tokens selbst; mehrere Instanzen können daher hinter einem Load Balancer betrieben werden (NFA-7), sofern sie dasselbe RSA-Schlüsselpaar und dieselbe Datenbank teilen.

3.4.4 Qualitätssicherung

NFA-8 und NFA-9 fordern eine automatisierte Prüfung von Architektur und Testabdeckung; das Zielsystem setzt dies über zwei Build-Bausteine um. Eine ArchUnit-Testklasse

(***ArchitectureTest***) prüft bei jedem Build unter anderem, dass Controller nie direkt von anderen Controllern aufgerufen werden, dass Repositories nicht von Services abhängen, dass Entitäten (*model*) nur aus Service-, Repository-, Konfigurations- oder DTO-Code referenziert werden, dass Klassen gemäß ihrer Spring-Annotation (*@RestController*, *@Service*, *@Repository*) im passenden Unterpaket liegen und dass ausschließlich Konstruktor- statt Feld-Injektion verwendet wird. JaCoCo erzwingt zusätzlich eine Mindest-Zeilenabdeckung von 70 %. Beide Prüfungen sind, anders als beim alten Backend, verbindlicher Bestandteil des Builds: Bei einer Verletzung schlägt der Build fehl; in Verbindung mit einer CI-Pipeline lässt sich so das Zusammenführen fehlerhafter Änderungen verhindern – eine direkte technische Antwort auf die in der Ausschreibung kritisierte Qualität des Ist-Systems. Das alte Backend ist dabei nicht völlig ohne automatisierte Prüfung: Eine PHPat-Testklasse sichert die Trennung der Top-Level-Module (*Core*, *Migrations*, *Test*, *Main*) sowie die Isolation des *Util*-Moduls. Sie prüft jedoch nicht die hier relevante Schichtenrichtung *Controller → Service → Repository → Model* innerhalb eines fachlichen Moduls; genau diese Lücke schließt NFA-8 im neuen Backend.

3.5 Anwendungsfälle für den Prototyp

Zur Demonstration der Funktionalität (AP5, Abschnitt 4.7) werden acht Anwendungsfälle definiert, die die zentralen funktionalen Anforderungen aus Abschnitt 3.2.2 abdecken. Tabelle 3.8 gibt einen Überblick; die folgenden Beschreibungen legen für jeden Anwendungsfall Vorbedingung, Ablauf und Nachbedingung fest.

Tabelle 3.8: Anwendungsfälle für den Prototyp

#	Anwendungsfall	FA
1	Login per Password Grant, danach Berechtigungsprüfung durch die DaaS-Anwendung	FA-1, FA-13
2	Token-Erneuerung per Refresh-Token-Grant	FA-2
3	Logout mit serverseitigem Tokenwiderruf	FA-3
4	Selbstregistrierung inkl. E-Mail-Verifikation	FA-5

5	Admin legt Benutzer/Admin an und (de)aktiviert Konten	FA-6, FA-7, FA-8
6	Admin legt Desktop/Desktop-Gruppe an und verknüpft sie mit einer Nutzergruppe	FA-10, FA-11
7	DaaS-Anwendung fragt Zugriffsentscheidung für Funktion und Nutzer ab	FA-13
8	Nutzer fragt Freischaltung einer Anwendung an (E-Mail)	FA-9

AF-1: Login und Berechtigungsprüfung. *Vorbedingung:* Ein aktiviertes Nutzerkonto existiert. *Ablauf:* Der Nutzer meldet sich über das Frontend mit Benutzername und Passwort an; das Backend stellt ein Zugriffs- und ein Refresh-Token aus. Löst der Nutzer anschließend eine Aktion aus, die eine DaaS-Funktion betrifft, fragt die DaaS-Anwendung beim Backend, ob er diese Funktion aufrufen darf. *Nachbedingung:* Der Nutzer besitzt ein gültiges Token, und die DaaS-Anwendung erhält eine eindeutige Allow-/Deny-Antwort gemäß der 4-stufigen Kaskade. Abbildung 3.4 zeigt diesen Ablauf als Sequenzdiagramm.

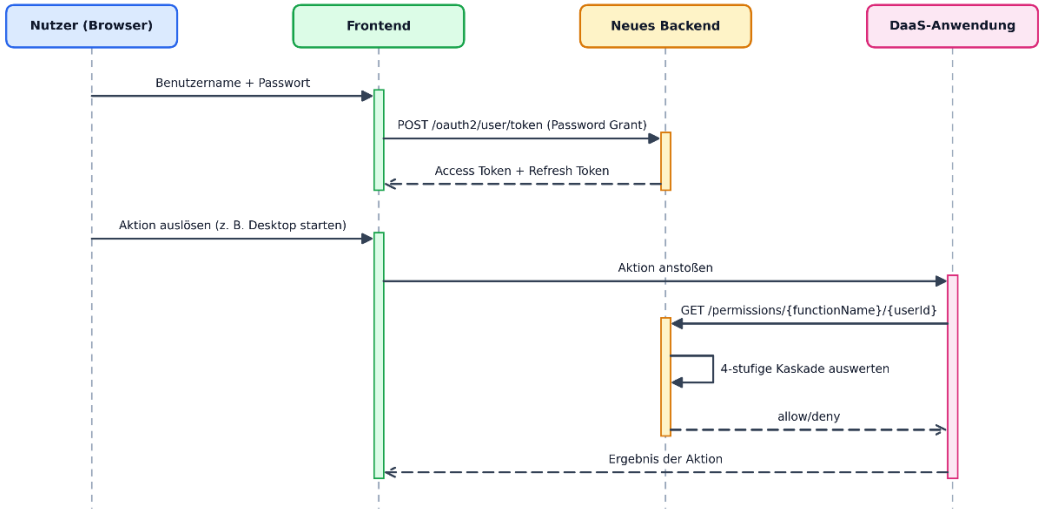


Abbildung 3.4: Login und Berechtigungsprüfung (AF-1) als Sequenzdiagramm (eigene Darstellung).

AF-2: Token-Erneuerung. *Vorbedingung:* Ein gültiges, nicht widerrufenes Refresh-Token liegt vor. *Ablauf:* Das Frontend sendet das Refresh-Token an *POST /oauth2/user/token* mit *grant_type=refresh_token*, ohne dass der Nutzer erneut Zugangsdaten eingeben muss. *Nachbedingung:* Ein neues Zugriffstoken und ein neues Refresh-Token werden ausgestellt; das verwendete Refresh-Token wird widerrufen.

AF-3: Logout. *Vorbedingung:* Ein gültiges Zugriffstoken liegt vor. *Ablauf:* Der Nutzer meldet sich über das Frontend ab (*POST /oauth2/user/logout*); das Backend markiert das zugehörige Token in der Widerrufsliste (Abschnitt 3.4.2) als widerrufen. *Nachbedingung:* Das Token wird von keinem nachfolgenden Request mehr akzeptiert, selbst wenn es formal noch nicht abgelaufen ist (NFA-6).

AF-4: Selbstregistrierung mit E-Mail-Verifikation. *Vorbedingung:* Es existiert noch kein Konto mit der angegebenen E-Mail-Adresse. *Ablauf:* Der Nutzer registriert sich über das Frontend (*POST /user*); das Backend legt ein deaktiviertes Konto mit zeitlich begrenztem Verifikationscode an und versendet diesen per E-Mail. Der Nutzer bestätigt den Code (*POST /user/validate_email*) oder fordert bei Bedarf einen neuen Code an (*POST /user/resend_code*). *Nachbedingung*: Das Konto ist aktiviert und kann sich anschließend wie in AF-1 anmelden.

AF-5: Administrative Verwaltung von Nutzern, Admins und Nutzergruppen. *Vorbedingung:* Ein angemeldeter Administrator (Scope *admin*) ist vorhanden. *Ablauf:* Der Administrator legt über das Frontend neue Nutzer oder weitere Administratoren an, ordnet Nutzer Nutzergruppen zu und aktiviert bzw. deaktiviert Konten bei Bedarf. *Nachbedingung:* Die Nutzer-, Admin- und Gruppendaten sind konsistent in der Datenbank gepflegt und wirken sich unmittelbar auf nachfolgende Berechtigungsprüfungen (AF-7) aus.

AF-6: Anlegen von Desktops und Desktop-Gruppen. *Vorbedingung:* Ein angemeldeter Administrator ist vorhanden. *Ablauf:* Der Administrator legt einen Desktop sowie eine Desktop-Gruppe an und verknüpft diese Desktop-Gruppe mit einer bestehenden Nutzergruppe. *Nachbedingung:* Alle Mitglieder der verknüpften Nutzergruppe gelten gemäß der in Abschnitt 3.1.2 beschriebenen Zuordnungskette (*Nutzer → Nutzergruppe → Desktop-Gruppe → Desktop*) fortan als zugriffsberechtigt für diesen Desktop.

AF-7: Berechtigungsabfrage durch die DaaS-Anwendung. *Vorbedingung:* Die DaaS-Anwendung besitzt ein gültiges Admin-Token. *Ablauf:* Die DaaS-Anwendung fragt für eine

design-daas-Funktion und einen Nutzer die Zugriffsentscheidung ab (*GET /permissions/{functionName}/{userId}* oder *POST /permissions_info*); das Backend durchläuft die 4-stufige Kaskade aus Abschnitt 3.1.2 und liefert eine eindeutige Allow-/Deny-Antwort. *Nachbedingung:* Die DaaS-Anwendung führt die angefragte Aktion nur bei einer Allow-Antwort aus; bei jedem unklaren Fall (unbekannter Nutzer, unbekannte Funktion, keine passende Regel) wird aufgrund von NFA-1 verweigert.

AF-8: Anfrage einer Anwendungsfreischaltung. *Vorbedingung:* Ein angemeldeter Nutzer ist vorhanden. *Ablauf:* Der Nutzer fragt über das Frontend die Freischaltung einer Anwendung an (*POST /user/request-application/{application}/{isoCode}*); das Backend versendet eine Benachrichtigungs-E-Mail an die zuständigen Administratoren. *Nachbedingung:* Die Administratoren sind informiert und können die Freischaltung über AF-5/AF-6 vornehmen.

1.0

4 Realisierung

Dieses Kapitel beschreibt, wie die in Kapitel 3 abgeleiteten funktionalen (FA) und nicht-funktionalen (NFA) Anforderungen im neuen Backend *design-daas-backend-java* umgesetzt wurden. Die Gliederung folgt den vier fachlichen Modulen aus Abschnitt 3.4 (*auth*, *permission*, *user*, *desktop*), ergänzt um die Architekturentscheidungen, die diesen Modulen vorausgehen (4.1), die Querschnittsthemen, die sie verbinden (4.6), sowie den Demonstrator (4.7) und die abschließende Evaluation anhand der Testsuite (4.8). Alle Codeausschnitte sind kurze, teilweise gekürzte Auszüge aus dem Repository *design-daas-backend-java* (Version 2.0.0); Vergleiche mit dem alten Backend beziehen sich auf *design-daas-application-backend*.

4.1 Architekturentscheidungen

Bevor im Folgenden die einzelnen Fachmodule im Detail beschrieben werden, klärt dieser Abschnitt die übergreifenden Architekturentscheidungen, die allen Modulen zugrunde liegen: die Gliederung des Quellcodes (4.1.2), ihre automatisierte Durchsetzung (4.1.3), die Wahl des Authentifizierungsmechanismus (4.1.4) und die Schemaverwaltung (4.1.5). Abschnitt 4.1.6 fasst diese Entscheidungen tabellarisch zusammen und ordnet sie den Anforderungen aus Kapitel 3 zu, bevor die Abschnitte 4.2–4.6 sie anhand von Code vertiefen.

4.1.1 Ausgangslage und Entscheidungsrahmen

Die in Kapitel 3 abgeleiteten Anforderungen verlangen zweierlei gleichzeitig: Erstens muss das neue Backend **verhaltenskompatibel** zum alten PHP-Backend sein (NFA-2, NFA-3), da Frontend und DaaS-Anwendung **nicht** Teil dieser Arbeit sind und **unverändert** weiterlaufen müssen. Zweitens soll die Neuimplementierung gerade die Mängel des alten Backends beheben – mangelnde Wartbarkeit und fehlende Dokumentation (vgl. Ausschreibung, Kapitel

1). Diese beiden Ziele bestimmen die zentralen Architekturentscheidungen dieses Kapitels: Jede Entscheidung wird daher im Folgenden nicht nur beschrieben, sondern auch begründet, gegen welche der beiden Zielsetzungen sie einzahlt.

4.1.2 Modulare Gliederung nach fachlicher Domäne

Das alte PHP-Backend ist bereits vertikal in Module gegliedert (Verzeichnisse *src/Main*, *src/Util*, *src/OAuth2*, *src/User*, *src/Desktop*, *src/Permissions*), jedes mit der internen Schichtung *Model → Repository → Service → Controller*. Diese Gliederung wurde für das neue Backend übernommen, jedoch umbenannt und auf vier fachliche Pakete konzentriert, die den vier in Abschnitt 3.2.1 beschriebenen Funktionsbereichen entsprechen:

```
de.frauas.design.backend
├── auth       (Token-Ausgabe/-Prüfung, Sessions, Lockout)
├── permission (Berechtigungskaskade)
├── user       (Nutzer, Admins, User-Groups, Registrierung)
├── desktop    (Desktops, Desktop-Groups)
├── bootstrap  (Start-Tasks, z. B. Default-Admin)
├── config     (Spring-Konfiguration)
└── shared     (DTOs, Exceptions, Security, Utils)
```

Code 4.1: Paketdiagramm der vier Fachmodule und des Querschnittscodes

Jedes fachliche Paket folgt intern derselben Schichtung *controller → service → repository → model* (ergänzt um lokale *dto*- und *exception*-Unterpakete). Diese Entscheidung wurde getroffen, weil sie die bestehende, aus dem PHP-Backend bekannte Struktur beibehält (geringeres Risiko von Bedeutungsverschiebungen beim Portieren fachlicher Logik) und gleichzeitig eine im Java-/Spring-Ökosystem übliche, leicht nachvollziehbare Schichtenarchitektur ergibt.

4.1.3 Automatisierte Durchsetzung der Architektur mit ArchUnit

Die größte dokumentierte Schwäche des alten Backends ist mangelnde Wartbarkeit. Um zu verhindern, dass die neue Architektur über die Projektlaufzeit informell "erodiert" (z. B. ein Repository, das direkt aus einem Controller aufgerufen wird), wird die Schichtenregel nicht nur dokumentiert, sondern als ausführbarer Test formuliert und bei jedem Build geprüft:

```
@ArchTest
static final ArchRule layerDependencies = layeredArchitecture()
      .consideringAllDependencies()
      .layer("Controllers").definedBy("de.frauas.design.backend..controll
er..")
      .layer("Services").definedBy("de.frauas.design.backend..service..")
      .layer("Repositories").definedBy("de.frauas.design.backend..reposit
ory..")
      .whereLayer("Controllers")
            .mayNotBeAccessedByAnyLayer()
      .whereLayer("Services")
            .mayOnlyBeAccessedByLayers("Controllers", "Config",
      "Services", "Bootstrap")
      .whereLayer("Repositories")
            .mayOnlyBeAccessedByLayers("Services", "Config",
      "Bootstrap");
```

Code 4.2: ArchUnit-Regel zur Schichtenarchitektur (gekürzt)

Zusätzliche Regeln in derselben Testklasse (*ArchitectureTest*) erzwingen Namenskonventionen (*@Service*-Klassen müssen auf *Service/ServiceImpl* enden, *@Repository*-Klassen auf *Repository* usw.), Konstruktorinjektion statt Feldinjektion in Services, ein Verbot von *java.util.logging/System.out*, sowie dass *shared* von keinem Fachmodul abhängen darf (Vermeidung zirkulärer Kopplung). Insgesamt umfasst *ArchitectureTest* 19 einzelne, bei jedem Build ausgeführte Regeln. Ein Verstoß gegen eine

dieser Regeln lässt *mvn test* fehlschlagen – die Architektur ist damit nicht nur eine Absicht, sondern ein Build-Gate. Dies ist die direkte technische Antwort auf die in der Ausschreibung genannte mangelnde Qualität des Vorgängersystems und setzt NFA-8 ("Interne Schichtung wird automatisiert geprüft, nicht nur per Konvention") unmittelbar um.

Bemerkenswert ist, dass auch das alte PHP-Backend bereits ein vergleichbares Werkzeug einsetzt: Die Testklasse *BasicArchitectureTest.php* definiert mittels *PHPat* drei Regeln, die den Kern-Namensraum *Application\Core* vor Zugriffen von außerhalb isolieren und *Application\Migrations* / *Application\Test* als jeweils eigenständige, nicht querverlinkte Namensräume absichern. Die Neuerung liegt also nicht darin, Architekturtests grundsätzlich einzuführen, sondern darin, sie erstens auf die interne Schichtung jedes Moduls auszuweiten (*Controller → Service → Repository*, im PHP-Code nur durch Konvention, nicht durch *PHPat*-Regeln gesichert) und zweitens mit einer harten Testabdeckungsschranke zu kombinieren (Abschnitt 4.8.1) – beides zusammen ergibt eine deutlich engmaschigere automatisierte Qualitätssicherung als im Ist-Zustand.

4.1.4 Zustandslose Authentifizierung statt vollem Autorisierungsserver

Das alte Backend verwendet *league/oauth2-server* mit einem Password-Grant. Für das neue Backend wurde geprüft, ob ein vollständiger *spring-security-oauth2-authorization-server* eingesetzt werden soll. Da nur zwei Grant-Typen benötigt werden (*password*, *refresh_token*) und *POST /oauth2/user/token* Zugangsdaten direkt per Formularparameter entgegennimmt – wie es der alte Vertrag mit dem Frontend vorsieht –, wurde stattdessen eine schlanke Eigenimplementierung gewählt: ein selbst gebauter RS256-JWT-Aussteller (*TokenIssuer*) in Kombination mit Spring Securitys OAuth2-Resource-Server-Validierung (*JwtDecoder*). Diese Entscheidung vermeidet den Konfigurationsaufwand eines vollständigen Autorisierungsservers für einen Vertrag, der ohnehin nur zwei Grant-Typen abdeckt. Dabei wird bewusst in Kauf genommen, dass RFC 9700 vom Password Grant abrät (Abschnitt 2.2.1); er wurde ausschließlich beibehalten, weil Frontend und DaaS-Anwendung ihn verwenden (vgl. Abschnitt 5.4).

4.1.5 Schema ausschließlich über Flyway

Hibernate wird im Modus ***ddl-auto: validate*** betrieben; es erzeugt zu keinem Zeitpunkt Schema automatisch. Alle Schemaänderungen laufen über versionierte Flyway-Migrationen (*V1__init.sql … V4__*.sql*). Diese Entscheidung macht jede Schemaänderung explizit review- und nachvollziehbar und vermeidet Hibernate-Auto-DDL-Überraschungen, die zwischen Entwicklungs-, Test- und Produktivumgebung unterschiedlich ausfallen könnten – ein Qualitätsziel, das direkt auf die Wart- und Testbarkeitsanforderungen NFA-8/NFA-9 (Kapitel 3.2) einzahlt und zugleich das Pendant zu den Doctrine Migrations des alten Backends bildet (Tabelle 3.1), sodass auch hier kein Werkzeug neu eingeführt, sondern dessen Funktion 1:1 ersetzt wird.

4.1.6 Zusammenfassung der Architekturentscheidungen

Tabelle 4.1 fasst die in 4.1.2–4.1.5 begründeten Entscheidungen zusammen und ordnet sie jeweils einer Anforderung aus Kapitel 3 zu. Die Tabelle dient als Scharnier zwischen Anforderungsanalyse und Realisierung: Jede Zeile wird in den nachfolgenden Abschnitten 4.2–4.6 anhand von Code vertieft.

Tabelle 4.1: Zentrale Architekturentscheidungen und ihr Anforderungsbezug

Entscheidung	Begründung	NFA
Vier fachliche Module, intern einheitlich geschichtet	Struktur des alten Backends beibehalten, Portierungsrisiko senken	NFA-2, NFA-3
Schichtenregeln als ArchUnit-Build-Gate	Architektur nicht nur Konvention, sondern erzwungen	NFA-8
Eigener RS256-JWT-Aussteller statt vollem Autorisierungsserver	Nur zwei Grant-Typen nötig, geringerer Konfigurationsaufwand	NFA-7
Flyway statt Hibernate-Auto-DDL	Nachvollziehbare, reviewbare Schemaänderungen	NFA-8, NFA-9
JaCoCo-Mindestabdeckung als	Testabdeckung ist kein optionales Ziel,	NFA-9

Build-Gate	sondern Build-Voraussetzung	

4.2 Authentifizierung

Das Fachmodul *auth* setzt die in Kapitel 3.2 beschriebenen Authentifizierungs-Anwendungsfälle FA-1 bis FA-4 um: Login per Benutzername/Passwort (FA-1), Token-Erneuerung (FA-2), Logout (FA-3) und Session-Introspektion (FA-4). Die folgenden Abschnitte folgen dabei bewusst dem tatsächlichen Kontrollfluss einer Anfrage: vom Dispatch der Grant-Art (4.2.1) über die eigentliche Passwort-Prüfung mit ihren Sicherheitsmaßnahmen (4.2.2), die Ausstellung der Tokens (4.2.3) und deren Erneuerung (4.2.4), bis zu Logout/Introspektion (4.2.5) und den beiden Querschnittsmaßnahmen Ratenbegrenzung (4.2.6) und deren nebenläufigkeitssichere Implementierung (4.2.7).

4.2.1 Grant-Dispatch

TokenController nimmt *POST /oauth2/user/token* entgegen und baut daraus ein typisiertes, sealed *GrantRequest* (entweder *PasswordGrantRequest* oder *RefreshGrantRequest*). Die eigentliche Geschäftslogik liegt in *TokenService.grantToken*, das über ein *Pattern-Matching-switch* auf die konkrete Variante verzweigt:

```
@Transactional(noRollbackFor = TokenGrantException.class)
public TokenResponseDto grantToken(GrantRequest request) {
    return switch (request) {
        case PasswordGrantRequest r
            -> passwordGrant(r.username(), r.password(), r.scope());
        case RefreshGrantRequest r
            -> refreshGrant(r.refreshToken());
    };
```

Code 4.3: Grant-Dispatch in TokenService.grantToken

Da *GrantRequest* als *sealed interface* modelliert ist, prüft der Java-Compiler, dass dieses *switch* erschöpfend ist: Würde ein dritter Grant-Typ ergänzt, ohne hier einen Fall hinzuzufügen, würde der Code nicht mehr kompilieren. Dies ersetzt eine sonst nötige

Testabdeckung "wurde der neue Grant-Typ auch im Dispatch berücksichtigt?" durch eine vom Compiler erzwungene Eigenschaft – ein Beispiel dafür, wie moderne Java-21-Sprachmittel zur Wartbarkeit beitragen, ohne dass dies im alten PHP-Code (keine vergleichbare Sprachunterstützung) möglich gewesen wäre. Der Token-Endpunkt akzeptiert gemäß RFC 6749 ausschließlich formularkodierte Parameter (application/x-www-form-urlencoded); Anfragen mit JSON-Rumpf werden mit dem Statuscode 415 abgelehnt.

4.2.2 Password Grant: Ablauf und Sicherheitsmaßnahmen

TokenService.passwordGrant setzt mehrere Sicherheitsmaßnahmen um, die über die reine OAuth2-Spezifikation hinausgehen. Zunächst wird geprüft, ob überhaupt ein Nutzer mit dem angegebenen Benutzernamen existiert:

```
var userOpt = userRepository.findByEmail(username);

if (userOpt.isEmpty()) {
    // always run BCrypt to prevent timing-based user enumeration
    passwordEncoder.matches(password, dummyHash());
    log.warn("passwordGrant — unknown user={}",
LogMasking.maskEmail(username));
    throw new InvalidCredentialsException();
```

Code 4.4: Schutz vor zeitbasierter Benutzer-Enumeration

Ist der Benutzername unbekannt, wird trotzdem ein BCrypt-Vergleich gegen einen Dummy-Hash ausgeführt, der beim Start mit demselben PasswordEncoder und damit mit demselben Kostenfaktor erzeugt wird wie echte Passwort-Hashes. Ohne diese Zeile würde eine Anfrage mit unbekanntem Benutzernamen messbar schneller fehlschlagen als eine mit bekanntem Benutzernamen und falschem Passwort (BCrypt ist absichtlich langsam), wodurch ein Angreifer per Zeitmessung gültige Benutzernamen enumerieren könnte. Diese Maßnahme war im alten PHP-Backend nicht dokumentiert und ist eine bewusste Verbesserung gegenüber dem Ist-Zustand (NFA-4).

Anschließend prüft *AccountLockoutService.assertNotLocked*, ob das Konto gerade gesperrt ist, bevor das Passwort überhaupt verglichen wird. Schlägt der Passwortvergleich fehl, zählt

AccountLockoutService.recordFailedAttempt die Fehlversuche und sperrt das Konto nach fünf aufeinanderfolgenden Fehlversuchen für 15 Minuten:

```
private static final int MAX_FAILED_ATTEMPTS = 5;
private static final Duration LOCKOUT_DURATION = Duration.ofMinutes(15);

public void recordFailedAttempt(BaseUser user) {
    int attempts = user.getFailedLoginAttempts() + 1;
    user.setFailedLoginAttempts(attempts);
    if (attempts >= MAX_FAILED_ATTEMPTS) {
        user.setLockedUntil(Instant.now(clock).plus(LOCKOUT_DURATION));
    }
    userRepository.save(user);
}
```

Code 4.5: Zählung von Fehlversuchen und Kontosperrung

Bemerkenswert ist die im Quellcode-Javadoc dokumentierte Randbedingung: Der Fehlversuchszähler wird nur bei erfolgreichem Login zurückgesetzt (*resetOnSuccess*), nicht automatisch nach Ablauf der Sperrzeit. Läuft die Sperre ab und der nächste Versuch schlägt erneut fehl, wird das Konto sofort wieder für die volle Dauer gesperrt, da der Zähler nie heruntergezählt wurde. Dieses Verhalten ist bewusst fail-closed gewählt; seine betrieblichen Implikationen werden in Abschnitt 4.8.4 als bekannte Einschränkung eingeordnet.

Nach bestandener Sperr- und Passwortprüfung wird zunächst geprüft, ob das Konto aktiviert ist; anschließend wird der angeforderte Scope gegen die Rolle des Nutzers validiert (*ScopeResolver.resolveAuthorizedScope*), um Scope-Eskalation zu verhindern (ein regulärer Nutzer könnte sonst scope=admin anfragen). Erst danach wird ein Token ausgestellt. Abbildung 3.4 zeigt den übergeordneten Ablauf aus Sicht der beteiligten Systeme.

4.2.3 Tokenausstellung: RS256-JWT und opaker Refresh-Token

TokenIssuer.issueAccessToken baut ein RS256-signiertes JWT mit eindeutiger ***jti*** (JWT-ID) und persistiert diese *jti* zusätzlich in der Tabelle *access_tokens*:

```
JwtClaimsSet claims = JwtClaimsSet.builder()
        .id(jti)
        .issuer(issuer)
        .subject(String.valueOf(user.getId()))
        .issuedAt(now)
        .expiresAt(now.plusSeconds(accessTokenTtlSeconds))
        .claim("email", user.getEmail())
        .claim("name", user.getName())
        .claim("scope", scope)
        .build();

JwsHeader header = JwsHeader.with(SignatureAlgorithm.RS256).build();
String tokenValue = jwtEncoder.encode(JwtEncoderParameters.from(header,
claims)).getTokenValue();
```

Code 4.6: Aufbau und Signatur des Zugriffstokens

Die Wahl von **RS256** (asymmetrisch, privater Schlüssel signiert, öffentlicher Schlüssel validiert) statt eines symmetrischen Verfahrens wie HS256 bedeutet, dass der öffentliche Schlüssel verteilt werden könnte, ohne das Signier-Geheimnis preiszugeben – relevant, falls künftig ein separater Dienst nur Tokens validieren, aber nicht ausstellen soll. Das Schlüsselpaar wird beim ersten Start erzeugt und unter *OAUTH_KEYS_DIR* persistiert; ein Verlust dieser Datei entwertet alle zuvor ausgestellten Tokens. Zugriffstoken sind standardmäßig 3600 Sekunden gültig, Refresh-Tokens 30 Tage; beide Werte sind über Umgebungsvariablen konfigurierbar.

Da JWTs normalerweise zustandslos und damit serverseitig nicht widerrufbar sind, speichert das Backend zusätzlich jede ausgestellte *jti* in der Datenbank (*access_tokens*) mit einem *revoked*-Flag – ein bewusster Kompromiss zwischen Zustandslosigkeit (NFA-7) und Widerrufbarkeit (NFA-6, Logout). Der Refresh-Token selbst ist dagegen kein JWT, sondern ein zufälliger 256-Bit-Hex-String (*SecureRandom*), da er nur als Datenbankschlüssel dient und nicht selbst Ansprüche transportieren muss:

```
private static final SecureRandom SECURE_RANDOM = new SecureRandom();
...
byte[] bytes = new byte[32];
SECURE_RANDOM.nextBytes(bytes);
String tokenValue = HexFormat.of().formatHex(bytes);
```

Code 4.7: Erzeugung des opaken Refresh-Tokens

4.2.4 Refresh-Token-Rotation

Beim Refresh-Grant wird nicht einfach ein neuer Access-Token zum bestehenden Refresh-Token ausgestellt; stattdessen wird bei jeder Erneuerung ein neuer Refresh-Token erzeugt und der alte widerrufen (Token-Rotation, *TokenService.refreshGrant*):

```
String newAccessToken = tokenIssuer.issueAccessToken(user, grantedScope);
String newRefreshToken = tokenIssuer.createRefreshToken(user,
grantedScope);

refreshTokenEntity.revoke();
refreshTokenRepository.save(refreshTokenEntity);
```

Code 4.8: Refresh-Token-Rotation in TokenService.refreshGrant

Dies begrenzt den Schaden, falls ein Refresh-Token abgefangen wird: Jeder Refresh-Token kann nur einmal eingelöst werden; ein bereits verwendeter Token wird abgelehnt, da er in der Datenbank als widerrufen markiert ist. Eine Erkennung der Wiederverwendung mit anschließendem Widerruf aller vom selben Login abgeleiteten Tokens, wie sie RFC 9700 empfiehlt (Lodderstedt et al., 2025), ist dagegen nicht umgesetzt (vgl. Abschnitt 5.5). Zusätzlich validiert *refreshGrant* vor jeder Erneuerung erneut, dass der zugrunde liegende Nutzer noch existiert und aktiviert ist (*userOpt.get().isEnabled()*); andernfalls wird der Refresh-Token gelöscht und die Erneuerung abgelehnt. Damit kann ein deaktiviertes Konto keinen neuen Access-Token mehr erhalten, selbst wenn ein zuvor ausgestellter Refresh-Token noch formal gültig wäre. Das zuvor ausgestellte Zugriffstoken bleibt dagegen bis zu seinem Ablauf gültig.

4.2.5 Logout und Session-Introspektion

POST /oauth2/user/logout ruft *TokenRevocationService.logout* auf, das sowohl die *jti* des aktuellen Access-Tokens als auch — sofern mitgegeben — den Refresh-Token als widerrufen markiert (FA-3). Dieser Pfad hat bewusst kein Gegenstück im alten PHP-Backend (vgl. Abschnitt 3.2.2): Da *jti*-Widerruf ein zusätzlicher, selbst gewählter Mechanismus ist (4.2.3), musste auch die zugehörige Logout-Funktionalität neu entworfen werden, statt lediglich eine bestehende Verhaltensweise zu übernehmen.

GET /oauth2/user/session erlaubt dem Frontend, den aktuellen Anmeldestatus zu prüfen, ohne eigene Token-Parsing-Logik implementieren zu müssen (*JwtUserResolver* extrahiert die Nutzer-ID aus dem validierten JWT-Principal, FA-4). Die Antwort enthält neben Nutzer-ID und Name auch die aktuelle Rolle des Nutzers, die bei jeder Abfrage aus der Datenbank gelesen wird; eine Rollenänderung wirkt sich damit auch auf bereits ausgestellte Tokens aus. Die DaaS-Anwendung nutzt denselben Endpunkt, um das Bearer-Token eines Nutzers zu prüfen.

4.2.6 Ratenbegrenzung sensibler Endpunkte

Unabhängig von Kontosperren begrenzt *RateLimitingFilter* die Anzahl Anfragen pro Client-IP für */oauth2/user/token*, */user* (Registrierung), */user/validate_email* und */user/resend_code* (NFA-5). Der Filter ist mit *@Order(-200)* so positioniert, dass er vor der Spring-Security-Filterkette *(-100)* läuft – überhöhte Anfragen werden also bereits abgewiesen, bevor überhaupt Authentifizierungslogik ausgeführt wird:

```
@Component
@Order(-200)
public class RateLimitingFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
HttpServletResponse response, FilterChain filterChain) throws
ServletException, IOException {
        String path = request.getServletPath();
        String clientIp = clientIpResolver.resolve(request);
        int limit = limitFor(path);

        if (rateLimiter.tryConsume(clientIp, path, limit)) {
            filterChain.doFilter(request, response);
            return;
        }
        response.setStatus(429);
        response.setHeader("Retry-After", "60");
        // ... JSON error body
    }
}
```

Code 4.9: RateLimitingFilter (gekürzt)

Die Client-IP wird über *ClientIpResolver* ermittelt, der *X-Forwarded-For* nur dann auswertet, wenn die Anfrage von einer konfigurierten, vertrauenswürdigen Proxy-Adresse stammt (*RATE_LIMIT_TRUSTED_PROXIES*) – andernfalls könnte ein Angreifer die Ratenbegrenzung durch Fälschen dieses Headers umgehen. Die vier Limits (*auth-token*, *register*, *validate-email*, *resend-code*) sind getrennt konfigurierbar (*application.yml*, Default 20/5/10/3 Anfragen pro Minute), da sich Missbrauchsszenarien unterscheiden: Ein Login-Endpunkt wird legitim häufiger aufgerufen als ein Code-Erneuerungs-Endpunkt, der im Normalbetrieb nur selten benötigt wird.

4.2.7 Sperrenfreie Implementierung der Ratenbegrenzung

Die eigentliche Zählung pro Client-IP und Pfad erfolgt in *TokenBucket*, einer intern von *RateLimiter* verwalteten Klasse, die pro Schlüssel (IP + Pfad) genau eine Instanz hält. Da mehrere Anfragen vom selben Client gleichzeitig eintreffen können, muss diese Zählung nebenläufigkeitssicher sein. Anstatt dafür eine *synchronized*-Sperre zu verwenden, wird der

Zustand (letzter Auffüllzeitpunkt und verbleibende Token) in ein einziges *long* gepackt und per *AtomicLong.compareAndSet* aktualisiert:

```
boolean tryConsume() {
    while (true) {
        long current = state.get();
        long lastRefillSeconds = current >>> 32;
        long storedTokens = current & 0xFFFFFFFFL;

        long elapsedMillis = Math.max(0, nowMillis - lastRefillSeconds *
1_000L);
        long refilledTokens = (long) (elapsedMillis *
refillTokensPerMillisecond);
        long availableTokens = Math.min(capacity, storedTokens +
refilledTokens);

        if (availableTokens <= 0) {
            return false;
        }
        long nextState = (nowSeconds << 32) | (availableTokens - 1);
        if (state.compareAndSet(current, nextState)) {
            return true;
        }
        // CAS failed: another thread updated state concurrently, retry
    }
}
```

Code 4.10: Sperrenfreie Aktualisierung in TokenBucket.tryConsume (gekürzt)

Diese verlustfreie Codierung von Zeitstempel und Tokenzahl in den oberen bzw. unteren 32 Bit eines einzelnen *long*-Werts vermeidet, dass zwei separate Felder inkonsistent gelesen werden könnten, und erspart eine explizite Sperre für einen Codepfad, der bei jeder Anfrage an einen der vier rate-limitierten Endpunkte durchlaufen wird. Schlägt der *compareAndSet*-Aufruf fehl (ein anderer Thread hat den Zustand zwischenzeitlich geändert), wird die Schleife einfach wiederholt (optimistische Nebenläufigkeitskontrolle), statt auf eine Sperre zu warten. Die Implementierung besitzt eine im Code dokumentierte Einschränkung: Da jeder erfolgreiche Aufruf den Auffüllzeitpunkt auf die aktuelle, auf ganze Sekunden gerundete Zeit setzt und Bruchteile von Token verwirft, füllt sich der Eimer bei gleichmäßigem Verkehr etwas langsamer auf als konfiguriert. Für die Missbrauchsabwehr ist das unkritisch, da es nur zu einer strengeren Begrenzung führt. Zudem werden die Zähler im Arbeitsspeicher der

jeweiligen Instanz gehalten, sodass die Limits beim Betrieb mehrerer Instanzen je Instanz gelten (vgl. Abschnitt 5.4).

4.3 Autorisierung / Berechtigungskaskade

Während Abschnitt 4.2 klärt, wer ein Nutzer ist (Authentifizierung), entscheidet das Fachmodul permission, was dieser Nutzer tun darf. Es implementiert FA-13 ("Berechtigungsprüfung pro DaaS-Funktion") und ist damit die sicherheitskritischste Komponente des Backends, da die Python-DaaS-Anwendung vor jedem Funktionsaufruf genau diese Entscheidung abfragt. Die folgenden Abschnitte beschreiben zunächst die Portierung der eigentlichen Kaskadenlogik (4.3.1) und zwei Detailfragen, die im alten Code nicht abschließend geklärt waren: Konflikte innerhalb einer Kaskadenstufe (4.3.2) und der Umgang mit fehlerhafter Eingabe (4.3.3). Abschnitt 4.3.4 erläutert eine Performance-Optimierung, Abschnitt 4.3.5 die resultierende Testabdeckung.

4.3.1 Portierung der PHP-Logik

Das alte Backend entscheidet über *PermissionCalculationService::getPermission()* nach einer vierstufigen Prioritätsliste. Der tatsächliche PHP-Quellcode (*PermissionCalculationService.php*) zeigt diese vier Stufen als eine Folge von frühen Rückgaben: zunächst eine direkte Nutzer-Endpunkt-Regel, danach — innerhalb einer Schleife über alle Gruppen des Nutzers — eine Nutzergruppen-Endpunkt-Regel, danach eine Nutzer-Endpunktgruppen-Regel und schließlich, wieder innerhalb einer Gruppen-Schleife, eine Nutzergruppen-Endpunktgruppen-Regel, mit ***AccessEnum::DENY*** als Rückfall, falls keine Regel greift. Diese exakte Reihenfolge musste erhalten bleiben, da die Python-DaaS-Anwendung ihre Berechtigungsprüfungen unverändert gegen das neue Backend stellt (NFA-2/NFA-3). Die neue *PermissionService.checkPermission*-Methode bildet die vier Stufen 1:1 nach, jedoch mit *Optional#or* statt verschachtelter *foreach*-Schleifen:

```
Optional<Boolean> allow = checkDirectUserRule(userId, functionName)
            .or(() -> checkUserGroupRule(groupIdsSupplier.get(),
functionName))
            .or(() -> checkUserEndpointGroupRule(userId, functionName))
            .or(() ->
checkUserGroupEndpointGroupRule(groupIdsSupplier.get(), functionName));

if (allow.isEmpty()) {
      return PermissionResultDto.deny(userId, userName);
}
```

Code 4.11: Vierstufige Berechtigungskaskade in PermissionService.checkPermission

Die Verkettung über *Optional#or* setzt die Kaskade direkt in Code um: Jede Stufe liefert entweder ein Ergebnis (dann stoppt die Kette) oder *Optional.empty()* (dann wird die nächste Stufe versucht). Liefert keine Stufe ein Ergebnis, greift der Default *deny* (fail-closed, NFA-1). Die vier Stufen entsprechen dabei unmittelbar den vier Zuordnungsentitäten EndpointUserAccess, EndpointUserGroupAccess, EndpointGroupUserAccess und EndpointGroupUserGroupAccess aus Abbildung 3.2, die im neuen Backend unter gleichem Namen als JPA-Entitäten abgebildet sind.

4.3.2 Konfliktauflösung innerhalb einer Stufe

Eine Detailfrage, die im alten Code nicht offensichtlich dokumentiert war: Was passiert, wenn innerhalb einer Stufe mehrere Regeln gleichzeitig zutreffen (z. B. ein Nutzer ist Mitglied zweier Gruppen mit widersprüchlichen Regeln für dieselbe Funktion)? Die neue Implementierung löst dies deterministisch über *allMatch*, wodurch ein einzelnes explizites *deny* stärker wiegt als beliebig viele *allow*-Einträge:

```
private <T> Optional<Boolean> resolveStepDecision(
        List<T> matches, Function<T, Boolean> allowExtractor, String
stepName, String principalDescription) {
    if (matches.isEmpty()) {
        return Optional.empty();
    }
    boolean allow =
matches.stream().map(allowExtractor).allMatch(Boolean::booleanValue);
    if (matches.size() > 1) {
        log.warn("checkPermission — {} matched {} rules for {}, resolving
result={} (explicit deny wins)",
                stepName, matches.size(), principalDescription, allow ?
"allow" : "deny");
    }
    return Optional.of(allow);
}
```

Code 4.12: Konfliktauflösung innerhalb einer Kaskadenstufe

Diese "explizite Verweigerung gewinnt"-Regel ist eine bewusste, zusätzlich zum ursprünglichen Verhalten im Javadoc der Klasse dokumentierte Entscheidung. Im PHP-Original (*PermissionCalculationService::getPermission*) iteriert eine *foreach*-Schleife über die Gruppen des Nutzers und gibt beim ersten Treffer sofort zurück (*return $endpointUserGroupAccess->relation();*); ist ein Nutzer Mitglied zweier Gruppen mit widersprüchlichen Regeln für dieselbe Funktion, hängt das Ergebnis dort also von der – nirgends spezifizierten – Iterationsreihenfolge der Gruppenzuordnung ab. Die neue Implementierung ersetzt dieses zufällige Verhalten durch eine deterministische, sicherheitsorientierte Regel: Ein einzelnes explizites *deny* wiegt stärker als beliebig viele *allow*-Einträge derselben Stufe. Dies verhindert, dass eine einzelne großzügige Gruppenregel eine andernorts bewusst gesetzte Sperre aushebelt, und wurde im Rahmen von AP2 eigens als Abweichung vom (undokumentierten) PHP-Verhalten analysiert und hier explizit spezifiziert. Der *log.warn*-Aufruf bei mehrdeutigen Treffern macht diesen Fall zusätzlich im Betrieb beobachtbar, ohne die Entscheidung selbst zu verändern.

4.3.3 Fail-closed bei fehlerhafter Eingabe

Unbekannte Nutzer-IDs, *null*-Werte oder leere Funktionsnamen werden vor jeder Datenbankabfrage abgefangen und führen direkt zu *deny*:

```
if (userId == null || functionName == null || functionName.isBlank()) {
    return PermissionResultDto.deny(userId != null ? userId :
UNRESOLVED_USER_ID, UNKNOWN_USER_NAME);
}

Optional<BaseUser> user = userRepository.findById(userId);
if (user.isEmpty()) {
    return PermissionResultDto.deny(userId, UNKNOWN_USER_NAME);
}
```

Code 4.13: Fail-closed-Behandlung ungültiger Eingaben

Dies verhindert zugleich unnötige Datenbanklast bei offensichtlich ungültigen Anfragen und schließt die Lücke, dass ein Programmfehler beim Aufrufer (z. B. vergessene Nutzer-ID) niemals versehentlich zu einem impliziten *allow* führen kann. Die Methode fragt dabei konsequent in zwei Schritten ab: zunächst die rein syntaktische Gültigkeit der Eingabe, danach erst die fachliche Existenz des Nutzers – beide Fälle enden in *deny*, werden aber mit unterschiedlichem Log-Level (*warn* bzw. *info*) protokolliert, da ein unbekannter Nutzer im Normalbetrieb erwartbarer ist als eine strukturell ungültige Anfrage.

4.3.4 Caching der Gruppenzugehörigkeit innerhalb einer Prüfung

Da Stufe 2 und Stufe 4 beide die Gruppenzugehörigkeit des Nutzers benötigen, wird diese nur bei Bedarf und höchstens einmal pro Anfrage geladen (*Supplier* mit internem Cache), um eine überflüssige zweite Datenbankabfrage zu vermeiden, falls Stufe 1 bereits entscheidet:

```
Supplier<List<Integer>> groupIdsSupplier = new Supplier<>() {
    private List<Integer> cachedGroupIds;

    public List<Integer> get() {
        if (cachedGroupIds == null) {
            cachedGroupIds = userRepository.findGroupIdsByUserId(userId);
        }
        return cachedGroupIds;
    }
};
```

Code 4.14: Bedarfsgesteuertes Laden der Gruppenzugehörigkeit

4.3.5 Testbarkeit der Kaskade

Da die Berechtigungskaskade die sicherheitskritischste Komponente des Backends ist (sie entscheidet laut NFA-1/NFA-2 fail-closed über jeden Funktionsaufruf der DaaS-Anwendung), ist sie zugleich diejenige mit der feingranularsten Testabdeckung: *PermissionServiceTest* deckt alle vier Kaskadenstufen einzeln ab, den Default-Deny-Fall, die in 4.3.2 beschriebene Konfliktauflösung bei mehreren Treffern je Stufe sowie die in 4.3.3 beschriebenen fail-closed-Sonderfälle (*null*-Nutzer-ID, leerer Funktionsname, unbekannter Nutzer). *PermissionControllerIntegrationTest* ergänzt dies um End-zu-End-Tests gegen den tatsächlichen HTTP-Endpunkt (Abschnitt 4.8.2).

4.4 Benutzerverwaltung

Das Fachmodul *user* deckt die Anwendungsfälle FA-5 (Registrierung mit E-Mail-Verifikation), FA-6 (Admin-Verwaltung regulärer Nutzer), FA-7 (Admin legt weitere Admin-Konten an), FA-8 (User-Groups) und FA-9 (Anwendungsfreischaltung) aus Kapitel 3.2 ab. Die Abschnitte 4.4.1–4.4.2 folgen dem Lebenszyklus eines Nutzerkontos von der Registrierung bis zur administrativen Verwaltung; Abschnitt 4.4.3 ordnet User-Groups als Bindeglied zu den Modulen *permission* und *desktop* ein; die Abschnitte 4.4.4–4.4.5 beschreiben zwei Funktionen, die zwar fachlich zum Modul gehören, aber nicht Teil des eigentlichen Registrierungs-Lebenszyklus sind.

4.4.1 Registrierung mit E-Mail-Verifikation

UserService.createUser legt einen zunächst deaktivierten Nutzer an, generiert einen 8-stelligen hexadezimalen Registrierungscode mit 24 Stunden Gültigkeit und versendet ihn per E-Mail:

```
private static final String REGISTRATION_CODE_CHARS = "ABCDEF0123456789";
private static final int REGISTRATION_CODE_LENGTH = 8;

private String generateRegistrationCode() {
    StringBuilder sb = new StringBuilder(REGISTRATION_CODE_LENGTH);
    for (int i = 0; i < REGISTRATION_CODE_LENGTH; i++) {

sb.append(REGISTRATION_CODE_CHARS.charAt(SECURE_RANDOM.nextInt(REGISTRATI
ON_CODE_CHARS.length())));
    }
    return sb.toString();
}
```

Code 4.15: Erzeugung des Registrierungscodes

Der E-Mail-Versand wird bewusst nach dem Datenbank-Commit ausgeführt (*runAfterCommit*), nicht innerhalb der Transaktion:

```
private void runAfterCommit(Runnable action) {
    if (TransactionSynchronizationManager.isSynchronizationActive()) {
        TransactionSynchronizationManager.registerSynchronization(new
TransactionSynchronization() {
            @Override
            public void afterCommit() {
                action.run();
            }
        });
    } else {
        action.run();
    }
}
```

Code 4.16: E-Mail-Versand nach erfolgreichem Commit

Diese Entscheidung verhindert, dass ein langsamer SMTP-Versand eine Datenbanktransaktion (und damit gehaltene Sperren) unnötig lange offenhält, und stellt zugleich sicher, dass keine E-Mail verschickt wird, falls die Transaktion am Ende fehlschlägt und zurückgerollt wird (sonst bekäme ein Nutzer einen Code für ein Konto, das gar nicht existiert). Dasselbe Muster verwendet *requestApplication* (Abschnitt 4.4.4) und *resendRegistrationCode*.

UserService.validateEmail prüft Code und Ablaufzeit und aktiviert das Konto erst danach:

```
if (user.getRegistrationCode() == null || !
user.getRegistrationCode().equals(registrationCode)) {
    return false;
}
if (user.getRegistrationCodeTimeout() != null
        && Instant.now(clock).isAfter(user.getRegistrationCodeTimeout()))
{
    return false;
}
user.setEnabled(true);
user.setRegistrationUsedMoment(Instant.now(clock));
user.setRegistrationCode(null);
userRepository.save(user);
```

Code 4.17: Prüfung des Registrierungscodes in UserService.validateEmail

Nach erfolgreicher Verifikation wird der Code gelöscht (*setRegistrationCode(null)*), sodass er kein zweites Mal eingereicht werden kann, und der Verifikationszeitpunkt (*registrationUsedMoment*) dauerhaft festgehalten. Mit 16 möglichen Zeichen an acht Stellen ergeben sich rund 4,3 Milliarden mögliche Codes (32 Bit); zusammen mit der Gültigkeit von 24 Stunden und der Ratenbegrenzung von zehn Prüfanfragen pro Minute und IP-Adresse (Abschnitt 4.2.6) ist ein Erraten des Codes sehr unwahrscheinlich.

resendRegistrationCode ist absichtlich wortkarg: Für unbekannte, bereits verifizierte oder Admin-Konten passiert nach außen sichtbar nichts (kein Fehler, keine Bestätigung) — dies verhindert, dass der Endpunkt zum Enumerieren registrierter E-Mail-Adressen missbraucht

werden kann, ähnlich der Timing-Maßnahme in 4.2.2. Der zuvor gültige Code wird dabei durch einen neuen ersetzt, sodass stets höchstens ein Code pro Konto gültig ist.

4.4.2 Admin-initiierte Verwaltung

AdminController/AdminService erlauben das Anlegen weiterer Admin-Konten (nur mit *SCOPE_admin*), *UserController/UserService* das Auflisten, Teil-Update (*PATCH*), Aktivieren und Deaktivieren regulärer Nutzer. Ändert sich das Passwort eines Nutzers oder wird sein Konto deaktiviert, veröffentlicht der Service ein *UserSessionsInvalidatedEvent*:

```
if (request.getPassword() != null) {
    accountValidator.validatePassword(request.getPassword());
    user.setPassword(passwordEncoder.encode(request.getPassword()));
    eventPublisher.publishEvent(new UserSessionsInvalidatedEvent(id));
}
public UserDto disableUser(Integer id) {
    User user = getUserOrThrow(id);
    user.setEnabled(false);
    userRepository.save(user);
    eventPublisher.publishEvent(new UserSessionsInvalidatedEvent(id));
    return UserDto.from(user);
}
```

Code 4.18: Invalidierung aller Sitzungen bei Passwortänderung und Deaktivierung

Dieses Event entkoppelt die Nutzerverwaltung von der Token-Widerrufslogik: *UserService* muss nicht wissen, wie Sessions invalidiert werden (das ist Aufgabe eines Listeners im *auth*-Modul) — ein Beispiel für die in 4.1 beschriebene strikte Schichtentrennung, die über reine Methodenaufrufe hinaus auch auf Ereignisebene eingehalten wird. In beiden Fällen (Passwortänderung, Deaktivierung) ist der Effekt derselbe: Alle zuvor ausgestellten Access- und Refresh-Tokens des betroffenen Nutzers werden ungültig, sodass ein Angreifer mit einem zuvor gestohlenen Token nach einer Admin-Intervention nicht länger zugreifen kann.

4.4.3 User-Groups als Bindeglied zum Berechtigungs- und Desktop-Modul

UserGroup ist das Bindeglied zwischen Benutzerverwaltung, Berechtigungskaskade (Stufen 2 und 4, Abschnitt 4.3) und Desktop-Inventar (Abschnitt 4.5): Eine Gruppe kann sowohl Berechtigungsregeln als auch Desktop-Group-Zuordnungen tragen. *UserGroupService* stellt CRUD-Operationen für Gruppen bereit; die eigentliche Verknüpfungslogik zu Desktop-Groups liegt bewusst im *desktop*-Modul (siehe 4.5), um zyklische Abhängigkeiten zwischen *user* und *desktop* zu vermeiden (eine automatisierte Prüfung auf zyklische Abhängigkeiten zwischen den Fachmodulen ist derzeit nicht Teil der ArchUnit-Regeln und bietet sich als Erweiterung an). Die zugrunde liegenden Beziehungen entsprechen weitgehend dem Datenmodell des alten Backends aus Abbildung 3.2.

4.4.4 Anwendungsfreischaltung (FA-9)

Ein im alten Backend als *UserService::requestApplication()* vorhandener Ablauf erlaubt es einem Nutzer, Zugriff auf eine weitere, im Frontend sichtbare Anwendung anzufragen; der zuständige Administrator wird daraufhin per E-Mail benachrichtigt. Dieser Ablauf wurde unverändert als eigener Anwendungsfall (FA-9, Kapitel 3.2) übernommen und in *UserService.requestApplication* portiert:

```
/**
 * Sends an application-request email on behalf of the authenticated
user.
 * Mirrors PHP UserService::requestApplication().
 */
@Transactional(readOnly = true)
public void requestApplication(Integer userId, String application) {
    BaseUser baseUser = userRepository.findById(userId).orElseThrow(() ->
new AccountNotFoundException(userId));
    String email = baseUser.getEmail();
    String name = baseUser.getName();
    String id = String.valueOf(baseUser.getId());
    runAfterCommit(() -> mailService.sendApplicationRequestEmail(email,
name, id, application));
}
```

Code 4.19: Anfrage einer Anwendungsfreischaltung in UserService.requestApplication

Bemerkenswert ist, dass diese Methode – im Gegensatz zu *createUser* – rein lesend transaktioniert ist (*readOnly = true*), da sie keine Datenbankänderung vornimmt, sondern nur eine E-Mail auslöst; dennoch wird dieselbe *runAfterCommit*-Infrastruktur wiederverwendet, damit ein eventueller Rollback der (hier lesenden) Transaktion nicht trotzdem zu einem E-Mail-Versand führen kann.

4.4.5 Bootstrap eines Standard-Administrators

Da ein frisch aufgesetztes Backend zunächst über kein Admin-Konto verfügt, über das sich weitere Admins anlegen ließen (FA-7 setzt bereits ein bestehendes Admin-Konto voraus), legt *DataInitializer* beim Anwendungsstart einmalig einen konfigurierten Standard-Administrator an, sofern noch keiner mit dieser E-Mail-Adresse existiert:

```
@Component
@RequiredArgsConstructor
public class DataInitializer implements ApplicationRunner {

    @Override
    @Transactional
    public void run(ApplicationArguments args) {
        if (!properties.isEnabled()) {
            return;
        }
        if (userRepository.existsByEmail(properties.getEmail())) {
            return;
        }
        Admin admin = new Admin();
        admin.setName("Default Admin");
        admin.setEmail(properties.getEmail());

admin.setPassword(passwordEncoder.encode(properties.getPassword()));
        admin.setEnabled(true);
        userRepository.save(admin);
    }
}
```

Code 4.20: Anlage eines Standard-Administrators beim Start (gekürzt)

E-Mail-Adresse und Passwort sind über *ADMIN_EMAIL/ADMIN_PASSWORD* konfigurierbar (Default: *admin@design.local/Admin123!*); die Funktion lässt sich über *app.admin-initializer.enabled=false* vollständig deaktivieren, etwa für eine produktive Umgebung, in der Administratorkonten ausschließlich manuell angelegt werden sollen. *DataInitializer* liegt bewusst im eigenen Paket *bootstrap* statt in *user*, da es sich um eine anwendungsweite Startaufgabe und keine fachliche Nutzerverwaltungsoperation handelt (vgl. die Paketgliederung aus 4.1.2). Da diese Standardwerte öffentlich dokumentiert sind, müssen sie vor jedem produktiven Einsatz über die Umgebungsvariablen ersetzt werden; andernfalls bestünde ein Zugang mit bekannten Zugangsdaten.

4.5 Desktop-Inventar

Das Fachmodul *desktop* setzt FA-10 und FA-11 (Verwaltung von Desktops und Desktop-Gruppen) um und bildet damit die fachliche Brücke zwischen Nutzerverwaltung (4.4) und der

von der DaaS-Anwendung tatsächlich bereitgestellten Infrastruktur. Da dieses Modul am wenigsten sicherheitskritische Logik enthält, konzentrieren sich die folgenden beiden Abschnitte auf zwei Entwurfsentscheidungen, die der Datenintegrität dienen: die Abgrenzung des Inventars von der Bereitstellung (4.5.1) und die bidirektionale n:m-Verknüpfung mit User-Groups (4.5.2).

4.5.1 Reines Inventar, keine Bereitstellung

DesktopService verwaltet ausschließlich Metadaten zu virtuellen Desktops und Desktop-Groups; die tatsächliche Bereitstellung von Containern/VMs erfolgt – wie in Kapitel 2.1 und 3.1 dargestellt – durch die DaaS-Anwendung, nicht durch dieses Backend. *createDesktop* erlaubt optional das kaskadierte Anlegen von Untergruppen in einem Aufruf:

```
@Transactional
public void createDesktop(DesktopCreateRequest request) {
    Set<String> subGroupDescriptions = new HashSet<>();
    if (request.getGroups() != null) {
        for (DesktopCreateRequest.SubGroupRequest sgReq :
request.getGroups()) {
            String description = sgReq.getDescription();
            if (!subGroupDescriptions.add(description) ||
desktopGroupRepository.existsByDescription(description)) {
                throw new
DesktopGroupAlreadyExistsException(description);
            }
        }
    }
    Desktop desktop = new Desktop();
    desktop.setDescription(request.getDescription());
    Desktop savedDesktop = desktopRepository.save(desktop);
    if (request.getGroups() != null) {
        for (DesktopCreateRequest.SubGroupRequest sgReq :
request.getGroups()) {
            DesktopGroup g = new DesktopGroup();
            g.setDescription(sgReq.getDescription());
            linkDesktopAndGroup(savedDesktop, g);
            desktopGroupRepository.save(g);
        }
    }
}
```

Code 4.21: Anlegen eines Desktops mit Untergruppen

Die Prüfung auf doppelte Beschreibung erfolgt zweistufig: zunächst innerhalb der aktuellen Anfrage selbst (*subGroupDescriptions.add(...)* gibt *false* zurück bei einem Duplikat innerhalb derselben Anfrage), danach gegen den bereits gespeicherten Datenbestand (*existsByDescription*). Ohne die erste Prüfung könnte eine einzelne Anfrage zwei Untergruppen mit identischem Namen anlegen, da die Datenbankprüfung zum Zeitpunkt der Anfrage noch keinen Konflikt sähe (beide Einträge existieren ja noch nicht).

4.5.2 Verknüpfung von User-Groups und Desktop-Groups

addUserGroupToDesktopGroup verknüpft eine User-Group mit einer Desktop-Group und stellt dabei sicher, dass keine doppelte Assoziation entsteht:

```
@Transactional
public void addUserGroupToDesktopGroup(Integer desktopGroupId, Integer
userGroupId) {
    DesktopGroup dg = findDesktopGroupById(desktopGroupId);
    UserGroup ug = findUserGroupById(userGroupId);
    if (ug.getDesktopGroups().contains(dg)) {
        throw new UserGroupAlreadyAssociatedException(desktopGroupId,
userGroupId);
    }
    linkUserGroupAndDesktopGroup(ug, dg);
    userGroupRepository.save(ug);
}
```

Code 4.22: Verknüpfung von Nutzergruppe und Desktop-Gruppe

Die Verknüpfung wird beidseitig gepflegt (*userGroup.getDesktopGroups().add(...)* und *desktopGroup.getUserGroups().add(...)* innerhalb von *linkUserGroupAndDesktopGroup*), da beide Entitäten eine n:m-Beziehung über JPA abbilden und die In-Memory-Konsistenz der bidirektionalen Assoziation sonst von der jeweiligen Abfragerichtung abhinge. Diese Verknüpfung bildet zugleich die Datengrundlage für FA-12 ("Nutzer kann prüfen, ob er Zugriff auf einen Desktop hat"): Über die Kette Nutzer → User-Group → Desktop-Group → Desktop lässt sich für jeden Desktop feststellen, welche Nutzer über ihre Gruppenmitgliedschaften Zugriff besitzen. Ein eigener Endpunkt für diese Prüfung wurde

jedoch nicht umgesetzt, da ihn weder Frontend noch DaaS-Anwendung aufrufen (Tabelle 3.5).

4.6 Querschnittsthemen

Die folgenden sechs Themen betreffen keines der vier Fachmodule allein, sondern durchziehen den gesamten Code: die Sicherheitsfilterkette, die jede Anfrage passiert (4.6.1), einheitliche Fehlerbehandlung (4.6.2), Eingabevalidierung (4.6.3), CORS/Protokollierung/PII-Schutz (4.6.4), die generierte API-Dokumentation (4.6.5) sowie ein geplanter Aufräummechanismus (4.6.6). Sie bündeln damit genau jene Aspekte, die in 4.1.2 als shared-Paket von den Fachmodulen getrennt wurden.

4.6.1 Sicherheitsfilterkette

SecurityConfig konfiguriert eine zustandslose Filterkette: CSRF ist deaktiviert (keine Cookies/Sessions), *SessionCreationPolicy.STATELESS* ist gesetzt, und bis auf eine explizite Positivliste öffentlicher Endpunkte erfordert jede Anfrage ein gültiges JWT:

```
http.cors(cors -> cors.configurationSource(corsConfigurationSource))
        .csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> {
            auth.requestMatchers(HttpMethod.POST, "/user").permitAll();
            auth.requestMatchers(HttpMethod.POST,
"/user/validate_email").permitAll();
            auth.requestMatchers(HttpMethod.POST,
"/user/resend_code").permitAll();
            auth.requestMatchers(HttpMethod.POST,
"/oauth2/user/token").permitAll();
            auth.requestMatchers(HttpMethod.GET,
"/actuator/health").permitAll();

            if (securityProperties.isSwaggerPublic()) {
                auth.requestMatchers(SWAGGER_ENDPOINTS).permitAll();
            } else {

auth.requestMatchers(SWAGGER_ENDPOINTS).hasAuthority(Authorities.SCOPE_AD
MIN);
            }

            auth.anyRequest().authenticated();
        })
        .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt ->
jwt.decoder(jwtDecoder)));
```

Code 4.23: Konfiguration der zustandslosen Sicherheitsfilterkette (gekürzt)

Diese "deny-by-default, erlaubt nur explizit gelistete Endpunkte"-Struktur ist die Anwendung desselben fail-closed-Prinzips wie in der Berechtigungskaskade (4.3), diesmal auf Ebene der HTTP-Schicht statt der fachlichen Autorisierung. Bemerkenswert ist die Sonderbehandlung der Swagger-UI-Endpunkte über die Konfigurationseigenschaft *app.security.swagger-public*: In Produktivumgebungen (*application-prod.yml*) bleibt sie *false*, sodass die API-Dokumentation nur mit einem Admin-Token einsehbar ist; im lokalen Entwicklungsprofil (*application-local.yml*) ist sie auf *true* gesetzt, um die in Abschnitt 4.7 beschriebene Demonstration ohne vorherigen Login zu ermöglichen. Neben den fachlich öffentlichen Endpunkten ist nur der Health-Check von Spring Boot Actuator ohne Token erreichbar, da Container-Orchestrierung und Load Balancer kein Token mitsenden können; dabei wird lediglich der Gesamtstatus ohne Komponentendetails ausgegeben.

4.6.2 Fehlerbehandlung

GlobalExceptionHandler sowie domänenspezifische Handler (*TokenExceptionHandler*, *SessionExceptionHandler*) übersetzen typisierte Exceptions (*InvalidCredentialsException*, *AccountLockedException*, *UserNotFoundException*, *WeakPasswordException* u. a.) in ein einheitliches *ErrorResponse*-JSON-Format mit passendem HTTP-Statuscode. Dies ersetzt die im alten PHP-Backend über Middleware (*InvalidUserInputHandlerMiddleware*) realisierte, laut Ausschreibung schlecht dokumentierte Fehlerbehandlung durch eine zentrale, typsichere Lösung: Jede fachliche Fehlerbedingung ist eine eigene, sprechend benannte Exception-Klasse statt eines generischen Fehlercodes, wodurch sowohl der Aufrufort im Service als auch die Zuordnung zum HTTP-Statuscode im Handler an einer Stelle nachvollziehbar sind.

4.6.3 Eingabevalidierung

Alle Request-DTOs nutzen Bean Validation (***jakarta.validation***), um strukturell ungültige Anfragen bereits vor Erreichen der Service-Schicht abzulehnen. Die Passwortregeln sind dabei zentral an einer Stelle definiert, damit Validierungsannotation und Fehlermeldung nicht auseinanderlaufen können:

```
public final class PasswordPolicy {
    public static final int MIN_LENGTH = 8;
    public static final int MAX_LENGTH = 72;
    public static final String MESSAGE =
            "password must be 8-72 characters and contain uppercase,
lowercase, and a digit";
    public static final String PASSWORD_REGEX = "^(?=.*[A-Z])(?=.*[a-z])
(?=.*\\d).{8,72}$";
}
```

Code 4.24: Zentrale Passwortrichtlinie

Die Obergrenze von 72 Zeichen ist kein willkürlicher Wert, sondern entspricht der technischen Grenze von BCrypt, das nur die ersten 72 Byte eines Passworts in den Hash einbezieht; ein längeres Passwort würde stillschweigend auf diese Länge gekürzt, was Nutzende fälschlich annehmen ließe, ein längeres Passwort erhöhe die Sicherheit.

PasswordPolicy wird sowohl von der Bean-Validation-Annotation auf den DTOs als auch von *AccountValidator.validatePassword* referenziert (serverseitige Zweitprüfung bei Registrierung und Passwortänderung, Abschnitt 4.4.1/4.4.2), sodass beide Prüfpfade garantiert dieselbe Regel anwenden.

4.6.4 CORS, Protokollierung und PII-Maskierung

CorsConfig erlaubt standardmäßig nur den in *app.frontend.url* konfigurierten Origin. Da JWTs personenbezogene Daten (E-Mail, Name) als Claims enthalten, maskiert *LogMasking.maskEmail* E-Mail-Adressen in allen Log-Zeilen, die Nutzeraktionen protokollieren (siehe z. B. Abschnitt 4.2.2); dies betrifft sämtliche *service*-Klassen, die mit Nutzerdaten arbeiten, und wurde — ebenso wie die Timing-Angriffs-Abwehr in 4.2.2 — nicht aus dem alten Backend übernommen, sondern als eigenständige Verbesserung gegenüber dem dokumentierten Ist-Zustand ergänzt.

4.6.5 API-Dokumentation über OpenAPI/Swagger

Anstelle der manuell gepflegten OpenAPI-Annotationen des alten Backends (*zircote/swagger-php*, vgl. Tabelle 3.1) generiert *springdoc-openapi* die API-Spezifikation automatisch aus den Controller- und DTO-Klassen. *OpenApiConfig* ergänzt lediglich die global benötigten Metadaten (Titel, Version) sowie das Bearer-Token-Sicherheitsschema, das die Swagger-UI für authentifizierte Testaufrufe anbietet:

```
@Bean
public OpenAPI openAPI() {
    return new OpenAPI()
            .info(new Info().title("Design DaaS Backend
API").version("1.0.0"))
            .addSecurityItem(new
SecurityRequirement().addList("bearerAuth"))
            .components(new Components().addSecuritySchemes("bearerAuth",
                    new
SecurityScheme().type(SecurityScheme.Type.HTTP).scheme("bearer").bearerFo
rmat("JWT")));
}
```

Code 4.25: OpenAPI-Konfiguration mit Bearer-Sicherheitsschema

Da die Spezifikation direkt aus dem Code abgeleitet wird, können Pfade, Parameter und Datenstrukturen nicht – wie bei manuell gepflegten Annotationen möglich – von der tatsächlichen Implementierung abweichen — jede Änderung an einem Controller oder DTO wirkt sich unmittelbar auf */v3/api-docs* und die Swagger-UI aus. Diese generierte Dokumentation ist zugleich die Grundlage des in Abschnitt 4.7 beschriebenen Demonstrators.

4.6.6 Geplante Aufräumarbeiten

TokenCleanupService entfernt abgelaufene/widerrufene Zeilen aus *access_tokens / refresh_tokens* zeitgesteuert über Spring Scheduling, um unbegrenztes Tabellenwachstum zu vermeiden – ein Aspekt, der im alten Backend über den separaten Yacron-Cron-Dienst gelöst war und hier in den Anwendungsprozess selbst integriert wurde (ein Betriebsartefakt weniger).

4.7 Demonstrator (AP5)

Arbeitspaket AP5 verlangt den Aufbau eines Demonstrators, der die zentralen Anwendungsfälle aus Abschnitt 3.5 vorführt. Da das neue Backend – anders als die DaaS-Anwendung – keine eigene grafische Oberfläche besitzt, dient die aus Abschnitt 4.6.5 generierte Swagger-UI als Demonstrationswerkzeug: Sie erlaubt, jeden Endpunkt direkt im Browser mit konkreten Beispieldaten aufzurufen und die Antwort inklusive HTTP-Statuscode einzusehen, ohne einen separaten HTTP-Client (z. B. Postman oder curl) zu benötigen.

4.7.1 Aufbau der Demonstrationsumgebung

Der Demonstrator wird aus dem unveränderten Produktionsartefakt betrieben: Das mehrstufige ***Dockerfile*** baut das Projekt mit Maven (*mvn package -DskipTests*) und kopiert ausschließlich das resultierende JAR in ein schlankes *eclipse-temurin:21-jre-alpine*-Laufzeitimage:

```
FROM maven:3.9.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B -q
COPY src ./src
RUN mvn package -DskipTests -B -q

FROM eclipse-temurin:21-jre-alpine
...
VOLUME /app/keys
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Code 4.26: Mehrstufiges Dockerfile (gekürzt)

Das Volume */app/keys* nimmt das in Abschnitt 4.2.3 beschriebene RS256-Schlüsselpaar auf, damit es einen Container-Neustart übersteht. Für die eigentliche Demonstration genügt daneben eine lokal erreichbare MariaDB-Instanz; die Konfiguration erfolgt vollständig über Umgebungsvariablen (*.env.example*):

```
DB_URL=jdbc:mariadb://localhost:3306/design_backend
DB_USERNAME=design
DB_PASSWORD=design_password
OAUTH_KEYS_DIR=./keys
MAIL_ENABLED=false
FRONTEND_URL=http://localhost:3000
```

Code 4.27: Umgebungsvariablen der Demonstrationsumgebung (Auszug aus .env.example)

Mit *MAIL_ENABLED=false* (Standardwert) versendet *MailService* keine echten E-Mails, protokolliert den Registrierungscode aber auf DEBUG-Level in der Anwendungskonsole. Dadurch lässt sich der vollständige Registrierungs-/Verifikationsablauf (FA-5) demonstrieren, ohne einen SMTP-Server bereitstellen zu müssen — der Code wird einfach aus dem Log abgelesen und in */user/validate_email* eingetragen. Beim ersten Start legt *DataInitializer* (Abschnitt 4.4.5) automatisch einen Administrator (*admin@design.local / Admin123!*) an, über den sich weitere Admin- und Nutzerkonten verwalten lassen, ohne zunächst manuell einen Datensatz in die Datenbank einfügen zu müssen. Im lokalen Profil (*application-local.yml*, Standardprofil) ist *app.security.swagger-public=true* gesetzt, sodass *http://localhost:8080/swagger-ui.html* ohne vorherige Anmeldung erreichbar ist.

4.7.2 Vorgeführte Abläufe

Anhand der Swagger-UI werden die in Abschnitt 3.5 beschriebenen Kernabläufe vorgeführt: Abbildung 4.1 und Abbildung 4.2 zeigen zwei dieser Abläufe.

1. **Login und Berechtigungsprüfung (FA-1, FA-13):** *POST /oauth2/user/token* mit dem Formularparameter *grant_type=password* und den Zugangsdaten des Standard-Administrators liefert ein Zugriffstoken; dieses wird über die "Authorize"-Schaltfläche der Swagger-UI hinterlegt. Anschließend liefert *GET /permissions/{functionName}/{userId}* eine Allow/Deny-Entscheidung gemäß der Kaskade aus Abschnitt 4.3.
2. **Token-Erneuerung und Logout (FA-2, FA-3):** *POST /oauth2/user/token* mit *grant_type=refresh_token* und dem zuvor erhaltenen Refresh-Token liefert ein neues Tokenpaar; das alte Refresh-Token ist danach gemäß Abschnitt 4.2.4 nicht mehr verwendbar. *POST /oauth2/user/logout* widerruft das aktuelle Token, eine erneute Anfrage mit demselben Access-Token wird anschließend mit *401 Unauthorized* abgelehnt.
3. **Registrierung und E-Mail-Verifikation (FA-5):** *POST /user* legt ein deaktiviertes Konto an; der Registrierungscode wird (bei *MAIL_ENABLED=false*) aus dem Log abgelesen und an *POST /user/validate_email* übergeben, wonach ein Login mit diesem Konto möglich ist.
4. **Desktop- und Desktopgruppen-Anlage (FA-10, FA-11):** *POST /desktop* legt mit dem Administrator-Token einen Desktop samt Untergruppe an; *POST /desktop_group/{id}/user_group/{userGroupId}* verknüpft diese Gruppe mit einer zuvor über *POST /user_group* angelegten Nutzergruppe.
5. **Anwendungsfreischaltung (FA-9):** *POST /user/request-application*/{application}/{isoCode} löst — ebenfalls sichtbar im Log, solange *MAIL_ENABLED=false* ist — die in Abschnitt 4.4.4 beschriebene Benachrichtigung aus.

[Screenshot aus der lokalen Demonstrationsumgebung hier einfügen]

Abbildung 4.1: Swagger-UI mit Beispielanfrage und -antwort für den Password Grant (POST /oauth2/user/token)

[Screenshot aus der lokalen Demonstrationsumgebung hier einfügen]

Abbildung 4.2: Swagger-UI mit einer allow- und einer deny-Antwort von GET /permissions/{functionName}/{userId}

Der Demonstrator belegt damit, dass die in Abschnitt 3.5 festgelegten Anwendungsfälle mit dem neuen Backend ausführbar sind. Da Frontend und DaaS-Anwendung im Betrieb dieselben, hier vorgeführten HTTP-Schnittstellen verwenden, eignet sich die Swagger-UI für diesen Nachweis; einen Ende-zu-Ende-Test mit beiden Clients ersetzt sie jedoch nicht (vgl. Abschnitt 5.4).
[Hinweis: Falls das Backend gemeinsam mit Frontend und DaaS-Anwendung betrieben wurde, diesen Test hier kurz beschreiben und mit einem Screenshot belegen]

4.8 Tests & Evaluation (AP6)

Nachdem die Abschnitte 4.1–4.7 beschrieben haben, wie die Anforderungen aus Kapitel 3 umgesetzt wurden, belegt dieser letzte Abschnitt des Kapitels, dass diese Umsetzung tatsächlich funktioniert und die eingangs in Kapitel 1 formulierten Qualitätsziele erfüllt. Dazu werden zunächst die im Build automatisch erzwungenen Qualitätsschranken erläutert (4.8.1), anschließend Art und Umfang der Testsuite (4.8.2) sowie die tatsächlich erreichte Testabdeckung (4.8.3) dargestellt. Abschnitt 4.8.4 ordnet eine bewusst in Kauf genommene Einschränkung ein, Abschnitt 4.8.5 schließt das Kapitel mit einem belastbaren, quellcodebasierten Vergleich zwischen altem und neuem Backend.

4.8.1[4.7.3] Automatisierte Qualitätssicherung im Build

Der Build (*mvn verify*) erzwingt zwei automatisierte Qualitätsschranken: die 19 ArchUnit-Regeln aus 4.1.3 und eine Mindest-Testabdeckung von 70 % der Codezeilen über JaCoCo:

```
<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <executions>
        <execution>
            <id>check</id>
            <phase>verify</phase>
            <goals>
                <goal>check</goal>
            </goals>
            <configuration>
                <rules>
                    <rule>
                        <element>BUNDLE</element>
                        <limits>
                            <limit>
                                <counter>LINE</counter>
                                <value>COVEREDRATIO</value>
                                <minimum>0.70</minimum>
                            </limit>
                        </limits>
                    </rule>
                </rules>
            </configuration>
        </execution>
    </executions>
</plugin>
```

Code 4.28: JaCoCo-Mindestabdeckung in der pom.xml (gekürzt)

Unterschreitet die Testabdeckung diesen Schwellenwert, schlägt der Build fehl — Testabdeckung ist damit kein optionales Qualitätsmerkmal, sondern eine harte Build-Voraussetzung (NFA-9).

4.8.2[4.7.4] Testarten und Umfang

Die Testsuite umfasst 31 Testklassen mit 232 JUnit-Testmethoden sowie 19 ArchUnit-Regeln, also insgesamt 251 Testfälle (Stand: Version 2.0.0, ermittelt aus den Surefire-Berichten unter *target/surefire-reports). Eine zusätzliche Basisklasse (BaseIntegrationTest)* bündelt die gemeinsame Konfiguration der Integrationstests. Tabelle 4.2 schlüsselt dies nach Testart auf.

[Zahlen nach erneutem Lauf von mvn verify bestätigen: 0 Fehlschläge, 0 Fehler]

Tabelle 4.2: Testsuite nach Art

Testart	Beispiele	Umfang
Architektur-Tests	ArchitectureTest (Schichtung, Namenskonventionen, Injektionsstil)	19 Regeln
Service-Unit-Tests	TokenServiceTest, PermissionServiceTest, AccountLockoutServiceTest, UserServiceTest, DesktopServiceTest, UserGroupServiceTest, AdminServiceTest, MailServiceTest	8 Klassen
Hilfs-/Infrastruktur-Unit-Tests	TokenIssuerTest, ScopeResolverTest, JwtUserResolverTest, SessionServiceTest, TokenCleanupServiceTest, TokenRevocationValidatorTest, SecurityPropertiesTest, GrantRequestTest, GlobalExceptionHandlerTest	9 Klassen
Controller-Unit-Tests (ohne vollen Spring-Kontext)	PermissionControllerTest, DesktopGroupControllerTest	2 Klassen
Integrationstests (voller Spring-Kontext, H2)	TokenControllerIntegrationTest, UserControllerIntegrationTest, AdminControllerIntegrationTest, DesktopControllerIntegrationTest, UserGroupControllerIntegrationTest,	10 Klassen

	PermissionControllerIntegrationTest, OAuth2SessionControllerIntegrationTest, AuthHardeningIntegrationTest, InputValidationIntegrationTest, PublicEndpointsIntegrationTest	
Anwendungskontext-Test	DesignBackendApplicationTests (Spring-Kontext startet fehlerfrei)	1 Klasse

Integrationstests laufen gegen eine In-Memory-H2-Datenbank statt der produktiv eingesetzten MariaDB (*spring-boot-starter-test*-Testprofil), um ohne externe Infrastruktur und mit kurzer Laufzeit lauffähig zu sein; die Flyway-Migrationen aus 4.1.5 werden dabei unverändert gegen H2 ausgeführt, sodass Schema und produktiver Pfad identisch bleiben. *AuthHardeningIntegrationTest* und *InputValidationIntegrationTest* prüfen gezielt die in 4.2 und 4.6.3 beschriebenen Sicherheits- und Validierungsmaßnahmen End-to-End (z. B. Kontosperrung nach wiederholten Fehlversuchen, Ablehnung zu schwacher Passwörter) statt nur auf Service-Ebene.

4.8.3[4.7.5] Tatsächlich erreichte Testabdeckung

Der zuletzt erzeugte JaCoCo-Bericht (*target/site/jacoco/index.html*) weist für das Gesamtprojekt folgende Werte aus (Tabelle 4.3):

Tabelle 4.3: JaCoCo-Testabdeckung, Gesamtprojekt (91 geprüfte Klassen)

Metrik	Abgedeckt	Gesamt	Anteil
Instruktionen	4.441	5.120	86 %
Verzweigungen (Branches)	245	332	73 %
Zeilen	973	1.118	87 %
Methoden	240	264	91 %
Klassen	87	91	96 %

Die tatsächliche Zeilenabdeckung von 87 % liegt damit deutlich über dem im Build erzwungenen Minimum von 70 % (4.8.1); die Schranke wirkt somit aktuell nicht limitierend, verhindert aber, dass zukünftige Änderungen diesen Wert unbemerkt und schleichend absenken. Die niedrigere Verzweigungsabdeckung (73 %) gegenüber der Zeilenabdeckung (87 %) ist typisch für Code mit vielen fail-closed-Sonderfällen wie der Berechtigungskaskade (4.3) und der Fehlerbehandlung (4.6.2): Jede zusätzliche if-Verzweigung für einen Randfall erhöht die Zahl der Verzweigungen schneller, als sie durch neue Tests vollständig in beiden Richtungen (wahr/falsch) abgedeckt wird.

[Werte nach erneutem Lauf von mvn verify aus target/site/jacoco/index.html aktualisieren; der Bericht stammt von vor Version 2.0.0]

4.8.4 Bekannte Einschränkung: Dauersperre nach Ablauf und erneutem Fehlversuch

Ein im Rahmen der Testerstellung (Abschnitt 4.8.2, *AccountLockoutServiceTest*) bewusst nachvollzogenes Verhalten betrifft die in 4.2.2 beschriebene Kontosperrung: Der Fehlversuchszähler wird laut Javadoc von *AccountLockoutService* ausschließlich bei erfolgreichem Login zurückgesetzt, nicht durch das Ablaufen der Sperrzeit selbst. Läuft eine Sperre ab und der nächste Anmeldeversuch schlägt – etwa durch ein weiterhin falsch erinnertes Passwort – erneut fehl, wird das Konto umgehend für weitere 15 Minuten gesperrt, da der Zähler weiterhin auf fünf Fehlversuchen steht. Für einen tatsächlichen Angreifer ist dies unkritisch (es verlängert lediglich die Sperre), kann aber bei einem Nutzer mit einem schlicht vergessenen Passwort zu einer gefühlten Dauersperre führen, da jeder weitere Fehlversuch die Sperrzeit erneut vollständig startet. Diese Einschränkung ist bewusst in Kauf genommen (fail-closed über Benutzerfreundlichkeit gestellt) und wird in Kapitel 5 als mögliche zukünftige Verbesserung (z. B. schrittweiser Zähler-Abbau statt alles-oder-nichts-Reset) wieder aufgegriffen.

4.8.5[4.7.6] Vergleich mit dem alten Backend

Tabelle 4.4 stellt die wichtigsten Qualitätssicherungsmaßnahmen beider Backends gegenüber. Grundlage sind die tatsächlichen Konfigurationsdateien und Quelltexte beider Repositories (*pom.xml* dieses Projekts; *phpunit.xml*, *composer.json* und *BasicArchitectureTest.php* des PHP-Projekts), nicht deren Dokumentation.

Tabelle 4.4: Qualitätssicherung, altes Backend (PHP) vs. neues Backend (Java)

Kriterium	Altes Backend (PHP)	Neues Backend (Java)
Schichtenarchitektur erzwungen?	Teilweise: PHPat prüft Modulabhängigkeiten, aber nicht die interne Model → Repository → Service → Controller-Schichtung je Modul	Ja: ArchUnit prüft zusätzlich die interne Schichtung jedes Fachmoduls sowie Namens- und Injektionskonventionen (19 Regeln)
Testabdeckung als Build-Gate?	Nein: phpunit.xml konfiguriert Coverage-Erfassung (<coverage includeUncoveredFiles="true"/>), aber keinen Mindestwert, der den Build scheitern lässt	Ja: JaCoCo erzwingt ≥ 70 % Zeilenabdeckung im Build; tatsächlich erreicht: 87 % (Tabelle 4.3)
Anzahl automatisierter Tests	69 PHPUnit-Testdateien (tests/src/{Unit,Integration})	31 JUnit-5-Testklassen mit 251 Testfällen
Berechtigungskaskade	4-stufig, PermissionCalculationService, Konfliktfall bei mehreren Treffern pro Stufe nicht dokumentiert	4-stufig, identische Priorität, zusätzlich dokumentierte und getestete Konfliktauflösung ("explizite Verweigerung gewinnt", 4.3.2)
Schema-Verwaltung	Doctrine Migrations	Flyway, Hibernate nur validate
Auth-Mechanismus	league/oauth2-server,	Eigener RS256-JWT-Aussteller

	Password Grant	+ Spring-Security-Resource-Server, zusätzlich Timing-Schutz und Kontosperrung (4.2.2)
API-Dokumentation	Manuell gepflegte zircote/swagger-php-Annotationen	Automatisch aus Code generiert (springdoc-openapi, 4.6.5) — kann nicht von der Implementierung abweichen

Die Zahl der Testdateien allein (69 vs. 31) ist dabei kein verlässlicher Qualitätsindikator, da PHPUnit und JUnit 5 Tests unterschiedlich granular organisieren (eine JUnit-5-Klasse kann mehrere, mit @Nested gruppierte Testmethoden enthalten); aussagekräftiger ist der direkte Vergleich der automatisierten Qualitätsschranken: Während das alte Backend Architekturregeln bereits besaß, aber keine erzwungene Testabdeckung, kombiniert das neue Backend beide Mechanismen als harte Build-Voraussetzung. Dies ist die unmittelbare, messbare Antwort auf die in der Ausschreibung konstatierte mangelnde Qualität des PHP-Backends und erfüllt NFA-8 und NFA-9 aus Kapitel 3.2 nicht nur konzeptionell, sondern nachweisbar durch einen aktuell grünen Build.

5 Fazit und Ausblick

Dieses Kapitel fasst die Ergebnisse der Arbeit zusammen (5.1), beantwortet die in Abschnitt 1.4 formulierte Forschungsfrage (5.2) und gleicht das realisierte Backend systematisch mit den Anforderungen aus Kapitel 3 ab (5.3). Anschließend werden die Grenzen der Arbeit kritisch reflektiert (5.4), mögliche Weiterentwicklungen skizziert (5.5) und die Arbeit mit einer Schlussbetrachtung abgeschlossen (5.6).

5.1 Zusammenfassung der Ergebnisse

Ausgangspunkt der Arbeit war das in PHP implementierte Backend des Desktop-as-a-Service-Systems DESIGN, das in der Aufgabenstellung als qualitativ schwach und unzureichend dokumentiert beschrieben wird (Baun, 2026). Da für dieses Backend keine verlässliche Dokumentation vorlag, wurde sein Verhalten zunächst aus dem Quellcode rekonstruiert (Kapitel 3). Daraus ergaben sich 13 funktionale und 9 nicht-funktionale Anforderungen. Ergänzend wurde anhand des Quellcodes von Frontend und DaaS-Anwendung ermittelt, welche Teile der alten Schnittstelle die beiden Clients tatsächlich verwenden (Tabelle 3.5).

Auf dieser Grundlage entstand mit Java 21 und Spring Boot ein eigenständiger, als Docker-Image paketierter Dienst, der in die vier fachlichen Module auth, permission, user und desktop gegliedert ist (Kapitel 4). Die Authentifizierung beruht auf RS256-signierten JSON Web Token, die durch eine datenbankgestützte Widerrufsprüfung ergänzt werden; Abmeldung, Passwortänderung und Deaktivierung eines Kontos wirken dadurch auch vor Ablauf eines Tokens. Die vierstufige Berechtigungskaskade des alten PermissionCalculationService wurde mit unveränderter Stufenreihenfolge und unverändertem Default-Deny übernommen. Für den im Altsystem nicht festgelegten Fall widersprüchlicher

Regeln innerhalb einer Stufe wurde eine deterministische Regel ergänzt, nach der ein explizites Deny Vorrang hat (Abschnitt 4.3.2).

Zur Absicherung der Wartbarkeit sind zwei automatisierte Prüfungen fester Bestandteil des Maven-Builds: 19 ArchUnit-Regeln für Schichtung, Abhängigkeitsrichtung und Namenskonventionen sowie eine JaCoCo-Mindestabdeckung von 70 % der Codezeilen. Die Testsuite umfasst 31 Testklassen; der zuletzt erzeugte JaCoCo-Bericht weist eine Zeilenabdeckung von 87 % und eine Verzweigungsabdeckung von 73 % aus (Abschnitt 4.8). Der Demonstrator aus Abschnitt 4.7 führt die zentralen Anwendungsfälle über die generierte Swagger-UI vor.

5.2 Beantwortung der Forschungsfrage

Die Forschungsfrage aus Abschnitt 1.4 besteht aus zwei Teilen: wie das Backend so migriert werden kann, dass Frontend und DaaS-Anwendung funktionsfähig bleiben, und welche architektonischen Maßnahmen dabei die Wartbarkeit verbessern.

Zum ersten Teil zeigt die Arbeit, dass eine Migration ohne Änderungen an der fachlichen Logik der Clients möglich ist, wenn sie auf einer quellcodebasierten Rekonstruktion des Altsystems aufbaut. Entscheidend war dabei weniger die Nachbildung jedes alten Endpunkts als die exakte Nachbildung derjenigen Verträge, auf die sich die Clients tatsächlich stützen: des Token-Endpunkts mit Password Grant, der Session-Abfrage, der vom Frontend genutzten Verwaltungsendpunkte und vor allem der Berechtigungsabfrage der DaaS-Anwendung mit ihrer vierstufigen, fail-closed Kaskade. Eine Migration ganz ohne Anpassung der Clients wurde allerdings nicht erreicht. Im Frontend wurden das Aktivieren und Deaktivieren von Konten auf PATCH umgestellt und ein Logout ergänzt; in der DaaS-Anwendung wurde die Auswertung der Session-Antwort erweitert (Abschnitt 1.5). Diese Änderungen sind klein und betreffen ausschließlich die Anbindung an das Backend. Der erste Teil der Forschungsfrage lässt sich daher so beantworten: Die Migration gelingt mit wenigen, klar abgegrenzten Anpassungen an der Anbindung der Clients, sofern der tatsächlich genutzte Schnittstellenvertrag vorab vollständig erhoben wird.

Zum zweiten Teil lässt sich die verbesserte Wartbarkeit auf drei Maßnahmen zurückführen, die sich gegenseitig ergänzen. Erstens folgt der Code einer nach fachlichen Modulen gegliederten Struktur mit einheitlicher interner Schichtung (Abschnitt 4.1.2). Zweitens wird diese Struktur nicht nur dokumentiert, sondern bei jedem Build durch ArchUnit geprüft, während das alte Backend mit PHPat lediglich die Grenzen der obersten Module prüft (Abschnitt 4.1.3). Drittens sind Datenbankschema und Testabdeckung über Flyway-Migrationen und eine JaCoCo-Schranke an den Build gebunden (Abschnitte 4.1.5 und 4.8.1). Hinzu kommen Maßnahmen, die die Nachvollziehbarkeit erhöhen, etwa die aus dem Code generierte OpenAPI-Dokumentation und die explizit spezifizierte Konfliktauflösung in der Berechtigungskaskade. Tabelle 4.4 zeigt, dass diese Maßnahmen über den Stand des Altsystems hinausgehen. Einschränkend ist festzuhalten, dass Wartbarkeit hier über strukturelle Merkmale bewertet wird; den tatsächlichen Wartungsaufwand über einen längeren Zeitraum zu messen, war im Rahmen der Arbeit nicht möglich.

5.3 Abgleich mit den Anforderungen aus Kapitel 3

Tabelle 5.1 und Tabelle 5.2 zeigen für jede Anforderung aus Abschnitt 3.2, ob und wo sie umgesetzt wurde. Der Erfüllungsgrad wird in den Stufen erfüllt, teilweise erfüllt und nicht umgesetzt angegeben.

Tabelle 5.1: Erfüllungsgrad der funktionalen Anforderungen

ID	Anforderung (Kurzform)	Status	Nachweis
FA-1	Login liefert Zugriffstoken	erfüllt	4.2.2; AF-1; TokenControllerIntegrationTest
FA-2	Erneuerung per Refresh-Token	erfüllt	4.2.4; AF-2
FA-3	Logout widerruft Token serverseitig	erfüllt	4.2.5; AF-3
FA-4	Session/Identität abfragbar	erfüllt	4.2.5; OAuth2SessionControllerInte

			grationTest
FA-5	Selbstregistrierung mit E-Mail-Bestätigung	erfüllt	4.4.1; AF-4
FA-6	Admin verwaltet Nutzer	teilweise	4.4.2; Einzelabruf eines Nutzers entfällt (Tabelle 3.5)
FA-7	Admin verwaltet Admin-Konten	teilweise	4.4.2; Anlegen und Auflisten umgesetzt, Löschen entfällt
FA-8	Nutzergruppen verwalten und verknüpfen	erfüllt	4.4.3; 4.5.2; AF-5
FA-9	Anwendungsfreischaltung anfragen	erfüllt	4.4.4; AF-8
FA-10	Desktops anlegen, einsehen, auflisten	erfüllt	4.5.1; AF-6
FA-11	Desktop-Gruppen verwalten und verknüpfen	teilweise	4.5.2; Einzelabruf einer Desktop-Gruppe entfällt
FA-12	Desktop-Zugriff des Nutzers prüfen	nicht umgesetzt	von keinem Client genutzt (Tabelle 3.5)
FA-13	Berechtigungsprüfung über vierstufige Kaskade	erfüllt	4.3; AF-7; PermissionServiceTest

Neun der dreizehn funktionalen Anforderungen sind vollständig umgesetzt. Bei FA-6, FA-7 und FA-11 fehlen einzelne Operationen (Einzelabruf, Löschen), und FA-12 wurde nicht umgesetzt. Allen vier Fällen ist gemeinsam, dass die betroffenen Endpunkte weder vom Frontend noch von der DaaS-Anwendung aufgerufen werden (Tabelle 3.5). Ihr Wegfall schränkt den Betrieb von DESIGN daher nicht ein. Er ist jedoch eine bewusste Abweichung von den aus dem Altsystem abgeleiteten Anforderungen, die bei einer späteren Erweiterung der Clients zu berücksichtigen ist.

Tabelle 5.2: Erfüllungsgrad der nicht-funktionalen Anforderungen

ID	Anforderung (Kurzform)	Status	Nachweis
NFA-1	Fail-closed bei Unklarheit	erfüllt	4.3.3; 4.6.1; PermissionServiceTest

NFA-2	Stufenreihenfolge und Default-Deny wie im Altsystem	erfüllt	4.3.1; 4.3.2
NFA-3	Gleiche Pfade, Methoden, Felder, Statuscodes	teilweise	genutzte Endpunkte übernommen; Abweichungen in Tabelle 3.5
NFA-4	Passwort-Hashing und Kontosperrung	erfüllt (mit Einschränkung)	4.2.2; 4.8.4
NFA-5	Ratenbegrenzung von Login und Registrierung	erfüllt	4.2.6; 4.2.7
NFA-6	Serverseitig widerrufbare Zugriffstoken	erfüllt	4.2.3 bis 4.2.5
NFA-7	Zustandslos und horizontal skalierbar	konzeptionell erfüllt	3.4.3; 4.1.4; kein Lasttest
NFA-8	Automatisiert geprüfte Schichtung	erfüllt	4.1.3; 4.8.1
NFA-9	Erzwungene Mindest-Testabdeckung	teilweise	4.8.1; 4.8.3; Merge-Sperre erfordert CI-Pipeline

Fünf der neun nicht-funktionalen Anforderungen sind ohne Einschränkung erfüllt. NFA-3 ist nur teilweise erfüllt, weil das neue Backend die Pfade, Felder und Statuscodes der genutzten Endpunkte übernimmt, beim Aktivieren und Deaktivieren aber PATCH statt POST verwendet und ungenutzte Endpunkte nicht anbietet. NFA-4 ist mit der in Abschnitt 4.8.4 beschriebenen Einschränkung der Kontosperrung erfüllt. NFA-7 ist durch den zustandslosen Entwurf konzeptionell erfüllt, wurde aber nicht unter Last nachgewiesen. Bei NFA-9 lässt eine Unterschreitung der Mindestabdeckung den Build zuverlässig fehlschlagen; dass dadurch auch das Zusammenführen von Änderungen verhindert wird, setzt eine CI-Pipeline voraus, die nicht Teil des Repositorys ist.

5.4 Kritische Reflexion und Grenzen der Arbeit

Die Ergebnisse sind vor dem Hintergrund mehrerer Grenzen zu bewerten, die sich teils aus der Abgrenzung in Abschnitt 1.5 und teils aus Entscheidungen während der Umsetzung ergeben.

Anforderungen aus Reverse Engineering. Alle Anforderungen wurden aus dem Quellcode des alten Backends und der Clients rekonstruiert. Sie geben daher das beobachtbare Verhalten des Altsystems wieder, nicht notwendigerweise dessen ursprünglich beabsichtigtes Verhalten. Entwurfsentscheidungen, die sich nicht im Code niederschlagen, konnten nicht erfasst werden, und Interviews mit den Entwicklern des Altsystems fanden nicht statt.

Verhaltenskompatibilität nur indirekt geprüft. Die Kompatibilität wurde über die Analyse der Clients, Integrationstests und den Demonstrator abgesichert. Automatisierte Vertragstests, die dieselben Anfragen gegen das alte und das neue Backend stellen und die Antworten vergleichen, wurden nicht erstellt. Abweichungen in selten genutzten Feldern oder Fehlerfällen lassen sich daher nicht vollständig ausschließen.

Pflege der Berechtigungsregeln. Das alte Backend bot für die Zuordnung von Nutzergruppen zu Endpunktgruppen eigene HTTP-Endpunkte und für die Stammdaten der Endpunkte ein CLI-Kommando. Im neuen Backend werden diese Regeln derzeit direkt in der Datenbank gepflegt. Für den Betrieb ist das ausreichend, für Administratoren aber weniger komfortabel und fehleranfälliger.

Sicherheitsbezogene Kompromisse. Der Password Grant wurde aus Kompatibilitätsgründen beibehalten, obwohl RFC 9700 von ihm abrät (Lodderstedt et al., 2025). Bereits verwendete Refresh-Tokens werden abgelehnt, eine Wiederverwendung führt aber nicht zum Widerruf aller zugehörigen Tokens. Die Kontosperrung kann bei vergessenem Passwort zu wiederholten Sperren führen (Abschnitt 4.8.4), und die Standardzugangsdaten des Initial-Administrators müssen vor einem produktiven Einsatz geändert werden (Abschnitt 4.4.5).

Skalierung und Betrieb. Die horizontale Skalierbarkeit wurde nicht unter Last nachgewiesen. Zudem hält die Ratenbegrenzung ihre Zähler im Arbeitsspeicher der jeweiligen Instanz, sodass die Limits beim Betrieb mehrerer Instanzen je Instanz gelten. Alle Instanzen müssen außerdem dasselbe RSA-Schlüsselpaar verwenden; geht es verloren, werden alle ausgestellten Tokens ungültig.

Aussagekraft der Tests. Die Integrationstests laufen gegen H2 statt gegen MariaDB, sodass datenbankspezifische Unterschiede unentdeckt bleiben können. Die Testabdeckung zeigt zudem nur, welcher Code ausgeführt wird, nicht, ob jede Eingabekombination korrekt behandelt wird; die niedrigere Verzweigungsabdeckung von 73 % deutet darauf hin, dass nicht jeder Sonderfall in beiden Richtungen getestet ist.

5.5 Ausblick

Aus den genannten Grenzen ergeben sich folgende Ansatzpunkte für künftige Arbeiten:

- ***Vertragstests mit den Clients:*** *Verbrauchergetriebene Vertragstests (Consumer-Driven Contracts) für Frontend und DaaS-Anwendung würden die Verhaltenskompatibilität dauerhaft und automatisiert absichern.*
- ***Ablösung des Password Grants:*** *Mittelfristig sollte das Frontend auf den Authorization-Code-Grant mit PKCE umgestellt werden (Sakimura et al., 2015; Lodderstedt et al., 2025), etwa mit Spring Authorization Server als eigenständigem Autorisierungsserver.*
- ***Verwaltung der Berechtigungsregeln:*** *Eine Schnittstelle zum Pflegen von Endpunkten, Endpunktgruppen und Zugriffsregeln, gegebenenfalls mit Import aus einer Datei, würde die direkte Pflege in der Datenbank ersetzen.*
- ***Token-Sicherheit:*** *Eine Erkennung wiederverwendeter Refresh-Tokens mit Widerruf der gesamten Token-Familie sowie eine geplante Schlüsselrotation über ein JSON Web Key Set mit kid-Kennung (Jones, 2015b) würden die Widerstandsfähigkeit gegen gestohlene Tokens und Schlüsselverlust erhöhen.*

- ***Skalierung:*** *Eine Ratenbegrenzung mit gemeinsamem Zustand für alle Instanzen und ein Lasttest mit mehreren Instanzen hinter einem Load Balancer sollten einem produktiven Einsatz vorausgehen.*
- ***Kontosperrung:*** *Statt eines vollständigen Zurücksetzens nur bei erfolgreichem Login könnte der Fehlversuchszähler nach Ablauf der Sperrzeit schrittweise abgebaut oder die Sperrdauer schrittweise erhöht werden.*
- ***Qualitätssicherung:*** *Integrationstests gegen eine MariaDB-Instanz in einem Container, zusätzliche Tests für die Randfälle der Berechtigungskaskade und eine CI-Pipeline mit Merge-Schutz würden die bestehenden Build-Prüfungen sinnvoll ergänzen.*

5.6 Schlussbetrachtung

Die Arbeit hat gezeigt, dass sich das Backend von DESIGN von PHP nach Java und Spring Boot migrieren lässt, ohne die fachliche Logik von Frontend und DaaS-Anwendung zu verändern. Voraussetzung dafür war eine sorgfältige, quellcodebasierte Rekonstruktion sowohl des Altsystems als auch der tatsächlich genutzten Schnittstelle. Zugleich wurde die Neuentwicklung genutzt, um die in der Aufgabenstellung bemängelte Qualität dauerhaft abzusichern: Architekturregeln und Testabdeckung hängen im neuen Backend nicht mehr allein von der Disziplin der Entwickler ab, sondern werden bei jedem Build geprüft. Die benannten Grenzen zeigen, dass mit dieser Arbeit ein tragfähiger, aber noch nicht abgeschlossener Schritt in der Weiterentwicklung von DESIGN vorliegt.

6 Abkürzungen

A

ABAC	Attribute-Based Access Control
ACL	Access Control List
AF	Anwendungsfall

ANSI	American National Standards Institute
API	Application Programming Interface
AP	Arbeitspakete
ArchUnit	Architecture Unit

C

CI	Continuous Integration
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CSRF	Cross-Site Request Forgery

D

DaaS	Desktop-as-a-Service
DI	Dependency Injection
DTO	Data Transfer Object

F

FA	Funktionale Anforderung

H

HMAC	Keyed-Hash Message Authentication Code
HS256	HMAC with SHA-256
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure

I

IETF	Internet Engineering Task Force
INCITS	InterNational Committee for Information Technology Standards
IoC	Inversion of Control
I/O	Input/Output

J

JaCoCo	Java Code Coverage
JAR	Java ARchive
JDBC	Java Database Connectivity-API
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JUnit	Java Unit Test
JVM	Java Virtual Machine
JWA	JSON Web Algorithms
JWK	JSON Web Key
JWKS	JSON Web Key Set
JWS	JSON Web Signature
JWT	JSON Web Token

K

KVM	Kernel-based Virtual Machine

N

NFA	Nicht-funktionale Anforderung

O

OASIS	Organization for the Advancement of Structured Information Standards
OAuth2	Open Authorization
OIDC	OpenID Connect
OpenAPI	Open Application Programming Interface

P

PEP	Policy Enforcement Point
PDP	Policy Decision Point
PHP	Hypertext Preprocessor
PII	Personally Identifiable Information
PKCE	Proof Key for Code Exchange

R

RBAC	Role-Based Access Control
RDP	Remote Desktop Protocol
RFC	Request for Comments
REST	Representational State Transfer
ROPC	Resource Owner Password Credentials Grant
RSA	Rivest–Shamir–Adleman
RS256	RSA Signature with SHA-256

S

SaaS	Software-as-a-Service
SHA-256	Secure Hash Algorithm (256 Bit)
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SSH	Secure Shell
STI	Single Table Inheritance

T

TLS	Transport Layer Security

U

UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

V

VE	Virtual Environment
VM	Virtuelle Maschine
VNC	Virtual Network Computing

X

XACML	eXtensible Access Control Markup Language

XML	Extensible Markup Language

Y

YAML	Yet Another Markup Language

1.0

7[6] Referenzen

American National Standards Institute. (2004). Role based access control (ANSI/INCITS 359-2004).

Apache Software Foundation. (2025). Guacamole architecture. https://guacamole.apache.org/doc/gug/guacamole-architecture.html [22.08.2026]

Apache Software Foundation. (2026). Introduction to the POM. https://maven.apache.org/guides/introduction/introduction-to-the-pom.html [22.08.2026]

ArchUnit. (n.d.). ArchUnit user guide. https://www.archunit.org/userguide/html/000_Index.html [11.09.2026]

Baun, C. (2026). Masterthesis: Entwicklung und Implementierung eines Backends für den innovativen Desktop-as-a-Service DESIGN [Ausschreibung]. Frankfurt University of Applied Sciences.

Baun, C., & Bouché, J. (2023). Closing the gap between web applications and desktop applications by designing a novel desktop-as-a-service (DaaS) with seamless support for desktop applications. Open Journal of Cloud Computing, 8(1), 1–19.

Baun, C., & Bouché, J. (2024). Challenges and solutions of developing and implementing a novel desktop-as-a-service. In Proceedings of the 20th International Conference on Mobile Web and Intelligent Information Systems (MobiWis 2024). Springer.

Connect2id Ltd. (2026). Nimbus JOSE + JWT. https://connect2id.com/products/nimbus-jose-jwt [10.10.2026]

Docker, Inc. (2023). Docker overview. https://docs.docker.com/get-started/overview/ [22.08.2026]

Eclipse Foundation. (2022). Jakarta Persistence specification (Version 3.1).

Ferraiolo, D. F., Sandhu, R., Gavrila, S., Kuhn, D. R., & Chandramouli, R. (2001). Proposed NIST standard for role-based access control. ACM Transactions on Information and System Security, 4(3), 224–274.

Hardt, D. (Ed.). (2012). The OAuth 2.0 authorization framework (RFC 6749). Internet Engineering Task Force. https://doi.org/10.17487/RFC6749

Hibernate. (2026). Hibernate ORM user guide. https://hibernate.org/orm/documentation/ [25.08.2026]

Hu, V. C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., & Scarfone, K. (2014). Guide to attribute based access control (ABAC) definition and considerations (NIST Special Publication 800-162). National Institute of Standards and Technology.

Jones, M. (2015a). JSON Web Algorithms (JWA) (RFC 7518). Internet Engineering Task Force. https://doi.org/10.17487/RFC7518

Jones, M. (2015b). JSON Web Key (JWK) (RFC 7517). Internet Engineering Task Force. https://doi.org/10.17487/RFC7517

Jones, M., Bradley, J., & Sakimura, N. (2015a). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Jones, M., Bradley, J., & Sakimura, N. (2015b). JSON Web Signature (JWS) (RFC 7515). Internet Engineering Task Force. https://doi.org/10.17487/RFC7515

Jones, M., & Hardt, D. (2012). The OAuth 2.0 authorization framework: Bearer token usage (RFC 6750). Internet Engineering Task Force. https://doi.org/10.17487/RFC6750

JUnit Team. (2026). JUnit 5 user guide. https://junit.org/junit5/docs/current/user-guide/ [25.08.2026]

Lodderstedt, T., Bradley, J., Labunets, A., & Fett, D. (2025). Best current practice for OAuth 2.0 security (BCP 240, RFC 9700). Internet Engineering Task Force. https://doi.org/10.17487/RFC9700

MariaDB Foundation. (2026). MariaDB Server documentation. https://mariadb.com/kb/en/documentation/ [25.08.2026]

Mockito contributors. (2026). Mockito documentation. https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html [25.08.2026]

Mountainminds GmbH & Co. KG. (2026). JaCoCo Java code coverage library documentation. https://www.jacoco.org/jacoco/trunk/doc/ [02.08.2026]

OASIS. (2013). eXtensible Access Control Markup Language (XACML) Version 3.0.

OpenID Foundation. (2023). OpenID Connect Core 1.0. https://openid.net/specs/openid-connect-core-1_0.html [22.08.2026]

OWASP Foundation. (2024). OWASP proactive controls: C7—Enforce access controls. https://owasp.org/www-project-proactive-controls/ [20.10.2026]

Popek, G. J., & Goldberg, R. P. (1974). Formal requirements for virtualizable third generation architectures. Communications of the ACM, 17(7), 412–421.

Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. In Proceedings of the USENIX Annual Technical Conference.

Redgate Software. (2026). Flyway documentation. https://documentation.red-gate.com/flyway [01.10.2026]

Rescorla, E. (2018). The Transport Layer Security (TLS) protocol version 1.3 (RFC 8446). Internet Engineering Task Force. https://doi.org/10.17487/RFC8446

Richer, J., & Sanso, A. (2017). OAuth 2 in action. Manning Publications.

Sakimura, N., Bradley, J., & Agarwal, N. (2015). Proof key for code exchange by OAuth public clients (RFC 7636). Internet Engineering Task Force. https://doi.org/10.17487/RFC7636

Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. Proceedings of the IEEE, 63(9), 1278–1308.

Samarati, P., & De Capitani di Vimercati, S. (2001). Access control: Policies, models, and mechanisms. In Foundations of security analysis and design (Lecture Notes in Computer Science, Vol. 2171, pp. 137–196). Springer.

Spilca, L. (2024). Spring Security in action (2nd ed.). Manning Publications.

The Coding Gopher. (2026). Containers vs. virtual machines: Understanding modern virtualization and containerization. Substack. https://thecodinggopher.substack.com/p/containers-vs-virtual-machines [24.10.2026]

VMware. (2026a). Spring Security reference documentation. https://docs.spring.io/spring-security/reference/ [13.08.2026]

VMware. (2026b). Spring Authorization Server reference documentation. https://docs.spring.io/spring-authorization-server/reference/ [13.08.2026]

VMware. (2026c). Spring Boot reference documentation. https://docs.spring.io/spring-boot/ [13.10.2026]

VMware. (2026d). Spring Framework reference documentation. https://docs.spring.io/spring-framework/reference/ [07.11.2026]

8[7] Anhang

Tabelle A.1 listet alle Endpunkte des neuen Backends in Version 2.0.0 auf, wie sie sich aus den Controller-Klassen ergeben. „admin“ bedeutet, dass ein Token mit dem Scope admin erforderlich ist; „angemeldet“ bedeutet, dass ein beliebiges gültiges Token genügt.

Tabelle A.1: Endpunkte des neuen Backends (Version 2.0.0)

Endpunkt	Zugriff	Zweck
POST /oauth2/user/token	öffentlich	Token per Password oder Refresh Grant
POST /oauth2/user/logout	angemeldet	Widerruf des aktuellen Tokens
GET /oauth2/user/session	angemeldet	Session-Informationen (ID, Name, Rolle)
POST /user	öffentlich	Selbstregistrierung
POST /user/validate_email	öffentlich	E-Mail-Verifikation
POST /user/resend_code	öffentlich	neuen Registrierungscode anfordern
GET /users	admin	Nutzer auflisten
PATCH /user/{userId}	admin	Nutzerdaten ändern
PATCH /user/{userId}/enable	admin	Konto aktivieren
PATCH /user/{userId}/disable	admin	Konto deaktivieren
POST /user/request-application/{application}/{isoCode}	angemeldet	Anwendungsfreischaltung anfragen
GET /admins	admin	Administratoren auflisten
POST /admin	admin	Administrator anlegen
GET /user_groups	admin	Nutzergruppen auflisten
POST /user_group	admin	Nutzergruppe anlegen

PATCH /user_group/{id}	admin	Nutzergruppe ändern
GET /desktops	admin	Desktops auflisten
GET /desktop/{id}	admin	Desktop lesen
POST /desktop	admin	Desktop anlegen (optional mit Untergruppen)
GET /desktop_groups	admin	Desktop-Gruppen auflisten
POST /desktop_group	admin	Desktop-Gruppe anlegen
POST /desktop_group/{id}/user_group/{userGroupId}	admin	Nutzergruppe mit Desktop-Gruppe verknüpfen
GET /permissions/{functionName}/{userId}	admin	Berechtigung eines Nutzers prüfen
POST /permissions_info	angemeldet	Berechtigung des aufrufenden Nutzers prüfen
GET /actuator/health	öffentlich	Health-Check